A project report on

VERIDEX: LLM-DRIVEN ADVERSARIAL EVALUATION OF MANIPULATION DETECTION IN SIMULATED COMMODITY FUTURES MARKETS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in PROGRAM NAME WITH SPECIALIZATION

by

YOUR NAME (REG. NO.)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

VERIDEX: LLM-DRIVEN ADVERSARIAL EVALUATION OF MANIPULATION DETECTION IN SIMULATED COMMODITY FUTURES MARKETS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in PROGRAM NAME WITH SPECIALIZATION

by

YOUR NAME (REG. NO.)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

DECLARATION

I hereby declare that the thesis entitled "Veridex: LLM-Driven Adversarial Evaluation of Manipulation Detection in Simulated Commodity Futures Markets" submitted by YOUR NAME (REG. NO.), for the award of the degree of Bachelor of Technology in PROGRAM NAME WITH SPECIALIZATION, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of GUIDE NAME.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date:

Signature of the Candidate

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "Veridex: LLM-Driven Adversarial Evaluation of Manipulation Detection in Simulated Commodity Futures Markets" is prepared and submitted by YOUR NAME (REG. NO.) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in [[PROGRAM NAME WITH SPECIALIZATION]]** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:
Name: Dr./Prof. GUIDE NAME
Date:

Signature of the Examiner	Signature of the Examiner
Name:	Name:
Date:	Date:

Approved by the Head of Department,
PROGRAM NAME WITH SPECIALIZATION

Name: HOD NAME
Date:

(Seal of the School of Computer Science and Engineering)

ABSTRACT

Spoofing, layering, wash trading and pump-and-dump schemes distort commodity futures prices. Surveillance systems are usually evaluated on historical cases, which says little about how they hold up once manipulators adapt. Veridex is a fully simulated surveillance laboratory built to study that question.

Veridex has five modules. An agent-based limit order book simulator, calibrated to eight commodity futures, generates realistic markets with noise, momentum, mean-reversion and market-making traders, high-frequency liquidity providers and institutional execution algorithms, together with four manipulation personas whose ground truth is logged exactly. A two-layer detector computes 51 trader-level features over rolling three-minute windows and combines an unsupervised Isolation Forest with four persona-specific XGBoost classifiers, explaining every alert with SHAP values. An adversarial evaluation loop then lets a large language model strategist (Cerebras gpt-oss-120b) re-tune the manipulators each round, scores the mutated schemes against the frozen detector before retraining, and penalises any mutation that stops manipulating. An LLM layer also writes grounded case notes and answers analyst questions, and a Streamlit dashboard presents the results.

In a six-round experiment over eight commodities (about 12.8 million order events and 2.6 million trades per round), the baseline detector reached F1 scores of 0.98 for spoofing, 0.97 for wash trading, 0.99 for layering and 0.72 for pump-and-dump. The LLM strategist's mutations reduced the frozen detector's coverage to 0.14 for pump-and-dump and 0.62 for wash trading in round 1, and to 0.59 for spoofing and 0.69 for layering in round 2, while keeping at least 62 percent of each scheme's original market effect. Cumulative retraining recovered most of the loss, with post-retrain F1 scores between 0.71 and 0.87 in the final round. The run cost about USD 0.07 in API credit. Adversarial, LLM-driven evaluation thus exposes weaknesses that static evaluation hides, which regular retraining largely repairs.

Keywords: market manipulation, spoofing, agent-based simulation, limit order book, anomaly detection, XGBoost, SHAP, adversarial machine learning, large language model agents.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to GUIDE NAME, DESIGNATION, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his/her constant guidance, continual encouragement, understanding; more than all, he/she taught me patience in my endeavor. My association with him/her is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of machine learning and financial market surveillance.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to HOD NAME, Head of the Department, B.Tech. PROGRAM NAME WITH SPECIALIZATION and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date:

YOUR NAME

CONTENTS

ABSTRACT ..... **i**
ACKNOWLEDGEMENT ..... **ii**
LIST OF FIGURES ..... **vi**
LIST OF TABLES ..... **viii**
LIST OF ACRONYMS ..... **ix**
Chapter 1 Introduction **1**
1.1 Background 1
1.2 Motivation 2
1.3 Problem Statement 2
1.4 Objectives 3
1.5 Scope of the Project 4
1.6 Contributions 4
1.7 Organisation of the Report 5
Chapter 2 Literature Survey **6**
2.1 Market Manipulation in Futures Markets 6
2.2 Regulation and Enforcement Cases 7
2.3 Agent-Based Simulation of Financial Markets 7
2.4 Approaches to Manipulation Detection 8
2.5 Explainable Machine Learning 9
2.6 Adversarial Machine Learning and Red Teaming 9
2.7 Large Language Model Agents and Tool Use 10
2.8 Research Gap 10
Chapter 3 Requirements and Analysis **11**
3.1 Stakeholders and Users 11
3.2 Functional Requirements 11
3.3 Non-Functional Requirements 12
3.4 Use Cases 12
3.5 Hardware and Software Environment 13
3.6 Constraints and Assumptions 13
Chapter 4 System Design **14**
4.1 Architectural Overview 14
4.2 Module Decomposition 16
4.3 Data Design 16
4.4 Adversarial Round Design 17
4.5 Configuration Design 18
4.6 Dashboard Design 19
4.7 Key Design Decisions 19
Chapter 5 Implementation: Agent-Based Market Simulator **21**

5.1 Limit Order Book and Matching Engine 21
5.2 Fundamental Value and Calibration 22
5.3 Legitimate Market Participants 22
5.4 Simulation Loop 23
5.5 Manipulation Personas 24
5.6 Behavioural Profiles of Legitimate and Manipulative Accounts 28
5.7 Realism Checks 28
Chapter 6 Implementation: Detection Engine and Adversarial Loop 31
6.1 Unit of Detection 31
6.2 Feature Engineering 32
6.3 Ground-Truth Labels 33
6.4 Unsupervised Layer: Isolation Forests 33
6.5 Supervised Layer: Per-Persona XGBoost Classifiers 33
6.6 Ensemble, Persona Assignment and SHAP Explanations 34
6.7 Evaluation Metrics 34
6.8 Adversarial Round Orchestration 35
6.9 Manipulation Effectiveness and the Adversary's Objective 35
6.10 The Effect Guard 36
Chapter 7 Implementation: LLM Layer and Dashboard 38
7.1 LLM Client, Provider Switch and Cost Control 39
7.2 Adversarial Strategist 39
7.3 Case-Note Generator 41
7.4 Analyst Assistant 41
7.5 Dashboard 41
Chapter 8 Results and Discussion 57
8.1 Experimental Setup 57
8.2 Baseline Detection (Round 0) 58
8.3 Evasion: Frozen-Detector Results 58
8.4 Recovery: Post-Retrain Performance 60
8.5 Manipulation Effectiveness and Mutation Scores 61
8.6 What the Strategist Did 63
8.7 What the Detector Relies On 65
8.8 False Alerts on Legitimate Look-Alike Traders 65
8.9 Unsupervised Layer and Detection Delay 67
8.10 LLM Usage and Cost 67
8.11 Software Testing 69
8.12 Discussion 70
Chapter 9 Conclusion and Future Work 71
9.1 Conclusion 71
9.2 Limitations 72
9.3 Future Work 72

REFERENCES **74**
APPENDICES **77**
Appendix 1 Running Veridex 77
Appendix 2 Detector Configuration 77
Appendix 3 LLM Configuration 78
Appendix 4 Sample Strategist Exchange 79
Appendix 5 Tactic Cards (Spoofing) 81
Appendix 6 Repository Structure 82

LIST OF FIGURES

Figure 4.1: Module-wise architecture of Veridex (rendered from a Mermaid diagram) **15**

Figure 4.2: Control flow of the adversarial evaluation loop **18**

Figure 4.3: Data flow of the Streamlit dashboard **19**

Figure 5.1: Work performed by the simulator in each tick **24**

Figure 5.2: Phases of the pump-and-dump persona **27**

Figure 5.3: Normalised price paths of the eight commodities (dashboard, round 5 test market) **29**

Figure 5.4: Cross-asset return correlation matrix (dashboard, round 5 test market) **30**

Figure 6.1: Detection pipeline from logs to explained flags **31**

Figure 7.1: LLM client: provider switch, cache, budget ledger and error handling **38**

Figure 7.2: Dashboard with the sidebar (run, round and test-market selection, round summary) **42**

Figure 7.3: Market Overview: headline metrics and cross-asset snapshot (round 5) **43**

Figure 7.4: Market Overview: simultaneous price profiles of the eight commodities **43**

Figure 7.5: Market Overview: trading volume by commodity **44**

Figure 7.6: Market Overview: flagged windows by persona and commodity **44**

Figure 7.7: Market Overview for WTI crude oil: headline metrics and active personas **45**

Figure 7.8: OHLCV chart of WTI crude oil with manipulation episodes shaded **45**

Figure 7.9: Microstructure series for WTI crude oil **46**

Figure 7.10: Activity heatmap of the most active WTI accounts (30-minute buckets) **46**

Figure 7.11: Flagged Trades: case counts, filters and the first case (round 5, test market 1) **48**

Figure 7.12: A layering case: detector belief, confidence, price, SHAP and evidence **49**

Figure 7.13: The same case with a verified AI case note **50**

Figure 7.14: Raw orders and trades of the flagged account in the case span **51**

Figure 7.15: Model Performance: headline metrics and the Evasion vs. Recovery panel **52**

Figure 7.16: Recall of the frozen (dashed) and retrained (solid) detector by round 52
Figure 7.17: F1 heatmap, persona by round (post-retrain) 53
Figure 7.18: Average precision, recall and F1: round 0 against round 5 53
Figure 7.19: Metric evolution: post-retrain F1 per persona across rounds 53
Figure 7.20: Precision–recall trade-off of every persona and round (marker size = support)..... 54
Figure 7.21: Full metrics table (excerpt)..... 54
Figure 7.22: Adversarial parameter evolution table (excerpt) 55
Figure 7.23: Strategist decision cards for round 5 55
Figure 7.24: A grounded answer with its sources..... 56
Figure 8.1: Any-alert coverage of the frozen detector (left) and the retrained detector (right)..... 59
Figure 8.2: Own-label recall before (grey) and after (colour) retraining, by persona and round..... 60
Figure 8.3: Post-retrain precision, recall and F1 by persona and round..... 61
Figure 8.4: Effectiveness ratio of each persona relative to round 0..... 62
Figure 8.5: Mutation score (the strategist's objective) per persona and round .. 62
Figure 8.6: Strategist decision cards for round 2 (dashboard) 64
Figure 8.7: Mean absolute SHAP values of the ten most important features of each classifier, round 5 65
Figure 8.8: False-alert rate on legitimate accounts by trader type 66
Figure 8.9: Trader Isolation Forest against any manipulation (left); frozen-detector detection delay (right) 67
Figure 8.10: Tokens per LLM call in the run (strategist calls, case notes and chat questions) 68
Figure 8.11: An analyst-chat answer that cites real numbers but mislabels them as frozen-detector recall 69

LIST OF TABLES

Table 2.1: Enforcement cases used to design the manipulation personas 7
Table 2.2: Comparison of manipulation detection approaches 9
Table 3.1: Functional requirements and implementation status 11
Table 3.2: Key non-functional requirements 12
Table 3.3: Principal use cases 13
Table 3.4: Development and execution environment 13
Table 4.1: Modules and their source packages 16
Table 4.2: Core data schemas 16
Table 4.3: Design decisions and their rationale 19
Table 5.1: Calibration constants per commodity 22
Table 5.2: Legitimate agent population per market 22
Table 5.3: Spoofing knobs (baseline value and strategist bounds) 25
Table 5.4: Layering knobs (baseline value and strategist bounds) 26
Table 5.5: Wash trading knobs (baseline value and strategist bounds) 26
Table 5.6: Pump-and-dump knobs (baseline value and strategist bounds) 27
Table 5.7: Median account-window feature values by account type (crude oil, development measurement) 28
Table 5.8: Stylized facts of 60-tick returns, round 0 test market 29
Table 6.1: Feature catalogue 32
Table 6.2: XGBoost hyperparameters 34
Table 6.3: Primary effectiveness measures 35
Table 7.1: Cost and reliability safeguards in the LLM client 39
Table 8.1: Experimental setup 57
Table 8.2: Data volume per round (training and test markets combined) 57
Table 8.3: Round 0 baseline detection (post-retrain, both test markets) 58
Table 8.4: Frozen (pre-retrain) detection of the mutated personas 58
Table 8.5: Post-retrain F1 by persona and round 60
Table 8.6: Effectiveness ratio and mutation score by round 61
Table 8.7: Parameter changes proposed in each round 63
Table 8.8: Share of account-windows flagged, by trader type and round (post-retrain detector) 65
Table 8.9: Strategist LLM usage in the experiment 67
Table 8.10: Automated test suite 69

LIST OF ACRONYMS

ABM	Agent-Based Model
API	Application Programming Interface
AUC	Area Under the Curve
CFTC	Commodity Futures Trading Commission (United States)
CI	Confidence Interval
CV	Cross-Validation / Coefficient of Variation
DOJ	Department of Justice (United States)
EMA	Exponential Moving Average
F1	Harmonic mean of precision and recall
FPR	False Positive Rate
HFT	High-Frequency Trading
IF	Isolation Forest
LLM	Large Language Model
LOB	Limit Order Book
MAR	Market Abuse Regulation (European Union)
ML	Machine Learning
OHLCV	Open, High, Low, Close, Volume
OOF	Out-Of-Fold
OTR	Order-to-Trade Ratio
OU	Ornstein–Uhlenbeck (mean-reverting stochastic process)
P&L	Profit and Loss
SHAP	SHapley Additive exPlanations
SRS	Software Requirements Specification
TTL	Time To Live (of a resting order)
UI	User Interface
XGBoost	eXtreme Gradient Boosting

Chapter 1
Introduction

1.1 BACKGROUND

Commodity futures markets set the reference prices for crude oil, natural gas, metals and grains. Refiners, airlines, miners, millers and farmers use these prices to plan production and hedge risk, and the same contracts are traded by banks, proprietary trading firms, asset managers and retail participants. Almost all of this trading now happens on electronic exchanges that match orders in a central limit order book (LOB). The book is visible to every participant: anyone can see how much is offered to buy and to sell at each price level, and most trading algorithms react to that information within milliseconds.

That visibility is also what makes manipulation possible. A trader who places large orders that they never intend to execute can make the book look heavier on one side, so other participants move their prices; the manipulator then trades at the improved price and cancels the fake orders. This is *spoofing*. Placing several such orders at consecutive price levels is *layering*. Trading with oneself or with colluding accounts creates the appearance of volume without any change in ownership, which is *wash trading*. Buying aggressively to trigger momentum traders and then selling into the demand that follows is a *pump-and-dump* or momentum-ignition scheme. All four are prohibited: in the United States, Section 747 of the Dodd-Frank Wall Street Reform and Consumer Protection Act (2010) made spoofing explicitly illegal, and the European Union's Market Abuse Regulation (Regulation (EU) No 596/2014, 2014) prohibits giving false or misleading signals about supply, demand or price.

Enforcement cases show that these schemes are not hypothetical. Navinder Sarao's spoofing of E-mini S&P 500 futures was linked by regulators to the 2010 “flash crash”; Michael Coscia became the first trader criminally convicted under the Dodd-Frank anti-spoofing provision; JPMorgan Chase agreed in 2020 to pay about USD 920 million to resolve spoofing in precious-metals and Treasury futures; and Tower Research Capital paid USD 67.4 million in 2019 for layering by its traders. Each of these schemes ran for months or years before it was stopped, which raises an obvious question for anyone building surveillance software: how well does a detector keep working once the people it is meant to catch start adapting to it?

1.2 MOTIVATION

Most published work on manipulation detection trains a model on labelled historical data, or on a fixed synthetic data set, and reports precision and recall on a held-out portion of the same data. This measures how well a detector recognises the manipulation it has already seen. It does not measure what happens when a manipulator changes tactics, for example by splitting a large spoof order into smaller pieces, randomising cancel timing, spreading activity across several accounts, or trading only during busy periods. Real manipulators did exactly these things; the enforcement records describe them in detail.

Testing a detector against adaptive manipulators is hard for three reasons. First, real labelled manipulation data is scarce, confidential and legally sensitive, and it contains only the schemes that were caught. Second, producing new evasive variants by hand is slow and limited by the imagination of the person writing them. Third, an “evasion” that simply stops manipulating (for example, a spoof so small it no longer moves the market) is not a real evasion, so any evaluation must check that the mutated scheme still works.

Two recent developments make it possible to address all three. Agent-based market simulation can generate realistic order-book data in which the ground truth is known exactly, because the simulator itself authors every manipulation episode. And large language models (LLMs) can now reason over structured feedback, read a detector's explanations and call tools, which makes them suitable for playing the role of an adaptive adversary that proposes new evasion tactics each round. Veridex combines these ideas into a closed-loop red-team environment for market surveillance.

1.3 PROBLEM STATEMENT

The problem addressed by this project can be stated as follows. Given a simulated commodity futures market with known manipulation ground truth, build a manipulation detector that identifies the manipulating accounts and explains its decisions, and then measure how its detection performance changes when an automated adversary repeatedly re-tunes the manipulation strategies to evade it, under the constraint that every mutated strategy must still achieve a meaningful fraction of its original market effect.

The detector must work at the level of individual trader accounts in short time windows, because that is how surveillance analysts investigate cases. It must distinguish manipulators from legitimate participants whose behaviour superficially resembles manipulation, such as high-frequency market makers that cancel most of their orders and institutional execution algorithms that rest and withdraw large orders. The adversary must be bounded: it can change only documented strategy parameters within validated limits, and it must not be able to "win" by doing nothing. Finally, the whole system must run on a student budget: the LLM is accessed through a paid API with about USD 5 of credit, so every call must be justified, cached and capped.

1.4 OBJECTIVES

The objectives of the project are:

1. To build a reproducible agent-based limit order book simulator, calibrated to eight commodity futures, in which legitimate traders and four manipulation personas (spoofing, layering, wash trading and pump-and-dump) interact, and every manipulation episode is logged as ground truth.
2. To design trader-level behavioural features and a two-layer detector, combining an unsupervised Isolation Forest with persona-specific supervised classifiers, that flags manipulating accounts and explains each alert with SHAP values.
3. To build an adversarial evaluation loop that measures detection both before retraining (against a frozen detector) and after retraining, together with a measure of each manipulation's market effect, so that evasion and effectiveness are judged together.
4. To use an LLM as an adaptive adversarial strategist that reads the detector's performance and explanations and proposes new manipulation parameters each round, with code-level guards that keep its proposals valid and effective.
5. To use the same LLM layer to generate grounded case notes for flagged accounts and to answer analyst questions about a run, while keeping API cost within a fixed budget.
6. To present the markets, alerts, explanations, model performance and adversarial history in an interactive dashboard suitable for analysts and examiners.

1.5 SCOPE OF THE PROJECT

Everything in Veridex is simulated. The project does not connect to any exchange, does not trade, does not use real accounts and does not produce real regulatory findings; every dashboard page carries a notice to that effect. Real market data is used only to calibrate the simulator (volatility, drift, tick size and typical volume of each commodity). The four manipulation personas are modelled on publicly documented enforcement cases. Cross-market manipulation (for example, spoofing one contract to move a related one), cornering and “banging the close” are outside the current scope. The detector is evaluated on simulated test markets that are generated independently of the training markets in every round.

1.6 CONTRIBUTIONS

The main contributions of this work are:

- **A realistic, adversarially useful market simulator.** Beyond the usual noise, momentum and mean-reversion agents, it includes high-frequency liquidity providers and institutional execution algorithms whose behaviour overlaps with manipulation, so that the detection problem is not trivially separable.
- **An evaluation protocol that judges evasion and effectiveness together.** Each mutation is scored on how much of each episode the frozen detector missed and how late it was caught, multiplied by how much of the original market effect the scheme kept. Being flagged under another persona's label still counts as being caught.
- **A bounded, cost-controlled LLM adversary** that uses tool calls, a structured track record, the detector's SHAP reliance and “tactic cards” derived from enforcement cases, protected by an automatic effect guard and a hard budget ledger.
- **An explainable analyst workflow**: SHAP explanations, a confidence panel, an evidence table, LLM case notes checked against the evidence, and a tool-grounded chat assistant, all presented in a Streamlit dashboard.

1.7 ORGANISATION OF THE REPORT

The rest of this report is organised as follows. Chapter 2 reviews the literature on market manipulation, its enforcement, market simulation, detection methods, explainability, adversarial machine learning and LLM agents, and identifies the research gap. Chapter 3 lists the requirements and the development environment. Chapter 4 presents the overall architecture and design. Chapters 5, 6 and 7 describe the implementation of each module: the market simulator; the detection engine and adversarial evaluation loop; and the LLM layer and dashboard. Chapter 8 presents and discusses the results of the six-round experiment. Chapter 9 concludes and outlines future work. References and appendices containing configuration listings, parameter bounds and sample LLM exchanges follow.

Chapter 2
Literature Survey

2.1 MARKET MANIPULATION IN FUTURES MARKETS

Economic theory distinguishes three broad kinds of manipulation (Allen & Gale, 1992): *action-based* manipulation, which changes the actual value of an asset; *information-based* manipulation, which spreads false information; and *trade-based* manipulation, in which the manipulator profits purely through trading, without any public action or false statement. All four schemes studied in this project are trade-based. Allen and Gale showed that trade-based manipulation can be profitable even with rational market participants, because uninformed traders cannot be sure whether a large buyer is informed or manipulating. Aggarwal and Wu (2006) studied enforcement cases from the U.S. Securities and Exchange Commission and found that manipulation is more common in less liquid markets and that manipulators tend to trade during periods of elevated volume, a tactic modelled in Veridex as the "busy period" knob. Putniņš (2012) surveyed the field and noted that empirical work is limited by the fact that only detected manipulation is ever observed.

Spoofing has received particular attention since high-frequency trading became dominant. Lee, Eom and Park (2013) analysed spoofing traders in the Korean stock market and showed that they place large orders away from the best price and cancel them quickly, and that the strategy is profitable. Cartea, Jaimungal and Wang (2020) modelled spoofing in an order-driven market and showed that because other traders use order-book imbalance to predict short-term price moves, a trader who can distort imbalance with orders that are unlikely to execute can profit. This mechanism (fake depth shifts the imbalance, the imbalance shifts other participants' quotes) is exactly how spoofing and layering are implemented in the Veridex simulator. Wash trading has been studied extensively in cryptocurrency exchanges; Cong, Li, Tang and Yang (2023) estimated that a large share of reported volume on unregulated exchanges was wash traded. Pump-and-dump schemes in thin markets were characterised by Kamps and Kleinberg (2018), who proposed detection rules based on sudden price and volume increases.

2.2 REGULATION AND ENFORCEMENT CASES

Regulators enforce anti-manipulation rules through surveillance and investigation. Table 2.1 summarises the enforcement cases that shaped the four Veridex personas and the evasion tactics available to the LLM strategist (Commodity Futures Trading Commission [CFTC], 2015, 2019, 2020, 2021; CFTC v. Kraft Foods Group, 2015; CFTC v. Oystacher, 2015; United States v. Coscia, 2017). The persona parameters and tactic cards cite these cases directly, so that every evasion the strategist can try corresponds to behaviour that real manipulators actually used.

Table 2.1: Enforcement cases used to design the manipulation personas

Case	Scheme	Documented tactics	Use in Veridex
Sarao / Nav Sarao Futures (CFTC, 2015)	Spoofing / layering in E-mini S&P 500 futures	Large sell orders kept a few levels from the best price and continually re-priced as the market moved	Spoofing knob `reprice_to_touch`
Coscia (U.S. v. Coscia, convicted 2015, affirmed 2017)	Spoofing in energy and metals futures	Large orders cancelled within fractions of a second after a small opposite order filled; later split into batches	Baseline spoofing cycle; split orders; cancel-timing jitter
Oystacher / 3Red Trading (CFTC, 2015)	Spoofing with "flipping"	After pulling spoof orders, trading aggressively in the other direction	Spoofing knob `flip_after_cancel`
JPMorgan Chase (CFTC/DOJ, 2020, about USD 920 million)	Spoofing in precious metals and Treasuries	Genuine and spoof orders entered by different traders on the same desk	Spoofing knob `n_accounts` (cross-account legs)
Tower Research Capital (CFTC/DOJ, 2019, USD 67.4 million)	Layering	Several orders on consecutive price levels, cancelled together after the opposite order filled	Layering persona: layer count, spacing, size taper
Kraft Foods / Mondelez (CFTC, complaint 2015, settled 2019)	Wheat futures manipulation including wash trades	Related accounts trading with each other while staying flat	Wash trading persona; camouflage trades
Coinbase (CFTC, 2021, USD 6.5 million)	Wash trading by automated programs	Two internal programs repeatedly matching each other's orders	Wash rings across several accounts

Regulators have also published the patterns they look for. Staff guidance from the U.S. Commodity Futures Trading Commission (CFTC) highlights large order sizes relative to the market, short order lifetimes, high cancellation rates, orders on the opposite side of executions, and activity concentrated around busy periods. These observations are encoded both in the detector's features (Chapter 6) and, from the attacker's side, in the tactic cards given to the LLM strategist (Chapter 7).

2.3 AGENT-BASED SIMULATION OF FINANCIAL MARKETS

Agent-based models (ABMs) simulate a market as a population of interacting traders with simple behavioural rules. Early models such as the Santa Fe artificial stock market

showed that realistic price dynamics can emerge from heterogeneous agents. Chiarella and Iori (2002) built an order-driven market with fundamentalist, chartist and noise traders and showed that the mix of strategies reproduces fat-tailed returns and volatility clustering. Gould et al. (2013) reviewed the empirical properties and models of limit order books, including price-time priority matching, which Veridex implements exactly. Byrd, Hybinette and Balch (2020) introduced ABIDES, a discrete-event multi-agent simulator for market research. Avellaneda and Stoikov (2008) derived optimal quoting for a market maker that skews its quotes according to its inventory; the Veridex market maker follows this idea.

A credible simulator must reproduce the *stylized facts* of real returns (Cont, 2001): heavy tails (positive excess kurtosis), volatility clustering (positive autocorrelation of absolute returns) and little linear predictability of returns themselves. Veridex checks these properties for every commodity (Section 5.7). The fundamental value of each commodity follows an Ornstein–Uhlenbeck mean-reverting process (Uhlenbeck & Ornstein, 1930), a standard model for commodity prices because of storage and supply responses (Deaton & Laroque, 1996; Geman, 2005), with parameters calibrated from published historical statistics (Erb & Harvey, 2006).

2.4 APPROACHES TO MANIPULATION DETECTION

Detection approaches fall into three groups. *Rule-based surveillance*, still common in industry, raises an alert when a hand-set threshold is crossed, for example an order-to-trade ratio above a limit. Rules are transparent but easy to evade once the thresholds are known, and they generate many false alarms on legitimate high-frequency traders. *Statistical and unsupervised methods* model normal behaviour and flag deviations; they need no labels, which suits a domain where labels are scarce. The Isolation Forest (Liu, Ting & Zhou, 2008) isolates anomalies with random partitioning trees: anomalous points are separated in fewer splits, so the average path length gives an anomaly score. It scales well and works with mixed feature types. *Supervised methods* learn from labelled examples and are usually more accurate on known schemes. Gradient-boosted decision trees, and XGBoost in particular (Chen & Guestrin, 2016), perform strongly on tabular data, handle class imbalance through instance weighting and are efficient enough to retrain every round.

Table 2.2: Comparison of manipulation detection approaches

Approach	Strengths	Weaknesses	Role in Veridex
Threshold rules	Transparent, simple, auditable	Easy to evade; many false positives on HFT	Basis for intuition, not used directly
Unsupervised anomaly detection (Isolation Forest)	No labels needed; catches unknown schemes	Flags any unusual account, including legitimate outliers	Trader-level and market-level forests
Supervised classification (XGBoost)	High accuracy on known schemes; fast to retrain	Needs labels; may miss novel variants	One classifier per persona
Graph / network analysis	Natural fit for wash rings and collusion	Needs counterparty graphs; costly	Approximated by counterparty features
Adversarial evaluation (this work)	Measures robustness to adaptive manipulators	Needs a simulator and an adversary	Core of Modules 3 and 4

2.5 EXPLAINABLE MACHINE LEARNING

A surveillance alert is only useful if an analyst can understand why it was raised, and regulators expect firms to be able to justify their alerts. SHAP (SHapley Additive exPlanations) attributes a model's prediction to its input features using Shapley values from cooperative game theory (Lundberg & Lee, 2017). For tree ensembles, TreeSHAP computes exact Shapley values in polynomial time (Lundberg et al., 2020). Veridex stores the five most important SHAP contributions with every alert, shows them in the dashboard, and passes the detector's mean absolute SHAP values to the adversarial strategist so it knows which features the detector currently relies on.

2.6 ADVERSARIAL MACHINE LEARNING AND RED TEAMING

Adversarial machine learning studies how models fail under deliberate manipulation of their inputs. Goodfellow, Shlens and Szegedy (2015) showed that small, targeted perturbations can fool neural networks, and Biggio and Roli (2018) reviewed a decade of evasion and poisoning attacks, emphasising that security evaluation should assume an adaptive attacker who knows how the defence works. In fraud and surveillance settings the attacker is constrained: an evasion must still achieve the attacker's goal, so the realistic threat model is “evade while still manipulating”. This project encodes that constraint directly in the adversary's objective (Section 6.9). The loop of attack, measurement and retraining also resembles red-team exercises, in which a dedicated team tries to defeat a system so that its weaknesses can be fixed before real attackers find them.

2.7 LARGE LANGUAGE MODEL AGENTS AND TOOL USE

Large language models can follow instructions, reason over structured context and call external functions. ReAct (Yao et al., 2023) interleaves reasoning with actions, and Toolformer (Schick et al., 2023) showed that models can learn when to call tools. Agentic use of LLMs, in which the model selects and sequences tool calls to complete a bounded task, is now common. Open-weight models such as gpt-oss-120b (OpenAI, 2025) can be served at high speed by inference providers such as Cerebras, which makes them practical for short, structured decisions. Veridex uses an LLM in three bounded roles: an adversarial strategist that calls a single validated `propose_mutation` tool, a narrative generator whose output is checked against the evidence it was given, and an analyst assistant that answers only from data returned by read-only query tools.

2.8 RESEARCH GAP

The literature provides the individual building blocks: realistic order-book simulation, unsupervised and supervised detection, explanation methods, adversarial evaluation and LLM agents. What is missing is an end-to-end, reproducible environment that puts them together to answer a practical question: *does a manipulation detector stay effective when manipulators adapt to it, and what does an adaptive manipulator learn to do?* Existing detection studies evaluate on static data; existing simulators are rarely paired with explainable detectors; and adversarial studies rarely enforce that an evasion must still manipulate. Veridex addresses this gap with a closed loop in which a bounded LLM adversary mutates documented manipulation tactics, a detector is measured before and after retraining, and every mutation's market effect is checked against the baseline.

Chapter 3
Requirements and Analysis

This chapter summarises what Veridex must do and the conditions it must satisfy. The requirements were written before implementation in a Software Requirements Specification (SRS) with stable identifiers (FR-x.y for functional requirements, NFR-x for non-functional ones), and the implementation was checked against them. The status column reports honestly what the final system delivers.

3.1 STAKEHOLDERS AND USERS

Three kinds of user were considered. A *surveillance analyst* reviews flagged accounts, reads explanations and asks questions about a run. A *researcher* configures simulations, runs the adversarial loop and studies how detection changes across rounds. An *examiner* (for example, a project evaluator) needs to understand each screen quickly without verbal explanation. All three use the same dashboard; researchers additionally use the command-line pipeline.

3.2 FUNCTIONAL REQUIREMENTS

Table 3.1: Functional requirements and implementation status

ID	Requirement (summary)	Status
FR-0.1	Modules communicate only through schema-validated files on disk (Parquet / YAML / JSON)	Implemented
FR-0.2	Every data file is validated against its schema before it is written	Implemented (pandera)
FR-0.3	Each run has a unique run_id directory with configuration, seed and timestamp	Implemented
FR-0.4	Same seed and configuration give identical output logs	Implemented; tested
FR-1.1	Limit order book with price-time priority, no learned component in price formation	Implemented
FR-1.2	At least four legitimate agent archetypes sharing one decide() interface	Implemented (six types)
FR-1.3	At least three manipulation personas trading in the same book, independently tunable	Implemented (four)
FR-1.4	Layering (should) and cornering (could) as extensions	Layering implemented; cornering not
FR-1.5	Every trade and every order lifecycle event logged	Implemented
FR-1.6	Ground-truth manipulation windows logged with accounts and parameters	Implemented
FR-1.7	Trades resampled to OHLCV bars	Implemented
FR-1.8	Calibration targets estimated from real commodity data, with fallback constants	Implemented
FR-1.9	Replay from any tick with a modified population (counterfactual)	Not implemented
FR-2.1	Rolling-window, per-trader behavioural features	Implemented (51 features)
FR-2.2	Unsupervised anomaly model trained without labels	Implemented (Isolation Forest)

ID	Requirement (summary)	Status
FR-2.3	One supervised classifier per persona	Implemented (XGBoost)
FR-2.4	SHAP attributions for supervised predictions	Implemented
FR-2.5	Single ensemble flag per (window, trader) with triggering layers and top features	Implemented
FR-2.6	Precision, recall, F1 and FPR reported per persona	Implemented
FR-2.7	Detection latency per detected episode	Implemented
FR-3.1	Round 0 baseline with obvious personas	Implemented
FR-3.2	Each round: new data with mutated personas, pre-retrain and post-retrain evaluation	Implemented
FR-3.3	Hand-authored mutation library grounded in enforcement cases	Implemented (fallback)
FR-3.4	SHAP feature-importance drift between rounds	Implemented
FR-3.5	Price impact avoided by early detection (counterfactual replay)	Not implemented
FR-4.1	Case narratives derived only from computed evidence	Implemented with grounding check
FR-4.2	Bounded tool-using agent that proposes next-round mutations	Implemented
FR-4.3	Analyst assistant answering only from logged data through function calls	Implemented
FR-4.4	Graceful degradation when the LLM is unavailable	Implemented
FR-4.5	Cached LLM outputs so a demonstration can run without new API calls	Implemented (response cache)
FR-4.6	Every LLM call logged with tokens, latency and cost	Implemented
FR-5.1 to 5.6	Dashboard: market charts with manipulation shading, performance curves, case review, chat, run selection	Implemented (5.5 omitted)

3.3 NON-FUNCTIONAL REQUIREMENTS

Table 3.2: Key non-functional requirements

Category	Requirement	How it is met
Performance	A 10,000-tick market with the full population completes in minutes on a laptop, without a GPU	About 15–20 s per commodity; the 8 commodities run in parallel processes
Reliability	An LLM failure never breaks the simulation or the detector	Fallback mutation library; LLM features are disabled with an explanation
Cost	Total LLM spending stays within about USD 5 of API credit	Hard budget ledger capped at USD 4.50, response cache, call spacing, per-process call cap
Security	API keys never committed; prompts separate instructions from data	.env is git-ignored; keys stripped from logs; tool output treated as data
Reproducibility	Identical seed and configuration give identical output	Seeded generators per agent; deterministic order ids; reproducibility test
Maintainability	Edge cases covered by automated tests; schemas defined once	118 automated tests; all schemas in src/utils/schemas.py
Usability	A first-time viewer understands each screen quickly	Titles, captions, hover help, colour plus icon for each persona
Ethics	Never presented as a real surveillance product; no real trader data	Synthetic ids only; “simulation output” notice on every page

3.4 USE CASES

Five use cases drove the design. They are summarised below; Chapter 7 shows the screens that implement them.

Table 3.3: Principal use cases

ID	Use case	Actor	Main flow
UC-1	Run a full adversarial experiment	Researcher	Start the pipeline with rounds, ticks and seed; the system simulates, trains, scores and mutates round by round
UC-2	Review a flagged case	Analyst	Select a round and market, open a flagged account, read confidence, SHAP and evidence, generate a case note
UC-3	Ask a question about a run	Analyst	Type a question in the chat; the assistant calls query tools and answers with cited sources
UC-4	Propose the next mutation	LLM strategist	Read track record, SHAP reliance and tactic cards; call propose_mutation for each persona
UC-5	Inspect round-by-round performance	Examiner	Open Model Performance; compare frozen and retrained detection, effect and strategist decisions

3.5 HARDWARE AND SOFTWARE ENVIRONMENT

Table 3.4: Development and execution environment

Item	Specification
Operating system	Windows 11 Pro (64-bit); the code is platform-independent Python
Processor / memory	Standard multi-core laptop CPU; no GPU required
Language	Python 3.14 in a project virtual environment
Data and numerics	pandas 2.3, NumPy 2.5, SciPy 1.17, PyArrow 25 (Parquet)
Validation	pandera 0.33 (DataFrame schemas)
Machine learning	scikit-learn 1.9 (Isolation Forest, GroupKFold), XGBoost 3.4, SHAP 0.49
LLM access	LiteLLM 1.100; Cerebras gpt-oss-120b (primary) or Google Gemini (alternative)
Dashboard	Streamlit 1.63, Plotly 7.0
Testing	pytest 9.1, Hypothesis (property-based tests)
Configuration	YAML files under configs/; secrets in a git-ignored .env file
Version control	Git, with the project hosted on GitHub

3.6 CONSTRAINTS AND ASSUMPTIONS

- The market is simulated; real data is used only for calibration constants. One simulation tick represents one second of trading, and a trading day is 23,400 ticks (6.5 hours).
- Ground truth is exact: an account-window is labelled as manipulation only if the account belongs to an episode and the window overlaps the episode by at least 15 ticks.
- The LLM is an untrusted component. Its proposals are validated against hard parameter bounds, its outputs are logged, and every function it can call is either read-only or validated.
- API credit is limited, so every call must have a clear purpose and identical requests must be answered from cache.

Chapter 4
System Design

4.1 ARCHITECTURAL OVERVIEW

Veridex is organised as five modules around a shared configuration layer (Figure 4.1). Module 1, the agent-based market simulator, produces order, trade, bar and ground-truth logs. Module 2, the detection engine, turns those logs into trader-level features and scores every account in every window. Module 3, the adversarial evaluation loop, runs the experiment round by round: it simulates new markets, scores them with the frozen detector, retrains, scores again and records how effective each manipulation remained. Module 4, the LLM and agentic layer, reads Module 3's results to propose the next round's manipulation parameters, and also writes case notes and answers analyst questions. Module 5, the dashboard, presents all of these artefacts.

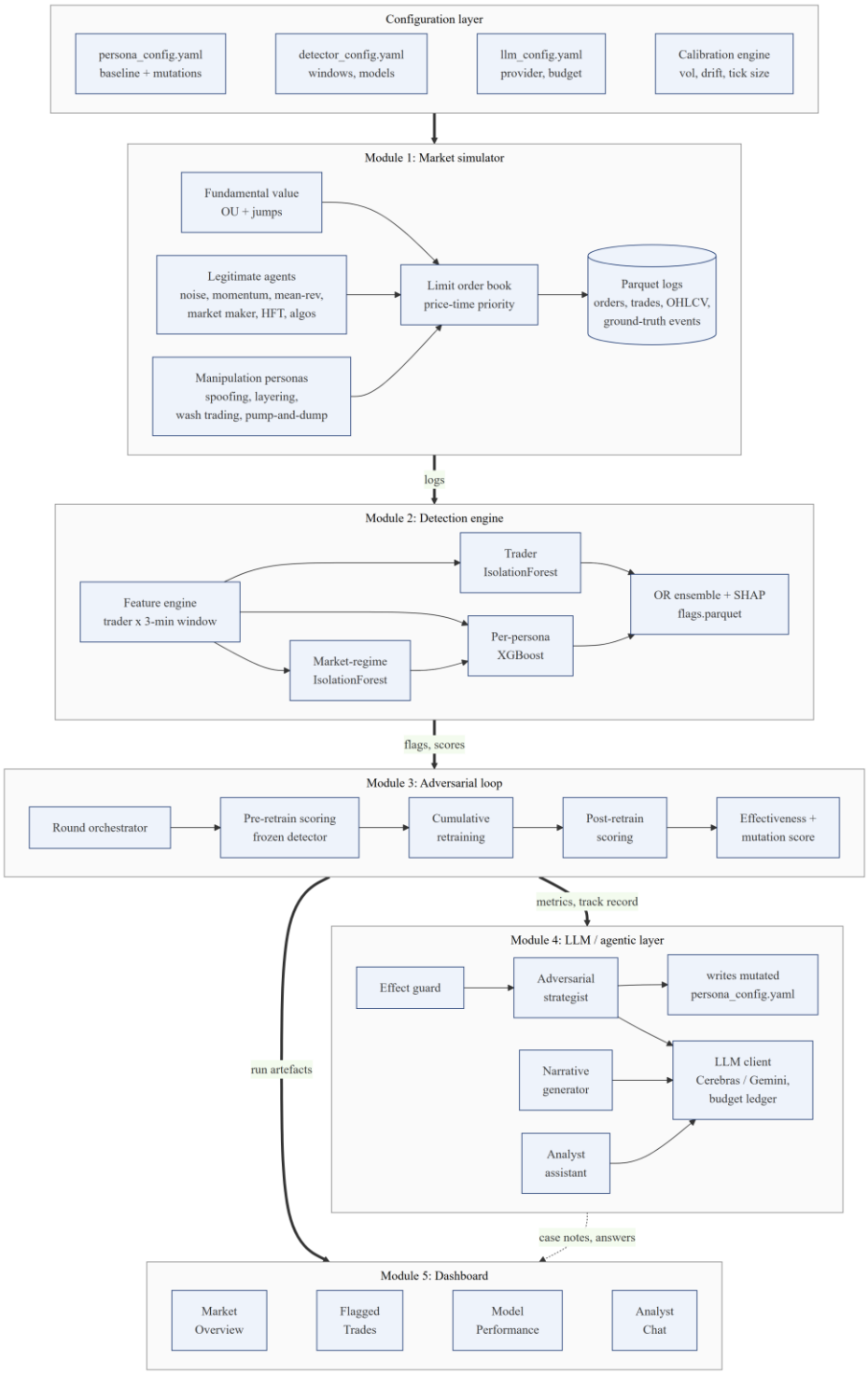


Figure 4.1: Module-wise architecture of Veridex (rendered from a Mermaid diagram)

Two design principles run through the architecture. First, modules communicate only through files on disk, each validated against a declared schema before it is written (SRS FR-0.1 and FR-0.2). This keeps the modules independent: the dashboard, for instance, can display any completed run without importing the simulator, and a run can be

analysed long after it was produced. Second, every source of randomness is seeded, so a run can be reproduced exactly from its seed and configuration.

4.2 MODULE DECOMPOSITION

Table 4.1: Modules and their source packages

Module	Package / files	Responsibility
Configuration	configs/*.yaml, src/core/calibration.py, src/core/persona_params.py	Calibration constants, persona parameters (baseline and mutations), detector and LLM settings
1. Market simulator	src/core/order_book.py, simulation.py, agents/, *agents/manipulation/*	Fundamental value, order book, legitimate agents, manipulation personas, logging
2. Detection engine	src/detector/features.py, anomaly.py, classifiers.py, pipeline.py	Features, Isolation Forests, XGBoost classifiers, ensemble, SHAP, flags
3. Adversarial loop	src/adversarial/orchestrator.py, metrics.py, effectiveness.py; scripts/run_full_pipeline.py	Round orchestration, pre/post metrics, episode coverage, effectiveness, mutation score
4. LLM layer	src/agentic/llm_client.py, adversarial_strategist.py, tools.py, mutation_library.py, narrative_generator.py, case_evidence.py, analyst_assistant.py	Provider switch and budget, strategist, effect guard, case notes, analyst chat
5. Dashboard	dashboard/app.py, data_loader.py, components/*	Market Overview, Flagged Trades, Model Performance and Analyst Chat tabs
Shared utilities	src/utils/schemas.py, io.py, logging_config.py	Schemas, validated I/O, logging

In total the project contains about 15,400 lines of Python across the source, dashboard, scripts and tests.

4.3 DATA DESIGN

Every simulated market writes four Parquet files, and every evaluated round adds detector outputs. Table 4.2 lists the main data contracts; each is defined once as a pandera schema and validated on write.

Table 4.2: Core data schemas

Artefact	Key fields	Produced by / used by
order_log	order_id, commodity, trader_id, side, price, quantity, tick, event_type (placed, cancelled, filled, partial_fill, rejected_cancel)	Simulator / feature engine, dashboard
trade_log	trade_id, commodity, tick, price, quantity, buyer_trader_id, seller_trader_id, aggressor_side	Simulator / feature engine, effectiveness, dashboard
ohlcv_bars	bar_start, commodity, open, high, low, close, volume	Simulator / dashboard
manipulation_events	event_id, commodity, persona, start_tick, end_tick, trader_ids, parameters	Simulator / labels, effectiveness, evaluation
features / labels	(commodity, window_start, trader_id) index; 51 features; one 0/1 column per persona	Feature engine / detector training and scoring
flags	final_flag, final_score, iso_score, predicted_persona, triggered_by, top_shap_features, persona_scores	Detector / dashboard, narrative generator, analyst tools
metrics.json	Per persona: post- and pre-retrain precision, recall, F1, FPR, coverage, latency; effectiveness, ratios, scores, parameters	Adversarial loop / strategist, dashboard

Artefact	Key fields	Produced by / used by
strategist.json	Per persona: source (LLM or hand-authored), parameter changes, reasoning, guard actions	Pipeline / dashboard

A run is stored as `data/runs/<run_id>/round_<N>/`, with a `train` market and two test markets (`test` and `test_1`) per round, each containing the logs for all eight commodities, plus the round's metrics, flags, thresholds and SHAP importance. The LLM audit log `llm_calls.jsonl` and the strategist's reasoning log sit at the run level.

4.4 ADVERSARIAL ROUND DESIGN

Figure 4.2 shows the control flow of one experiment. Round 0 establishes the baseline: the personas use their documented, obvious parameters, the detector is trained on the round-0 training market, and the test markets give the baseline metrics and the baseline market effect of each scheme. Every later round has a mutation step and an evaluation step. In the mutation step the effect guard first checks whether any persona's effect collapsed in the previous round; the strategist then proposes new parameters, which are validated against hard bounds, and the at-risk rule is enforced. In the evaluation step new markets are simulated with the mutated personas and scored twice: first by the frozen detector from the previous round (which measures evasion), then by a detector retrained on all rounds so far (which measures recovery).

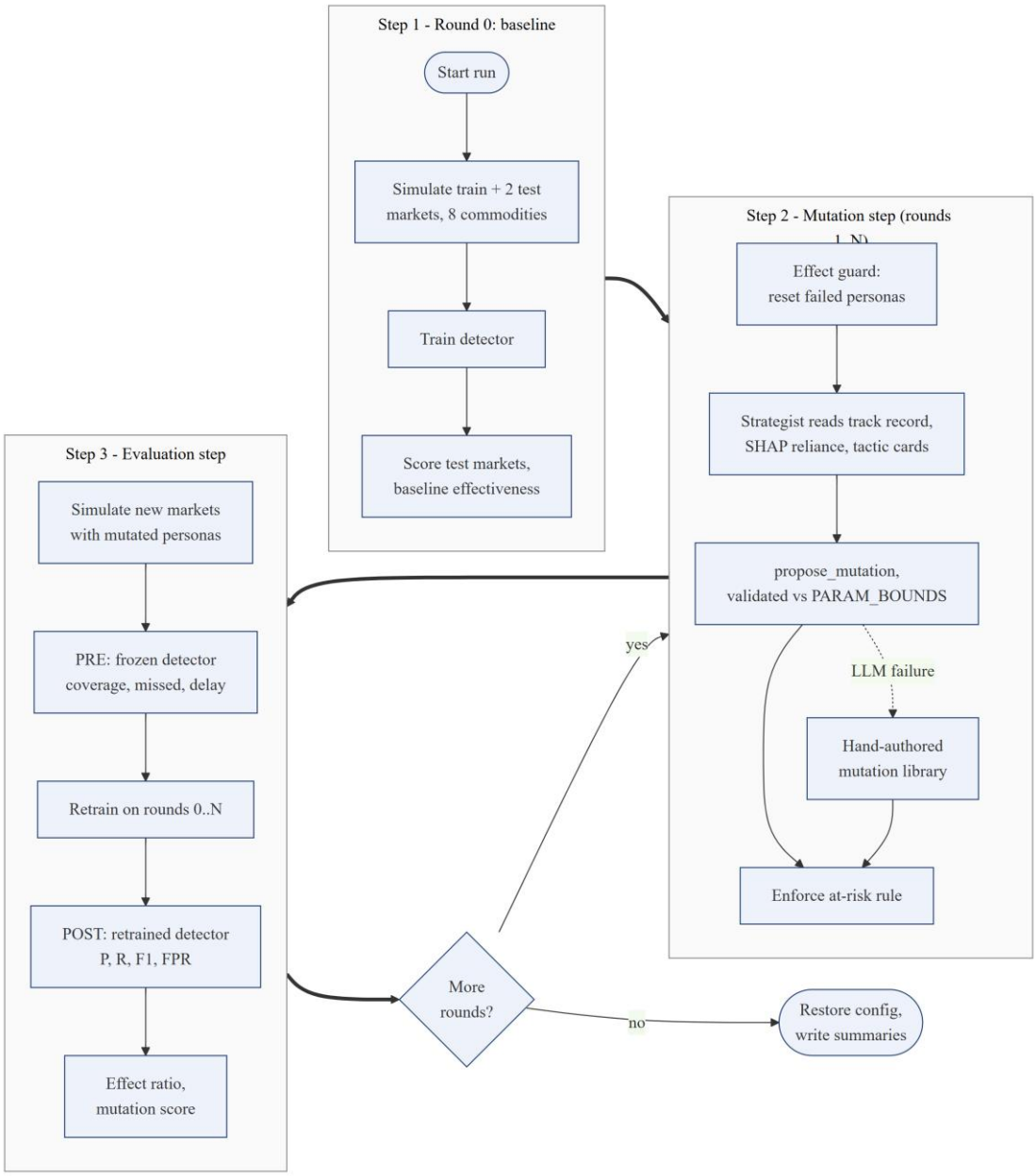


Figure 4.2: Control flow of the adversarial evaluation loop

Using two independently simulated test markets per round, rather than one, reduces the chance that a single unusual market drives the round's conclusions. Each round's train and test markets use different seeds and tick offsets, so the detector is never scored on data it was trained on.

4.5 CONFIGURATION DESIGN

All behaviour that a user might want to change is in configuration files rather than code. `persona_config.yaml` holds a `round_0` baseline for each persona; mutations are written as top-level persona keys that override the baseline. A single mapping table

(`PERSONA_PARAM_MAP`) translates persona parameter names into simulator settings, so the strategist, the hand-authored library and the simulator all use the same names. `detector_config.yaml` sets the window length and step, the Isolation Forest contamination, the XGBoost hyperparameters and the threshold search range. `llm_config.yaml` selects the LLM provider and sets token limits, the budget cap and call spacing. The pipeline copies the baseline configuration at the start of a run and restores it at the end, even if the run is interrupted, so one experiment's mutations can never leak into the next.

4.6 DASHBOARD DESIGN

The dashboard is a read-only consumer of run artefacts (Figure 4.3). A cached data-loader layer reads the files of the selected run, round and test market. Four tabs present the data: Market Overview, Flagged Trades, Model Performance and Analyst Chat. Only two actions call the LLM: generating a case note for a flagged account, and asking the analyst assistant a question. If the LLM is unavailable, those two features show an explanation and everything else continues to work.

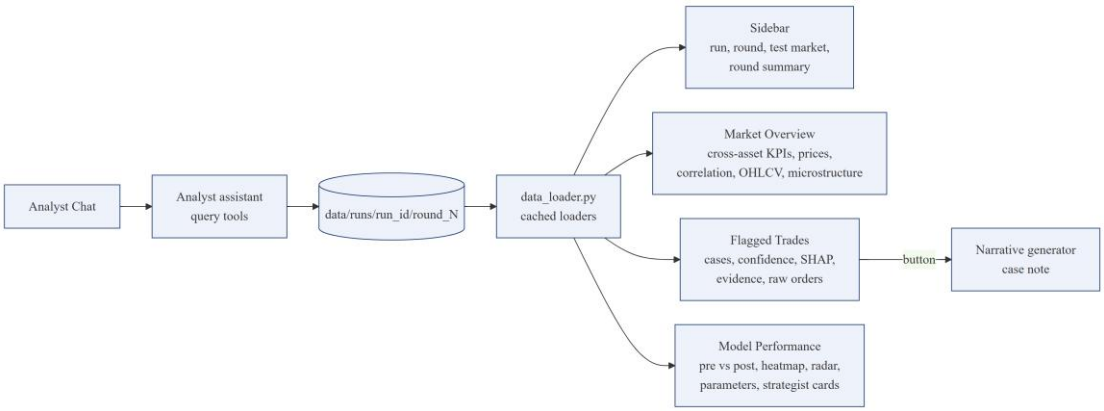


Figure 4.3: Data flow of the Streamlit dashboard

4.7 KEY DESIGN DECISIONS

Table 4.3: Design decisions and their rationale

Decision	Alternative considered	Reason
Trader-level detection over 3-minute windows	Market-level (whole-book) detection	Analysts investigate accounts, not markets; account-level labels are exact in simulation
One XGBoost classifier per persona	One multi-class model	Persona-specific metrics, thresholds and SHAP explanations (SRS FR-2.3, FR-2.6)
OR ensemble of classifiers and Isolation Forest	Weighted score blend	Unknown or mutated schemes can still be caught by the anomaly layer
Frozen (pre-retrain) and retrained (post-retrain) scoring	Retrained scoring only	Evasion can only be measured against a detector that has not yet seen the mutation
Evasion multiplied by effect ratio	Evasion alone	Prevents the adversary from “evading” by not manipulating

Decision	Alternative considered	Reason
Any-alert coverage in the adversary's score	Persona-specific coverage	Being flagged under another persona's label is still being caught
Single validated tool for the LLM	Many read tools in an agent loop	Context is pre-fetched once; fewer calls, lower cost, no unvalidated actions
Legitimate look-alike traders	Only simple legitimate agents	Without them every large or heavily cancelling account is trivially anomalous

Chapter 5
Implementation: Agent-Based Market Simulator

This chapter describes Module 1, the simulator that produces every market used in the project. Its purpose is to generate order-book data that behaves like a real futures market, contains manipulation whose timing and accounts are known exactly, and also contains legitimate behaviour that can be mistaken for manipulation. The simulator is a discrete-time agent-based model: one tick represents one second, and in each tick every agent may submit orders to a central limit order book.

5.1 LIMIT ORDER BOOK AND MATCHING ENGINE

The order book (`src/core/order_book.py`) keeps resting buy and sell orders in price-level dictionaries, each level holding a queue of orders in arrival order. A new limit order is matched against the opposite side while its price crosses the best opposite quote; each match executes at the resting order's price, and the earliest order at the best price is filled first. This is price-time priority, the matching rule used by futures exchanges. Any unfilled remainder rests in the book. Every trade records the aggressor side (whether the incoming order was a buy or a sell), which several detection features need. Prices are snapped to each commodity's real tick size, so a gold order can only be placed in steps of 0.10 and a natural gas order in steps of 0.001.

Before agents act, the simulator builds a read-only snapshot of the book, `BookState`. It contains the best bid and ask, the mid price, depth at each level, and the visible volume on each side within ±0.2% of the mid. From those volumes the snapshot computes the *order-book imbalance*,

$$\textit{imbalance} = (\textit{bid volume} - \textit{ask volume}) / (\textit{bid volume} + \textit{ask volume})$$

which ranges from −1 (all visible depth on the ask side) to +1 (all on the bid side). Real liquidity providers lean their quotes on this quantity, and that is precisely what spoofing and layering exploit. The snapshot also carries an activity signal (traded volume over the last 60 ticks compared with a slow exponential moving average), which personas use to decide whether the market is busy. Order identifiers are deterministic strings built from a run prefix, the trader id and a counter, so that two runs with the same seed produce byte-identical logs.

5.2 FUNDAMENTAL VALUE AND CALIBRATION

Each commodity has a hidden fundamental value that follows an Ornstein–Uhlenbeck (OU) process with occasional jumps:

$$dX_t = \kappa(\theta - X_t)\, dt + \sigma\, dW_t + J_t$$

where κ is the speed of mean reversion, θ the long-run level, σ the volatility, W_t Brownian motion and J_t a compound-Poisson jump term (on average four jumps per simulated day). Daily drift and volatility are converted to per-tick values by dividing by the number of ticks in a day (23,400) and its square root respectively. The calibration engine can fit these parameters by ordinary least squares on an AR(1) regression of real log prices, the standard discrete-time estimator for this process (Vasicek, 1977); when market data is unavailable it uses pre-calibrated constants from published commodity studies. Table 5.1 lists the constants used in the experiments. The fundamental value is never traded directly. Only mean-reversion traders observe it, so the reversion of market prices towards value is an emergent result of their trading rather than an imposed formula.

Table 5.1: Calibration constants per commodity

Commodity	Initial price	Tick size	Daily volatility	Daily drift	Mean reversion speed
WTI crude oil	75.00	0.01	2.20%	0.005%	0.035
Brent crude oil	82.00	0.01	2.10%	0.005%	0.033
Gold	2,400.00	0.10	0.94%	0.010%	0.050
Silver	28.50	0.005	1.50%	0.010%	0.060
Natural gas	2.50	0.001	4.00%	−0.010%	0.100
Copper	4.20	0.0005	1.20%	0.020%	0.040
Wheat	5.80	0.0025	1.60%	0.002%	0.080
Corn	4.30	0.0025	1.40%	0.002%	0.080

5.3 LEGITIMATE MARKET PARTICIPANTS

Six types of legitimate agent populate each market (Table 5.2). Their mix follows the broad categories of the CFTC Commitments of Traders report: uninformed retail flow, speculative trend followers and commercial participants with a view on value, together with liquidity providers.

Table 5.2: Legitimate agent population per market

Agent	Count	Behaviour	Role
Noise trader	20	With probability 0.6 per tick, a small limit order on a random side, priced around the microprice; unfilled orders expire after a random lifetime	Background liquidity and order flow; cover for manipulation

Agent	Count	Behaviour	Role
Momentum trader	8	Measures the price change over the last 60 seconds; if it exceeds a threshold, sends a marketable order in the trend direction, within a position limit	Speculators; amplifies pumps (exploited by pump-and-dump)
Mean-reversion trader	6	Trades at the touch against deviations of price from the fundamental value	Commercial participants; anchors price to value
Market maker	1	Requotes both sides almost every tick; skews quotes against its inventory (in half-spreads) and leans on order-book imbalance	Designated liquidity; the participant that spoofing deceives
HFT liquidity provider	3	Requotes with probability 0.8 per tick as a ladder of 1–3 levels per side; cancels about 93% of its volume; one-sided when inventory is high	Legitimate heavy canceller and multi-level quoter (hard negative)
Execution algorithm	3	Works large parent orders (5–20× level depth) by resting big child orders (1–8× depth) at or near the touch, chasing the price, with occasional hedges	Legitimate large-order canceller (hard negative)

The last two agent types were added specifically to make detection realistic. Without them, the only accounts that ever placed large orders or cancelled most of their orders were manipulators, so even a simple anomaly detector found every spoofer regardless of its tactics. Real futures markets contain many legitimate firms that cancel over 90% of their orders and many institutions that rest and withdraw large orders, which is why regulators have to look for more specific patterns. Section 5.6 shows the effect of these agents on the feature distributions.

Manipulator accounts also behave like ordinary small traders outside their manipulation window (a background order with probability 0.05 per tick, kept close to flat). Without this, a detector could learn "this account only exists while manipulating" rather than the manipulation behaviour itself.

5.4 SIMULATION LOOP

Figure 5.1 shows the work done in every tick. The fundamental value is advanced first. Orders whose lifetime has expired, and orders whose cancellation a persona scheduled, are cancelled next. The market maker then cancels and replaces its quotes. All other actors (noise, momentum and mean-reversion traders, HFT liquidity providers, execution algorithms and the persona controllers) then act in a random order drawn from a seeded generator, so no participant systematically acts first. Each order is matched immediately, trades and order events are logged, inventories and price histories are updated, and the activity signal is refreshed.

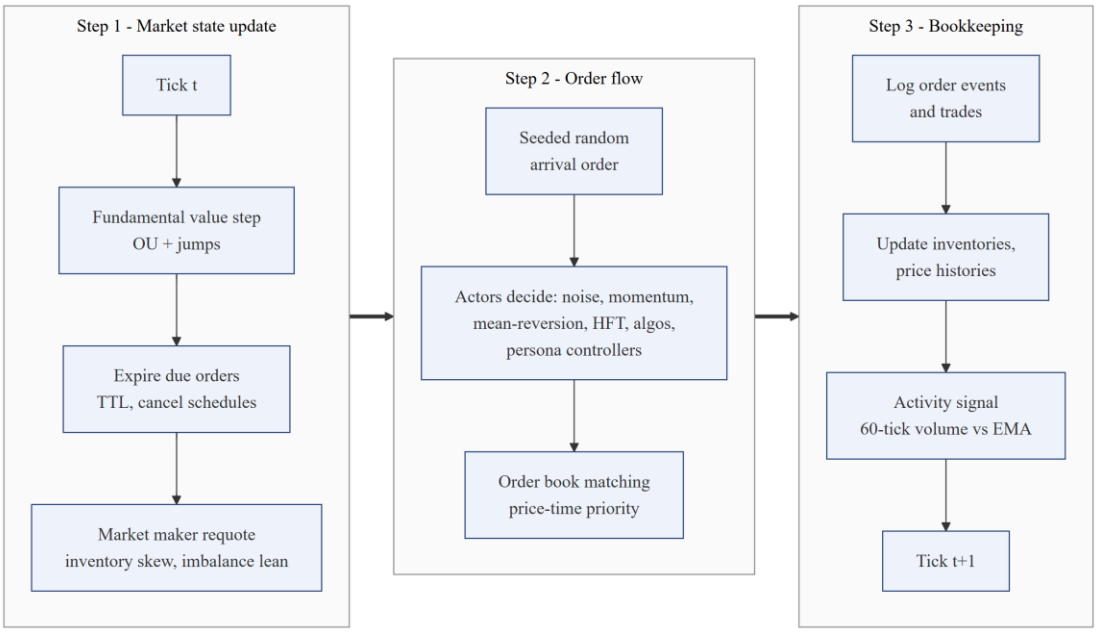


Figure 5.1: Work performed by the simulator in each tick

After the last tick, trades are resampled into OHLCV bars, the manipulation episodes are written as ground-truth rows (one row per episode, listing every account involved and the exact parameters used), and all four outputs are validated and written as Parquet files. Eight commodities are simulated in parallel processes, each with its own seed derived from the run seed, so prices in different commodities do not move in lockstep.

5.5 MANIPULATION PERSONAS

A persona is a controller that operates one or more manipulator accounts during an episode. Episodes are scheduled at random times in each market; each commodity has an affinity profile that sets how likely each persona is, and each persona has a typical duration (for example 140–420 ticks for spoofing and 220–600 ticks for wash trading). Personas place real orders into the same book as everyone else, so their market effect is genuine. Every persona has a baseline (round 0) parameter set that reflects the documented, unsophisticated version of the scheme, and a set of "knobs" with hard bounds that the adversarial strategist may change. Orders carry simulator-only audit tags (for example `s` for a spoof leg and `g` for a genuine leg) that the effectiveness accounting uses; the detector never reads them.

5.5.1 SPOOFING

Each spoofing cycle has two legs. A small *genuine* order is placed at the best price on one side, for example a sell at the best ask. A large *spoof* order is placed on the other side, slightly behind the best bid, sized at a multiple of the average visible level depth (8× in the baseline). The spoof sits inside the ±0.2% band, so the book suddenly looks bid-heavy: the market maker leans its quotes upward and noise traders price off a higher microprice. The price drifts towards the genuine sell, which fills, and a few ticks later the spoof is cancelled before anyone can trade against it. Table 5.3 lists the knobs. They encode evasion tactics from the enforcement cases: randomised size and cancel timing (Coscia), splitting the spoof across levels, using a second account for the spoof leg (JPMorgan), keeping the spoof pinned behind a moving price (Sarao), trading aggressively after pulling the spoof (Oystacher), acting mainly in busy periods, and trading genuinely alongside the spoof (cover trading).

Table 5.3: Spoofing knobs (baseline value and strategist bounds)

Knob	Baseline	Bounds	Meaning
order_size_multiplier	8.0	1 – 15	Spoof size as a multiple of average level depth
cancellation_delay_ticks	5	1 – 30	Mean ticks the spoof rests before cancellation
cancel_jitter	0.0	0 – 0.8	Cancel delay drawn from delay × U(1−j, 1+j)
size_jitter	0.0	0 – 0.8	Spoof size multiplied by U(1−j, 1+j)
n_split_orders	1	1 – 4	Spoof split into k orders on consecutive levels
n_accounts	1	1 – 3	Two or more: genuine and spoof legs from different accounts
frequency	0.4	0.05 – 1.0	Chance per tick of starting a new cycle
price_aggressiveness	0.0005	0.0001 – 0.01	Distance behind the best price (fraction of price)
reprice_to_touch	0	0 – 1	Keep the spoof pinned behind the moving best price
flip_after_cancel	0.0	0 – 1.0	Chance of an aggressive trade in the genuine direction after the cancel
busy_period_bias	0.0	0 – 1.0	Skip quiet periods; act mainly when the market is busy
cover_rate	0.0	0 – 0.5	Per-account chance per tick of a genuine cover order
cover_size	0.0	0 – 3.0	Cover order size as a multiple of level depth

5.5.2 LAYERING

Layering, modelled on the Tower Research case, places a small genuine order at the best price on one side and several moderate orders on consecutive price levels on the other side, all cancelled together after the genuine order fills. No single layer is extreme; the deception is the *shape* of the depth. Layer i sits $(i+1)$ × spacing behind the best price, so the spacing directly controls whether the layers count as near-touch depth. Measurements made during development showed that a spacing of 0.0004 keeps the

full effect, 0.0005 keeps 88%, and 0.001 keeps only about 42% because the layers fall outside the band that other participants watch. Table 5.4 lists the knobs.

Table 5.4: Layering knobs (baseline value and strategist bounds)

Knob	Baseline	Bounds	Meaning
n_layers	4	2 – 8	Orders stacked on consecutive levels per cycle
layer_spacing	0.0002	0.00005 – 0.003	Gap between layers as a fraction of price (at least one tick)
layer_size_multiplier	2.0	0.5 – 6.0	Layer size as a multiple of level depth
size_taper	0.0	0 – 0.9	Smaller layers deeper in the book
cancellation_delay_ticks	5	1 – 30	Mean ticks the layers rest
cancel_jitter	0.0	0 – 0.8	Randomisation of cancel timing
frequency	0.3	0.05 – 1.0	Chance per tick of a new layering cycle
busy_period_bias	0.0	0 – 1.0	Act mainly when the market is busy
cover_rate / cover_size	0.0 / 0.0	0 – 0.5 / 0 – 3.0	Genuine cover trading during the episode

5.5.3 WASH TRADING

Wash trading uses groups of colluding accounts. In each cycle one account rests an order strictly inside the spread and a partner immediately takes it at the same price and size, so no outsider can trade in between (if the spread is only one tick wide, the group waits). Roles rotate around a ring (A sells to B, B to C, C to A), so every account's net position stays close to zero while gross volume prints to the market. Several groups can split the persona's total cross rate, which dilutes per-account statistics without adding volume. Camouflage cycles replace some crosses with genuine trades with the market (Table 5.5).

Table 5.5: Wash trading knobs (baseline value and strategist bounds)

Knob	Baseline	Bounds	Meaning
trade_frequency	0.5	0.05 – 1.0	Total cross rate per tick (split across groups)
n_groups	1	1 – 5	Independent colluding groups
ring_size	2	2 – 6	Accounts per group; crosses rotate around the ring
size_jitter	0.0	0 – 0.9	Varied rather than scripted lot sizes
camouflage_rate	0.0	0 – 0.5	Share of cycles replaced by a genuine market trade
busy_period_bias	0.0	0 – 1.0	Act mainly when the market is busy

5.5.4 PUMP-AND-DUMP (MOMENTUM IGNITION)

The pump-and-dump persona is a coalition of accounts that moves through four phases (Figure 5.2). During *accumulation* the accounts quietly buy with passive bids. In the last 30% of the burst they *ignite*: they send marketable buys through the ask, which moves the price and triggers the momentum traders. During *distribution* they offer their inventory at the ask into the demand they created, and finally they *dump* whatever

remains with aggressive sells. Staggering gives each account its own random start, decoys are small trades in the opposite direction, and the busy-period bias delays action until the market is active (Table 5.6).



Figure 5.2: Phases of the pump-and-dump persona

Table 5.6: Pump-and-dump knobs (baseline value and strategist bounds)

Knob	Baseline	Bounds	Meaning
n_coordinated_accounts	3	2 – 15	Number of colluding accounts
accumulation_size	150	50 – 1000	Target position per account
burst_duration_ticks	200	50 – 1000	Accumulate plus ignite duration (ignite is the last 30%)
dump_delay_ticks	100	10 – 500	Passive distribution period before the dump
stagger	0.0	0 – 1.0	Random per-account start delay (fraction of the burst)
ignition_aggression	0.002	0.0002 – 0.005	How far through the ask the ignition buys go
decoy_rate	0.0	0 – 0.4	Chance per tick of a small opposite-side trade
busy_period_bias	0.0	0 – 1.0	Act mainly when the market is busy

5.5.5 COVER TRADING

Real spoofers such as Sarao and Coscia also traded genuinely while they manipulated, so their manipulative orders were a minority of their activity. With `cover_rate` above zero, every spoofing or layering account keeps sending genuine orders during the episode: 35% are marketable and the rest rest at the account's own side of the touch, and the account leans against its position so it stays roughly flat. `cover_size` sets their

size relative to the level depth. Because the detector's cancel, fill and large-order ratios are weighted by volume, only cover volume comparable to the scheme's own volume dilutes them; the depth reference is fixed when cover trading starts, so the account's own resting cover orders cannot inflate the size of later ones. Cover trading costs the manipulator inventory risk, and its genuine resting orders partly offset the fake depth.

5.6 BEHAVIOURAL PROFILES OF LEGITIMATE AND MANIPULATIVE ACCOUNTS

Table 5.7 compares the median feature values of each account type in a crude-oil market during development. The high-frequency liquidity providers cancel 93% of their volume, quote on several levels and often have orders cancelled on one side while they trade on the other, so cancellation and layering features alone no longer separate manipulators from legitimate firms. The execution algorithms place orders up to 36 times the median order size and cancel about two thirds of their volume. Spoofers remain more extreme in size, which leaves the adversary a real but costly option: smaller spoofs look more like algorithm orders but create less fake depth.

Table 5.7: Median account-window feature values by account type (crude oil, development measurement)

Account type	Avg order size	Max order size	Large-order share	Cancel ratio	Fill ratio	Layered share	Spoof signature
Other legitimate	0.6×	1.9×	0.00	0.21	0.79	0.00	0.00
HFT provider	3.3×	5.7×	0.39	0.93	0.07	0.43	0.47
Execution algo	13.1×	35.9×	0.99	0.68	0.31	0.00	0.20
Layering	10.3×	24.0×	0.98	0.98	0.02	0.98	0.92
Spoofing	34.7×	103.7×	0.99	0.99	0.01	0.00	0.96

Sizes are relative to the commodity's median order size; ratios are volume-weighted.

5.7 REALISM CHECKS

Table 5.8 reports stylized facts of one-minute (60-tick) log returns in the round-0 test market of the experiment, together with the simulated one-minute volatility and the target implied by the calibrated daily volatility (daily volatility divided by $\sqrt{390}$). Most commodities show heavy tails (positive excess kurtosis) and volatility clustering (positive autocorrelation of absolute returns), the two most robust stylized facts of real markets. The order-to-trade ratio of about 2.0–2.4 and the overall cancel ratio of about 50% are within the range seen in electronic futures markets.

Table 5.8: Stylized facts of 60-tick returns, round 0 test market

Commodity	Excess kurtosis	ACF(1) of |r|	ACF(1) of r	Sim. vol ($\times 10^{-3}$)	Target vol ($\times 10^{-3}$)	Order-to-trade	Cancel ratio
Brent crude	1.74	0.166	0.112	1.09	1.06	2.22	0.54
Copper	10.68	0.234	−0.084	1.05	0.61	2.36	0.57
Corn	−0.21	0.036	−0.416	1.50	0.71	2.08	0.51
WTI crude	3.98	0.261	−0.096	1.38	1.11	2.19	0.53
Gold	8.21	0.210	0.231	0.50	0.48	2.37	0.57
Natural gas	−0.23	0.058	0.106	2.36	2.03	2.03	0.49
Silver	6.34	0.261	0.016	1.25	0.76	2.20	0.53
Wheat	5.72	0.052	−0.241	1.55	0.81	2.12	0.52

The checks also reveal limitations that are reported rather than hidden. Simulated volatility is within about 25% of the target for Brent, WTI, gold and natural gas, but between 1.6 and 2.1 times too high for copper, silver, wheat and corn, because microstructure noise from discrete ticks and small order sizes adds to the fundamental volatility in markets where the tick is large relative to the daily move. Corn and wheat show strongly negative first-order return autocorrelation (bid-ask bounce), and corn and natural gas show no excess kurtosis in this sample. These differences do not affect the detection experiments, which compare behaviour within the same markets, but they would need further tuning for studies of price dynamics.

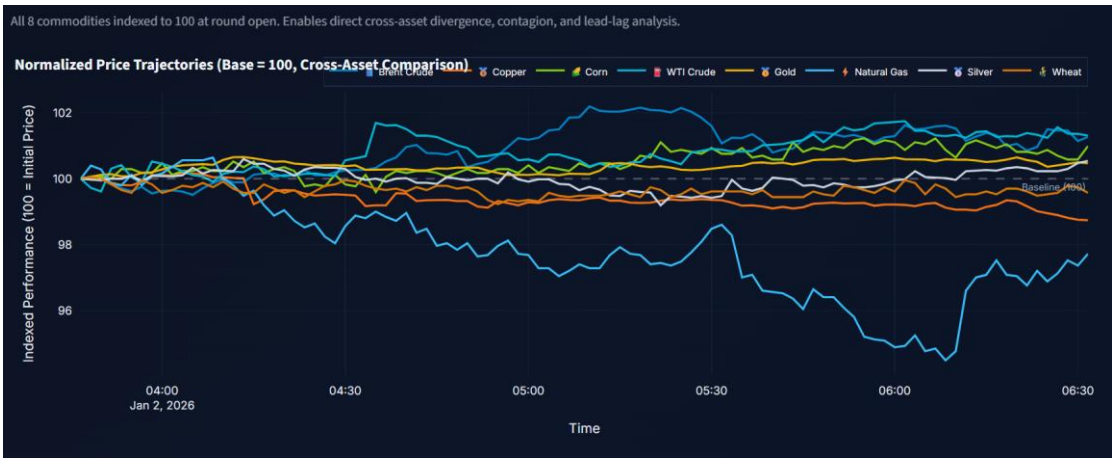


Figure 5.3: Normalised price paths of the eight commodities (dashboard, round 5 test market)

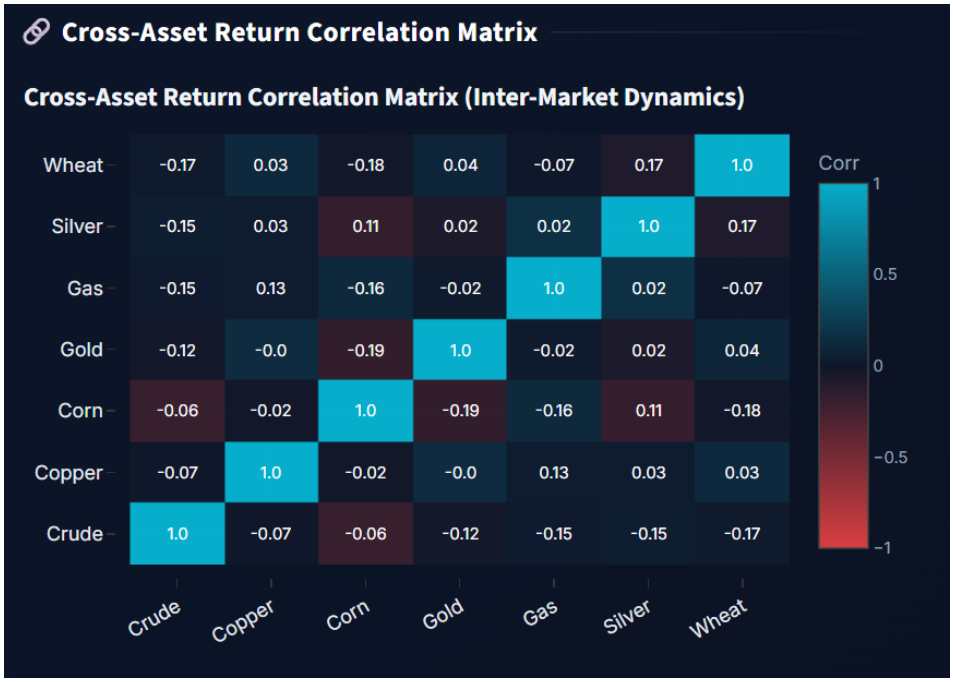


Figure 5.4: Cross-asset return correlation matrix (dashboard, round 5 test market)

Figure 5.3 shows the eight simulated price paths indexed to 100. Each commodity moves independently, with the expected differences in volatility (natural gas is the most volatile). Figure 5.4 confirms that returns are nearly uncorrelated across commodities (absolute correlations below 0.2), as intended: each market is simulated with its own seed, and cross-market linkages are outside the scope of this project.

Chapter 6

Implementation: Detection Engine and Adversarial Loop

This chapter describes Module 2, which turns raw order-book logs into alerts with explanations, and Module 3, which runs the adversarial experiment and measures both evasion and effectiveness. Figure 6.1 gives an overview of the detection pipeline.

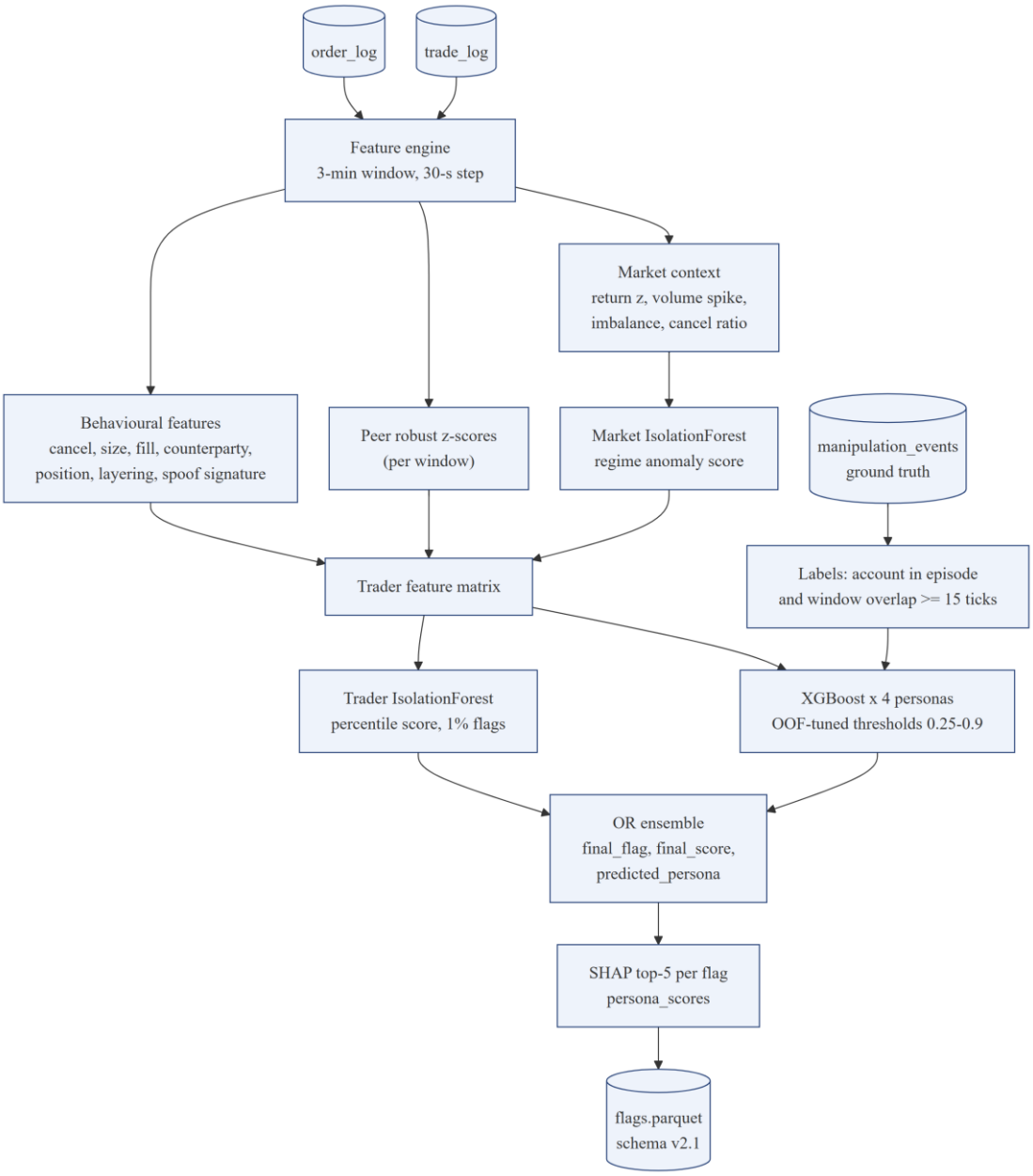


Figure 6.1: Detection pipeline from logs to explained flags

6.1 UNIT OF DETECTION

The detector scores *account-windows*: one row for each trader account that was active in a three-minute window of one commodity's market. Windows advance in 30-second

steps, so consecutive windows overlap and an episode of a few minutes is seen by several windows. Trader-level rows match how surveillance analysts work: they investigate accounts over periods of time, not abstract market states. The designated market maker is excluded from the feature table because exchanges surveil it under separate quoting obligations; the HFT liquidity providers and execution algorithms are *not* excluded, so the detector must learn to tell them apart from manipulators.

6.2 FEATURE ENGINEERING

For each account-window the feature engine (`src/detector/features.py`) computes 50 features from the order and trade logs, grouped in Table 6.1, and the market-regime anomaly score described in Section 6.4 is appended as a context feature. Ratios are weighted by volume, so a few tiny orders cannot hide a large one. Sizes are expressed relative to the commodity's median order size, so the same feature works for gold and for natural gas.

Table 6.1: Feature catalogue

Group	Features	What they capture
Activity	n_orders, n_trades, exec_vol, gross_volume_share, activity_vs_history	How much the account does, and how this compares with its own history
Order size	avg_order_size_rel, max_order_size_rel, order_size_cv, large_order_share (orders ≥ 4× median)	Unusually large or irregular orders
Cancellation	cancel_ratio, fast_cancel_ratio (≤ 10 s), large_cancel_ratio, mean_cancel_lifetime, n_large_cancels, cancel_side_imbalance, cancel_replace_ratio	Orders placed and quickly withdrawn; cancel-and-replace behaviour
Layering	layered_order_share (same tick, same side, ≥ 3 price levels)	The Tower layering fingerprint
Execution	fill_ratio, exec_side_imbalance, aggressor_ratio, passive_distance_bp, cancel_exec_opposition	Which side the account actually trades, and how aggressively
Spoofing signature	spoof_signature: share of the account's large cancelled orders during which it was filled on the opposite side	The bait-and-switch itself
Counterparty and position	top_cpty_share, reciprocal_ratio, n_counterparties, net_flatness, trade_size_cv, price_dev_bp	Trading repeatedly with the same partner while staying flat (wash trading)
Position build-up	prev_net_ratio, long_net_ratio, reversal_score, coordinated_peers, signed_flow_return	Building a position, pushing the price, then reversing; other accounts doing the same (pump-and-dump)
Market context	mkt_return_bp, mkt_volatility_bp, mkt_range_bp, mkt_volume_spike, mkt_trade_imbalance, mkt_order_imbalance, mkt_cancel_ratio, mkt_abnormal_return_z, mkt_anomaly_score	What the whole market was doing in the window
Peer comparison	peer_z_* robust z-scores of eight key features against all accounts in the same window	How unusual the account is relative to its peers right now

Two features deserve explanation. `spoof_signature` looks at every large order the account cancelled and asks whether, while that order was resting, the account was filled on the opposite side. For an honest trader this almost never happens; for a spoofer it is

the whole strategy. `coordinated_peers` counts other accounts in the same window that traded aggressively in the same direction, which captures the coordination of a pump-and-dump coalition. The peer z-scores use the median and the median absolute deviation rather than the mean and standard deviation, so a single extreme account cannot distort the scale against which everyone else is compared.

6.3 GROUND-TRUTH LABELS

Because the simulator logs every episode, labels are exact. An account-window is labelled positive for a persona if the account is one of the episode's accounts and the window overlaps the episode by at least 15 ticks. The overlap condition avoids labelling a window positive because of one or two seconds of overlap at its edge. Each persona has its own label column, so the same account-window can be evaluated separately by each classifier.

6.4 UNSUPERVISED LAYER: ISOLATION FORESTS

Two Isolation Forests, implemented with scikit-learn (Pedregosa et al., 2011), are trained without labels.The *market-regime forest* is trained on the market-context features of each window (contamination 5%); its anomaly percentile becomes the `mkt_anomaly_score` context feature. The *trader forest* is trained on the behavioural features of every account-window (200 trees, contamination 1%). Its inputs are robustly scaled, and its raw scores are converted into percentiles of the training-score distribution, so a score of 0.99 means “more anomalous than 99% of training rows” and the flag threshold is simply 0.99. The trader forest is persona-agnostic: it flags accounts that are unusual for any reason, which lets it catch schemes the supervised layer has not seen, at the cost of also flagging unusual legitimate accounts.

6.5 SUPERVISED LAYER: PER-PERSONA XGBOOST CLASSIFIERS

For each persona a separate XGBoost binary classifier predicts whether an account-window belongs to that persona. Separate models give persona-specific metrics, thresholds and explanations, which a single multi-class model would blur. Positive rows are rare, so each model weights positives by the ratio of negatives to positives

(capped at 50). Table 6.2 lists the hyperparameters; a smaller variant is used when very few positive rows are available.

Table 6.2: XGBoost hyperparameters

Parameter	Value	Parameter	Value
n_estimators	300 (150 small)	subsample	0.8
max_depth	4 (3 small)	colsample_bytree	0.8
learning_rate	0.05	gamma	0.5
min_child_weight	1	reg_alpha / reg_lambda	0.1 / 2.0
eval_metric	logloss	scale_pos_weight	n_neg / n_pos, capped at 50

Threshold tuning. A default threshold of 0.5 is rarely right for rare classes. Each classifier's decision threshold is chosen from a grid of 27 values between 0.25 and 0.9 to maximise the F1 score on *out-of-fold* predictions. The folds are formed with GroupKFold over 600-tick time blocks, so overlapping neighbouring windows always fall in the same fold; otherwise information would leak between training and validation through the overlap. Keeping the threshold inside [0.25, 0.9] protects against extreme values when a round contains only a few episodes.

6.6 ENSEMBLE, PERSONA ASSIGNMENT AND SHAP EXPLANATIONS

An account-window is flagged if any persona classifier exceeds its threshold *or* the trader Isolation Forest fires (an OR ensemble). The flag records which layers triggered, a final score (the highest persona probability, or the anomaly percentile if that is higher), and a predicted persona: the persona whose probability exceeds its own threshold by the largest margin, or “anomaly” if only the Isolation Forest fired. Every flagged row also stores all four persona probabilities (so the dashboard can show the confidence for each persona) and the five largest SHAP contributions of the classifier that produced the flag, computed exactly with TreeSHAP. The result is written to `flags.parquet`, schema version 2.1.

6.7 EVALUATION METRICS

Metrics are always reported per persona, never as one combined number. At row level the detector is evaluated with precision, recall, F1 and false-positive rate, using each classifier's tuned threshold. At episode level three further measures are computed:

- **Events detected and detection latency**: an episode is detected if any of its accounts is flagged in a window overlapping the episode; latency is the number of ticks from the episode's start to the end of the first flagged window.
- **Episode coverage**: for each episode, the share of its labelled account-windows that were flagged, averaged over episodes. Unlike row recall, coverage does not change when a mutation simply adds more accounts to an episode.
- **Any-alert variants** (caught coverage, caught events, caught latency), computed on the final flag rather than the persona's own classifier. An account flagged under another persona's label, or only by the anomaly layer, is still caught. These are the measures used to score the adversary.

6.8 ADVERSARIAL ROUND ORCHESTRATION

The round orchestrator (`src/adversarial/orchestrator.py`) runs each round as follows. It simulates one training market and two test markets, each covering all eight commodities, with different seeds and tick offsets. It loads or computes their features and labels (features are cached next to the logs, so cumulative retraining does not recompute earlier rounds). It then scores the test markets with the detector as it stood at the end of the previous round, the *pre-retrain* or *frozen* evaluation. It retrains the detector on the training markets of all rounds so far, scores the test markets again (*post-retrain*), builds and writes the flags, measures the effectiveness of every manipulation episode, and saves everything to the round's `metrics.json`. The frozen evaluation answers “did the new tactic evade the detector that existed when it was introduced?”; the retrained evaluation answers “can the detector learn to catch it?”.

6.9 MANIPULATION EFFECTIVENESS AND THE ADVERSARY'S OBJECTIVE

An evasion only matters if the manipulation still works. The effectiveness module measures, for every episode and from the test market's own logs, one primary quantity per persona (Table 6.3). The median across episodes is compared with the round-0 baseline to give the *effectiveness ratio*.

Table 6.3: Primary effectiveness measures

Persona	Primary measure	Definition
Spoofing	fake_depth_share	Share of the visible near-touch depth (±0.2% band) on the spoofed side that the spoof orders made up when placed

Persona	Primary measure	Definition
Layering	fake_depth_share	The same measure for the stacked layers
Wash trading	wash_volume_share	Self-crossed volume divided by all market volume during the episode
Pump-and-dump	ignition_pressure	Peak share of market volume taken by the coalition's marketable buying in any 60-tick window

These measures were chosen after simpler ones proved too noisy. The realised price move of a spoof, for example, varied between −0.25× and 5.3× of baseline from one round to the next because it is dominated by unrelated price noise, whereas the fake-depth share measures the lever the spoofer actually pulls. The adversary's objective for each persona combines evasion of the frozen detector with the effectiveness ratio:

$$evasion = 0.5 \times (1 - coverage) + 0.3 \times missed + 0.2 \times \min(1, delay / 300)$$

$$score = evasion \times \min(1, effect\ ratio), \quad and\ score = 0\ if\ effect\ ratio < 0.5$$

Here *coverage* is the frozen detector's any-alert episode coverage, *missed* is the share of episodes never flagged at all, and *delay* is the mean detection latency of caught episodes in ticks (capped at 300, about five minutes). An effect above the baseline earns nothing extra, because a bigger scheme is easier to catch and would otherwise be rewarded for the wrong reason. An effect below half of the baseline scores zero: that is not evading, it is not manipulating.

6.10 THE EFFECT GUARD

During development the LLM strategist sometimes made changes that quietly destroyed a scheme's effect (for example, widening the layer spacing until the layers no longer counted as depth) and then failed to undo them for several rounds. The effect guard enforces the effectiveness constraint in code rather than relying on the LLM. Before each mutation step it classifies every persona by its last effectiveness ratio:

- **Failed** (ratio below 0.5): the persona's parameters are reset to its best effective earlier set, and the strategist is told which changes broke it and not to re-apply them.
- **At risk** (ratio below 0.65): the strategist is warned and shown the effect-costing changes since the last comfortably effective round. After its proposal, any *new* effect-costing change is undone, and if it undid none of the earlier ones, they are all reverted. Evasion-only changes such as cancel timing are kept.

- **Surplus** (ratio at least 1.5): the strategist is told that the extra effect earns nothing and can be spent on evasion, for example with knobs it has never tried.

The direction in which each knob lowers effectiveness is recorded in a small table (for example, wider layer spacing, fewer layers and smaller layers all reduce fake depth). In the six-round experiment reported in Chapter 8, spoofing entered the at-risk band after round 3 (0.62×). Following the warning, the strategist raised the spoof size from 4× to 5× level depth in round 4 (its stated reason was “undo prior down move, restores effect”), so the guard had nothing to revert and the effect recovered to 0.74×. No persona fell below the 0.5 floor, so no reset was needed. The reset and revert paths are covered by unit tests built from scenarios observed in earlier development runs.

Chapter 7
Implementation: LLM Layer and Dashboard

This chapter describes Module 4, the LLM and agentic layer, and Module 5, the Streamlit dashboard. The LLM is used in three bounded roles: as the adversarial strategist, as a case-note writer, and as an analyst assistant. All three share one client that enforces the provider choice and the budget (Figure 7.1).

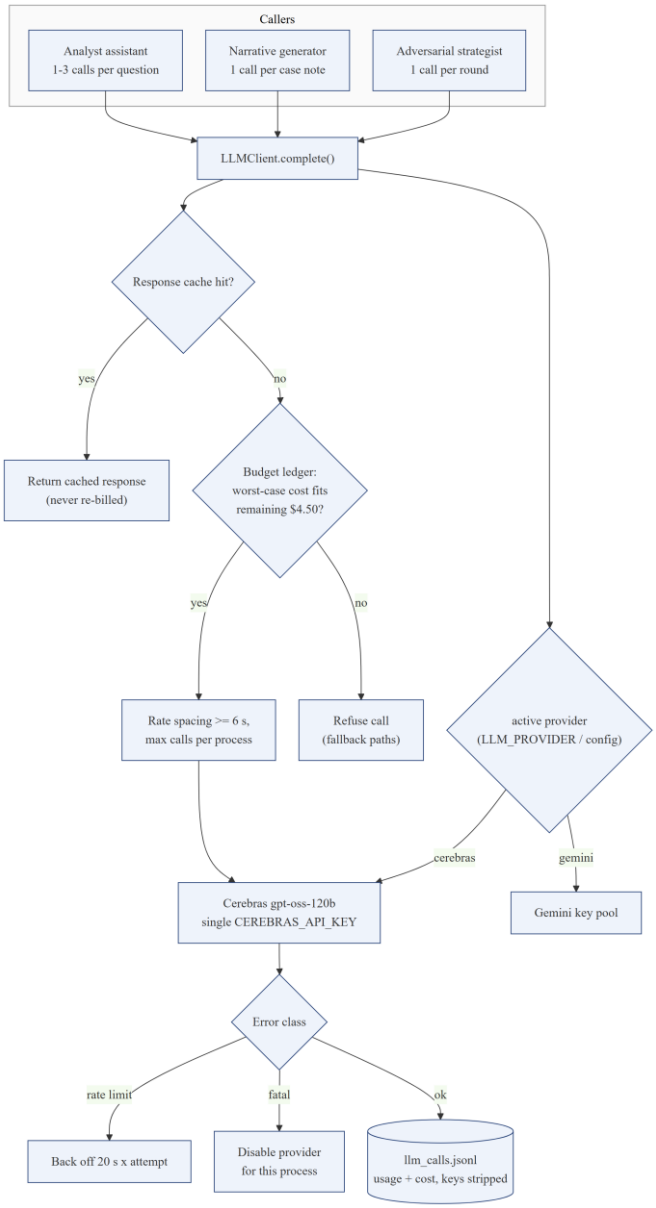


Figure 7.1: LLM client: provider switch, cache, budget ledger and error handling

7.1 LLM CLIENT, PROVIDER SWITCH AND COST CONTROL

All LLM calls go through `LLMClient.complete()` (`src/agentic/llm_client.py`), which uses LiteLLM so that the same code can talk to different providers. A single switch, the `LLM_PROVIDER` environment variable or `active_provider` in `llm_config.yaml`, selects the provider for the whole project. When Cerebras is selected, only the Cerebras key is used, for every call. The experiments used Cerebras-hosted gpt-oss-120b, an open-weight reasoning model, because it is fast and inexpensive (USD 0.35 per million input tokens and USD 0.75 per million output tokens at the time of writing). Because the project had only about USD 5 of credit, the client applies several safeguards (Table 7.1).

Table 7.1: Cost and reliability safeguards in the LLM client

Safeguard	Setting	Purpose
Budget ledger	Hard cap USD 4.50, persisted across runs	Before each call, the worst case (estimated input plus the maximum output) must fit in the remaining budget; otherwise no request is sent
Response cache	On for every role	Identical requests are answered from disk and never billed twice
Call spacing	At least 6 s between calls	Stays under the account's requests-per-minute limit
Call cap	60 calls per process	Bounds the damage of any unexpected loop
Token limits	Strategist 2,500 (medium reasoning); case note 700; chat 600	Bounds output cost; reasoning tokens count as output
Rate-limit back-off	20 s × attempt	Recovers from temporary limits without hammering the API
Fatal-error switch	Provider disabled for the process	An invalid key or exhausted quota cannot cause a retry loop
SDK retries	Disabled (max_retries = 0)	Only the client's own, bounded retry logic runs
Message sanitising	Echoed reasoning fields removed	Prevents provider errors on multi-turn tool conversations
Audit log	llm_calls.jsonl per run	Prompt, response, tokens, latency and cost of every call; API keys stripped

The whole six-round experiment used 22 strategist calls costing about USD 0.066 in total, and the cumulative spend across all development runs stayed below USD 0.45 of the USD 4.50 cap.

7.2 ADVERSARIAL STRATEGIST

Before every round after round 0, the strategist (`src/agentic/adversarial_strategist.py`) proposes new parameters for each persona. The design goal was a bounded agent that produces useful mutations from as few calls as possible. Instead of letting the model explore the run with many read-only tool calls, which would multiply the cost, the system pre-fetches a compact context

once per round and offers the model a single tool, `propose_mutation`. For each persona the context contains:

- every knob with its type, bounds, current value and a one-line meaning;
- the full track record: for every earlier round, the parameters used, who chose them, the frozen detector's any-alert coverage, episodes caught and delay, the retrained recall, the effect ratio and the score;
- knobs never changed so far and knobs currently at a bound, as exploration hints;
- the detector's current reliance (mean absolute SHAP of its six most important features) and the features whose importance rose since the last round;
- the tactic cards for the persona, and an effect-guard note (failed, at risk or surplus) when applicable.

The system prompt states the objective formula from Section 6.9, explains that being flagged under another persona's label still counts as caught and that effect above 1.0× earns nothing, and prescribes a method: read the track record, start from the best parameter set, change one to three knobs aimed at features the detector currently relies on, prefer low-cost tactics, try untried knobs when the score stalls, and never resubmit a past parameter set. Each proposal is validated against the hard bounds; a rejected proposal is returned once with the exact validation error, and a second rejection (or any LLM failure) falls back to the hand-authored mutation library for that round. The loop is limited to five model turns per round. Appendix 4 reproduces an actual exchange from the experiment.

7.2.1 TACTIC CARDS

Tactic cards (`docs/tactic_cards.yaml`) are short, structured descriptions of documented evasion tactics: the source case, the knobs that implement it, the detector features it is expected to weaken, and what it costs the manipulator. For example, the spoofing card “Cancel-timing randomisation” cites the Coscia case, maps to `cancel_jitter` and `cancellation_delay_ticks`, weakens `fast_cancel_ratio` and `mean_cancel_lifetime`, and warns that longer-resting spoofs risk being filled. Some cards record measured trade-offs from development experiments, such as the hard spacing limit for layering. Keeping the cards short matters because they are sent to a paid model every round.

7.3 CASE-NOTE GENERATOR

When an analyst clicks *Generate case note*, the narrative generator writes a four-to-six sentence explanation of what the account actually did. It does not simply restate the SHAP chart, which the analyst can already see. Instead it receives three inputs: the flag row with its top SHAP features; a short, persona-specific description of the manipulation *mechanism*; and an *evidence* dictionary computed from the account's own orders and trades in the flagged span (`src/agentic/case_evidence.py`). For a layering case, for example, the evidence includes the number of multi-level batches, the median number of levels per batch, the layer size relative to the market median, the price gap between layers, the share of layer orders cancelled and their median lifetime.

After generation, a grounding check extracts every number in the note and verifies that it appears in the evidence or the flag row. If any number is unsupported, the generator retries once, naming the unsupported numbers; if the note still fails, it is discarded and the dashboard shows only the SHAP evidence. Every note is labelled as AI-generated simulation output.

7.4 ANALYST ASSISTANT

The analyst assistant answers free-text questions about the selected run. It has read-only tools that query the run's files: listing rounds, querying round metrics, querying flags with filters, and retrieving an account's history in a round. The assistant is instructed to answer only from tool results and to cite them; the dashboard shows the exact tool calls under a *Sources* expander so the analyst can verify each answer. Context sent to the model is bounded, and the number of tool calls per question is limited; a question that would need more calls is answered with a request to narrow it, rather than a guess.

7.5 DASHBOARD

The dashboard (`dashboard/app.py`) is a Streamlit application with a dark theme, a sidebar for selection and four tabs. Figures 7.2 to 7.24 are screenshots of the dashboard showing the experiment analysed in Chapter 8 (run `adversarial_20261014T151727Z_seed42`), mostly at round 5. Every page carries the notice “All data is synthetic. No real trading, real accounts, or real regulatory findings are represented in this application.”

7.5.1 SIDEBAR AND NAVIGATION

The sidebar (Figure 7.2) selects the adversarial run, the round (a slider from 0 to the last completed round) and the test market, because each round is evaluated on two independently simulated markets. Below the selectors, a round summary lists precision, recall, F1 and false-positive rate for every persona. The four tabs are reached through a segmented control at the top of the page, and only the selected tab is computed.

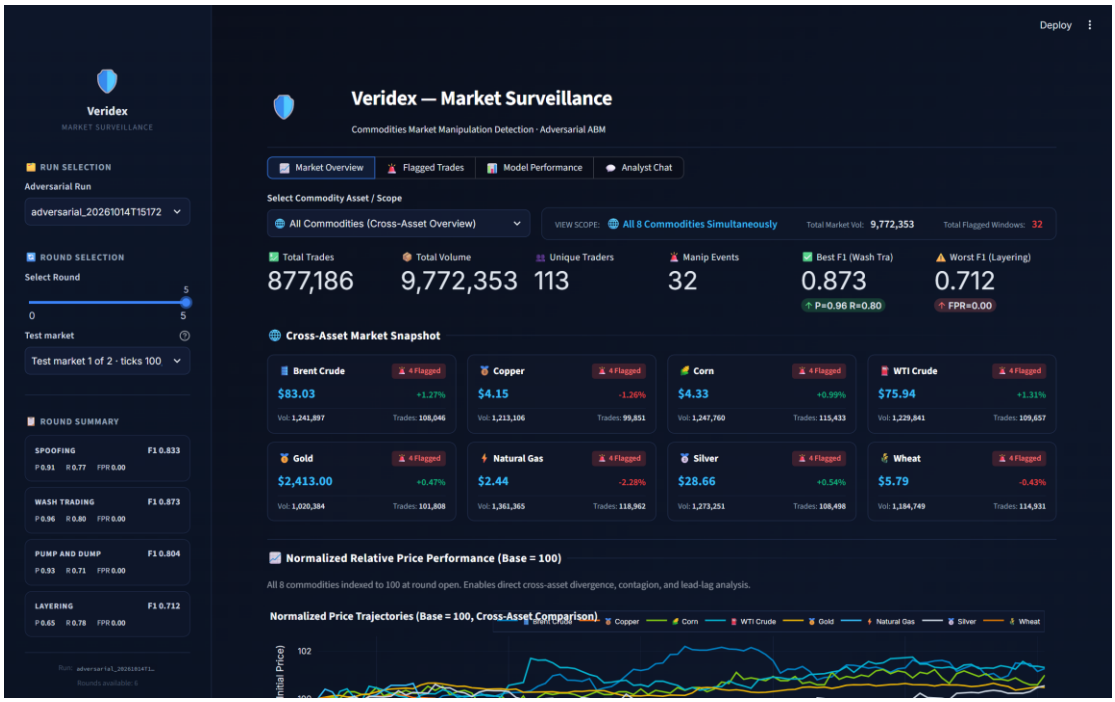


Figure 7.2: Dashboard with the sidebar (run, round and test-market selection, round summary)

7.5.2 MARKET OVERVIEW

With *All Commodities* selected, the Market Overview tab shows headline figures for the selected market (total trades and volume, unique traders, manipulation episodes, and the best and worst persona F1), a snapshot card for each commodity with its last price, change, volume and number of flagged windows (Figure 7.3), the normalised price paths shown earlier in Figure 5.3, a grid of the eight price profiles with ground-truth manipulation intervals shaded (Figure 7.4), the correlation matrix (Figure 5.4), the volume distribution across commodities (Figure 7.5) and the number of flagged windows by persona and commodity (Figure 7.6).

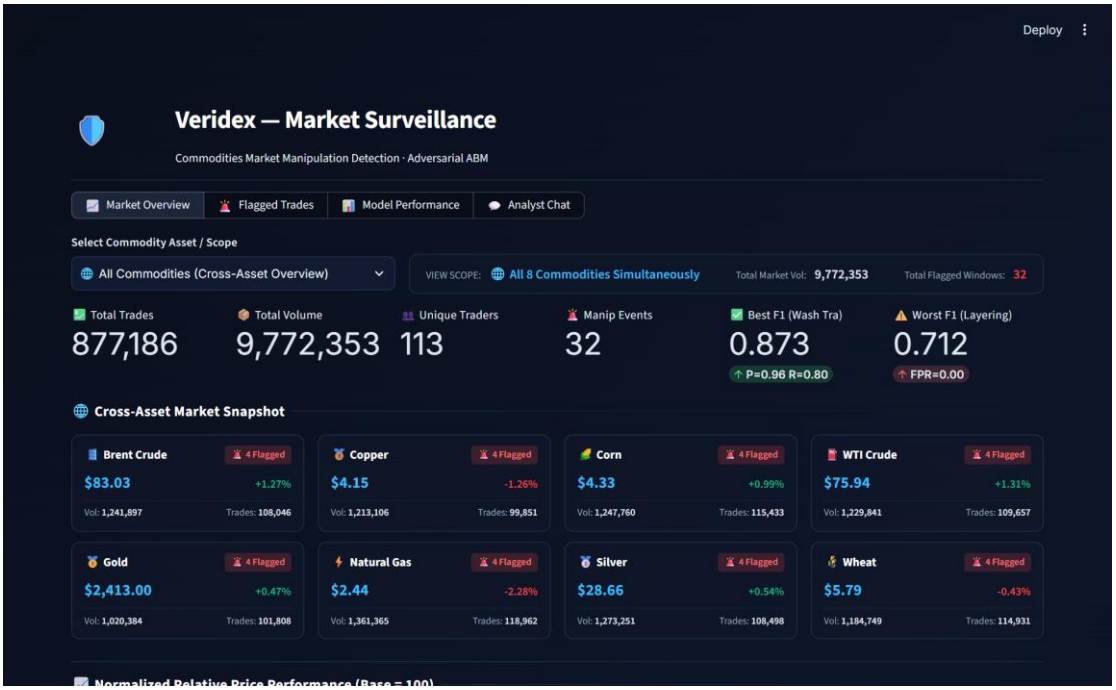


Figure 7.3: Market Overview: headline metrics and cross-asset snapshot (round 5)

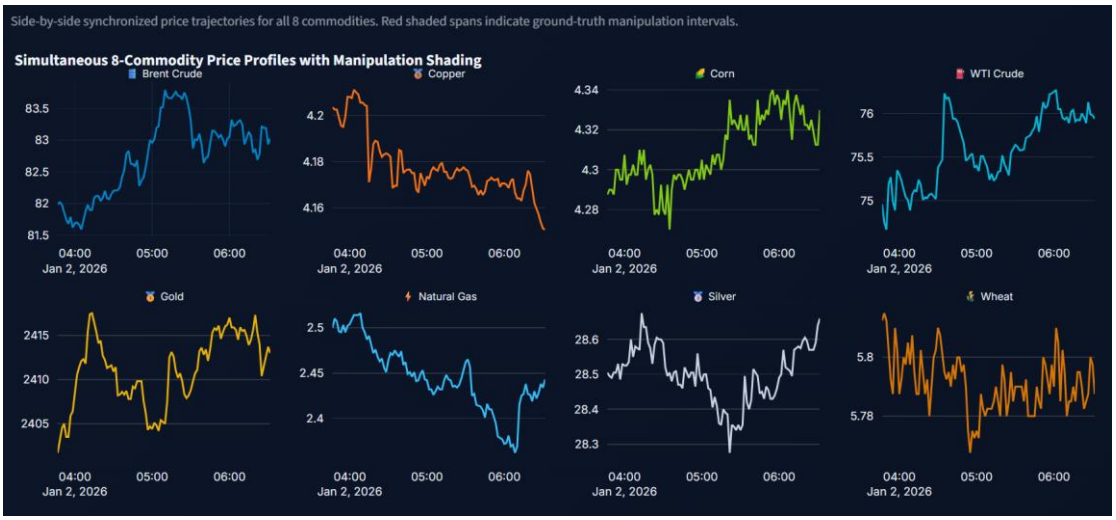


Figure 7.4: Market Overview: simultaneous price profiles of the eight commodities

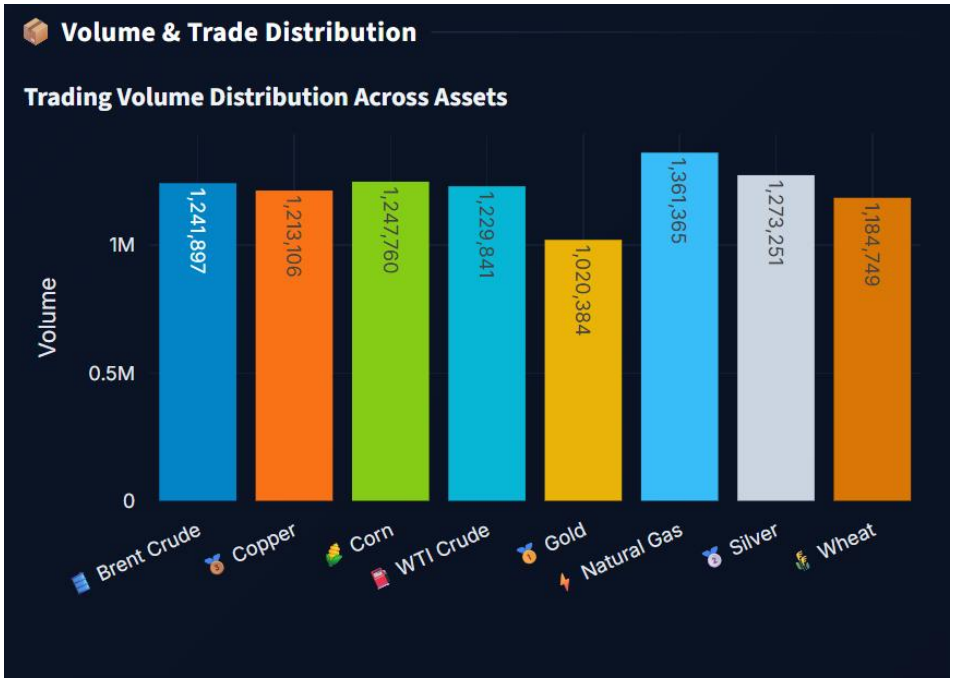


Figure 7.5: Market Overview: trading volume by commodity

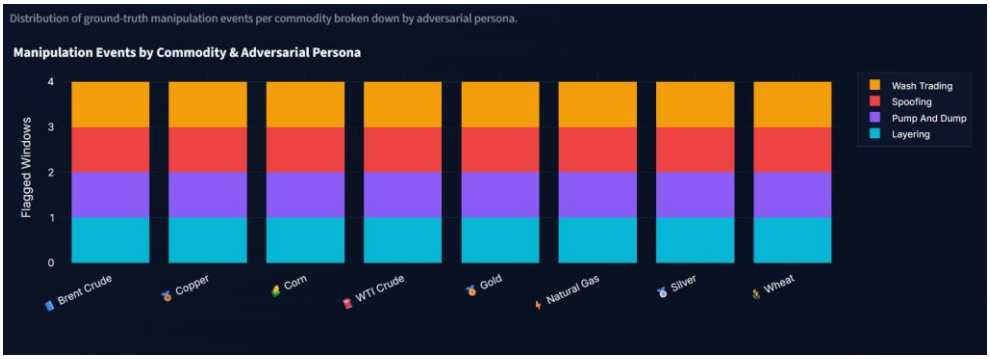


Figure 7.6: Market Overview: flagged windows by persona and commodity

Selecting a single commodity switches to a detailed view (Figure 7.7) that lists the personas active in the market, followed by an OHLCV candlestick chart with each manipulation episode shaded (Figure 7.8), market microstructure series (cancel rate, order-to-trade ratio and buy/sell volume imbalance, Figure 7.9) and a heatmap of the most active accounts over time (Figure 7.10). The heatmap makes the HFT liquidity providers' high activity immediately visible.

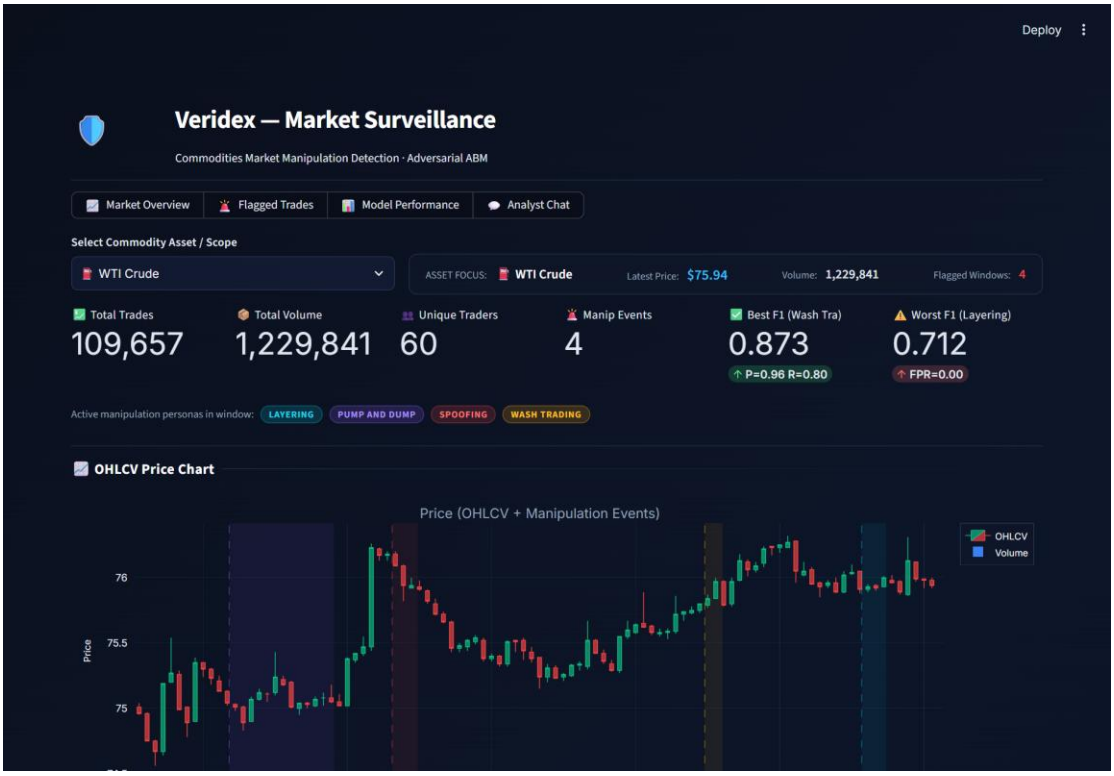


Figure 7.7: Market Overview for WTI crude oil: headline metrics and active personas



Figure 7.8: OHLCV chart of WTI crude oil with manipulation episodes shaded

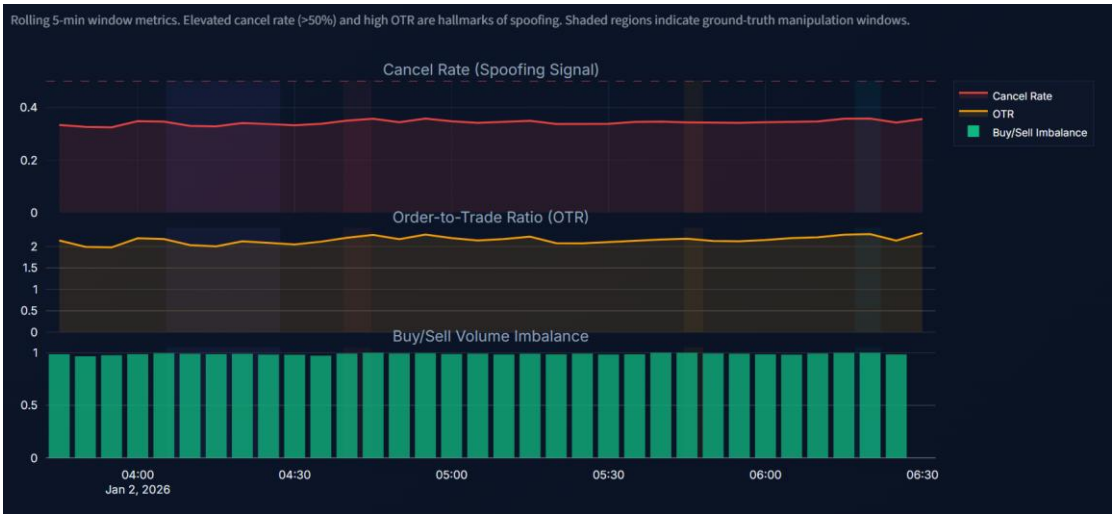


Figure 7.9: Microstructure series for WTI crude oil

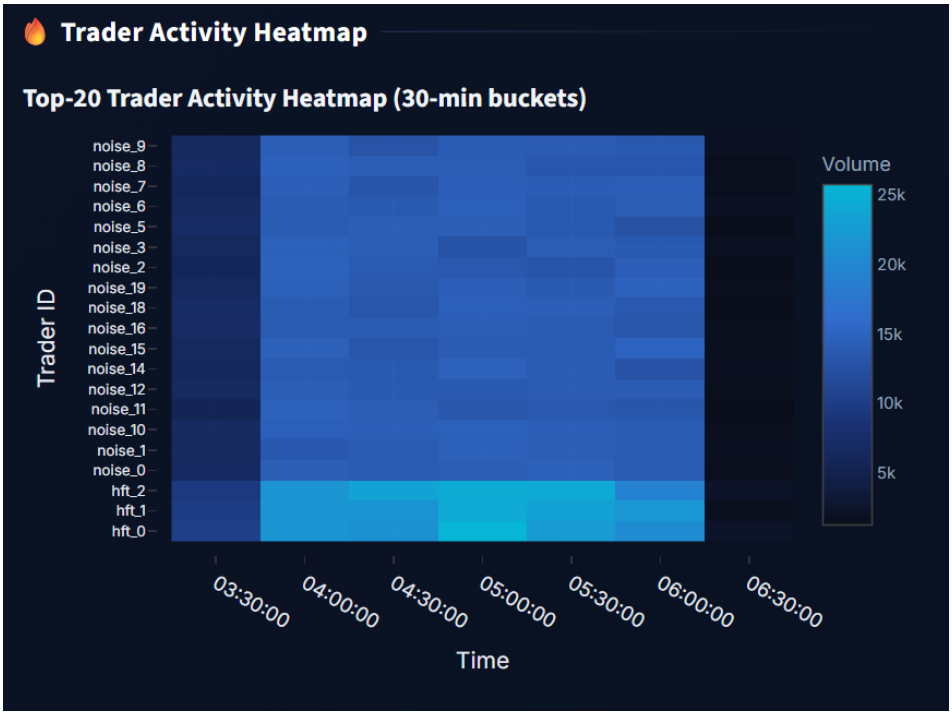


Figure 7.10: Activity heatmap of the most active WTI accounts (30-minute buckets)

7.5.3 FLAGGED TRADES

The Flagged Trades tab is the analyst's case-review screen (Figure 7.11). Its header counts *persona cases* (consecutive flagged windows of one account and persona merged into a case), how many of them the simulator's ground truth confirms, how many have no matching episode, missed episodes, and windows flagged only by the unsupervised layer. Filters select personas, ground-truth status and a minimum score. Each case is an expandable card (Figure 7.12) containing:

- **Detector belief**: persona, account, commodity, number of windows flagged, score, the layers that fired, and the simulator's ground-truth verdict;
- **Prediction confidence**: the persona classifier's probability against its decision threshold, and the probabilities of all four classifiers, each with its own threshold marker;
- **Price around the case**: the market price with the flagged span shaded and the account's own buys and sells marked;
- **SHAP attribution**: a bar chart of the top five contributions and a table giving each feature's plain-language meaning;
- **What the account did**: the evidence table measured from the account's own orders and trades;
- **Case note**: the AI-generated mechanism explanation with its grounding verdict (Figure 7.13), and the account's raw orders and trades in the span (Figure 7.14).

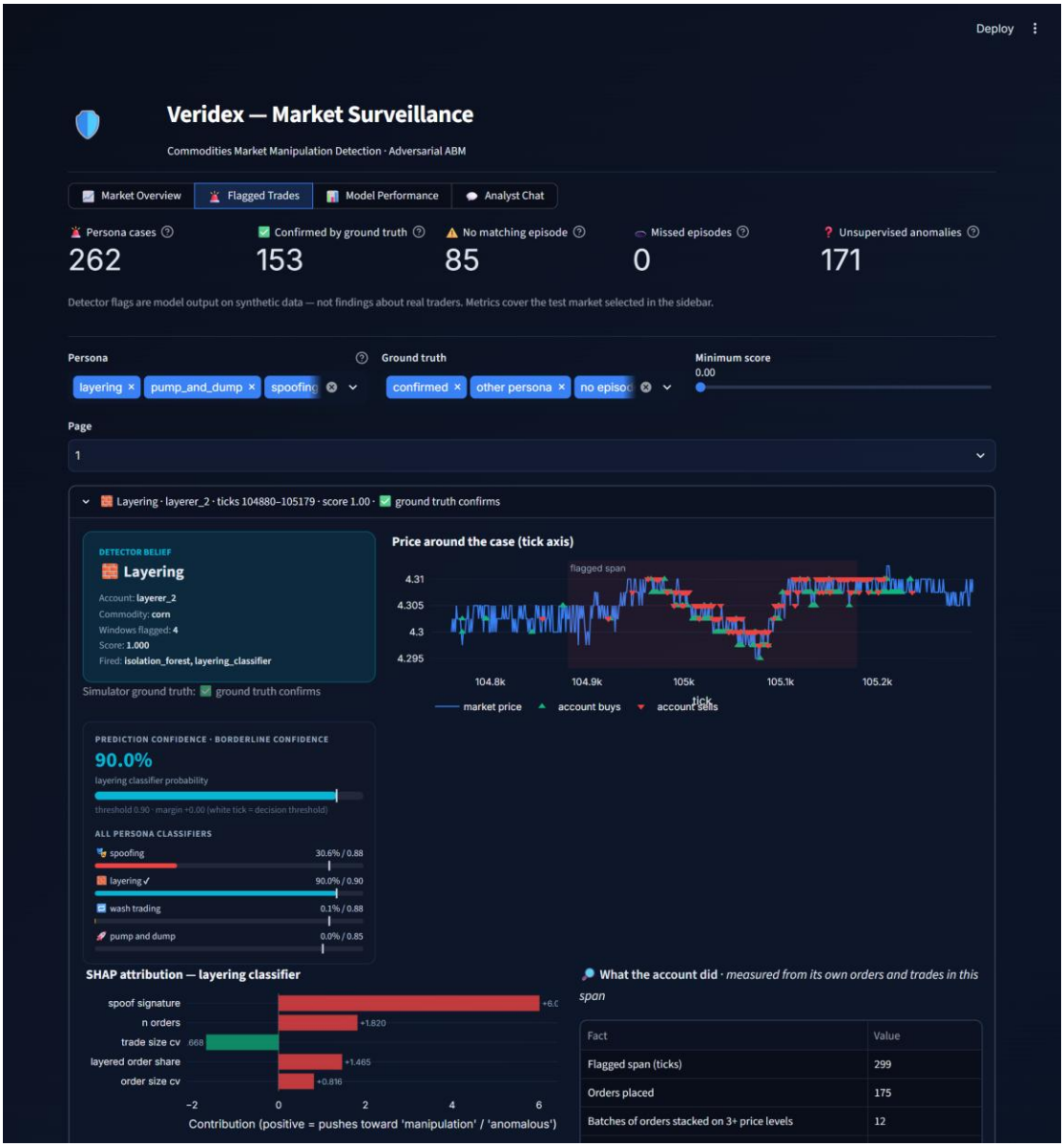


Figure 7.11: Flagged Trades: case counts, filters and the first case (round 5, test market 1)

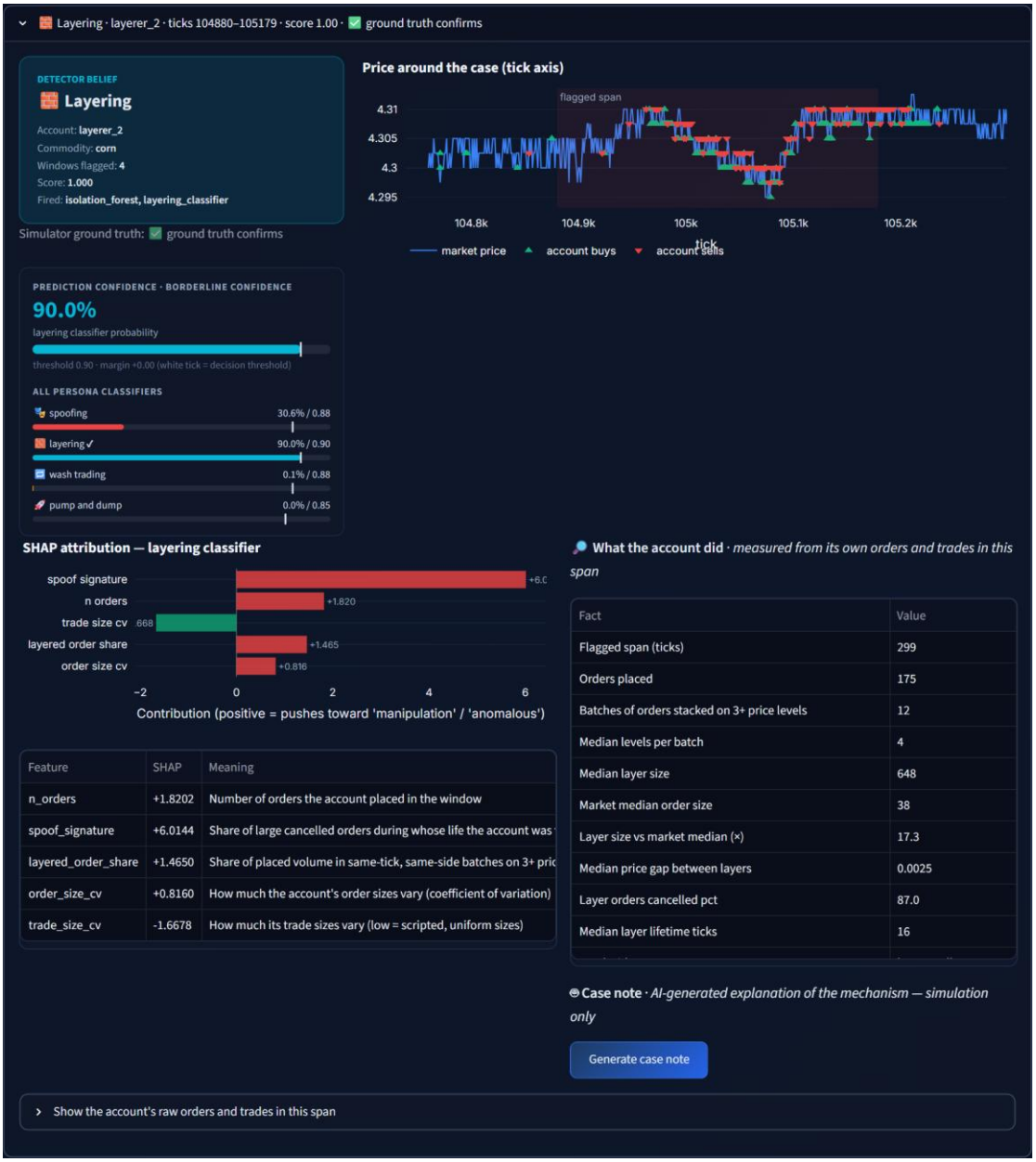


Figure 7.12: A layering case: detector belief, confidence, price, SHAP and evidence

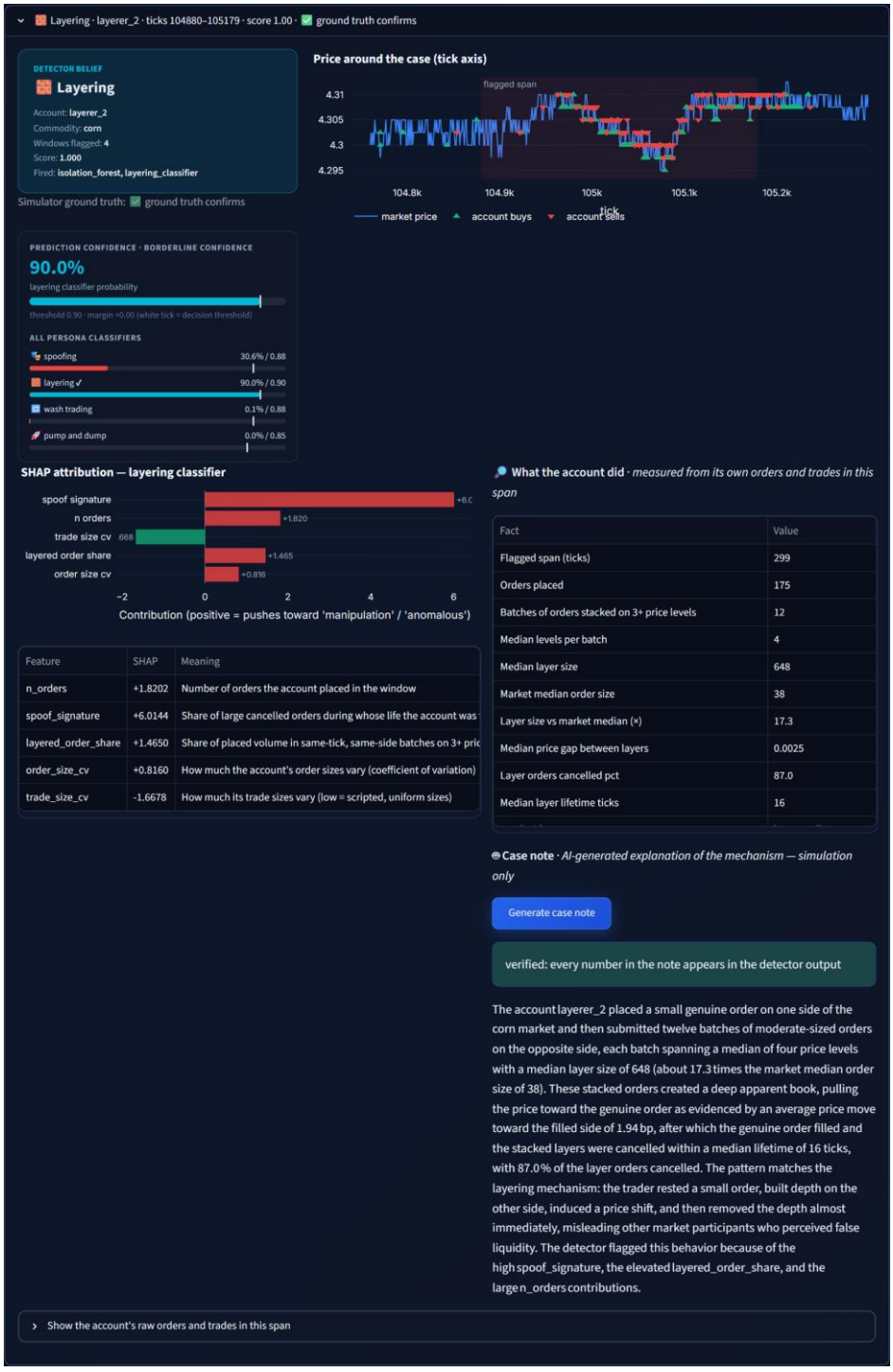


Figure 7.13: The same case with a verified AI case note

The case in Figures 7.12 and 7.13 is account `layerer_2` in the corn market. The layering classifier gave a probability of 0.90, just at its threshold of 0.90; the anomaly layer also fired, and the ground truth confirms the episode. The evidence shows 175 orders placed in the 299-tick span, 12 batches stacked on a median of four price levels, layers 17.3 times the market's median order size, 87% of the layer orders cancelled and a median layer lifetime of 16 ticks. The case note explains the mechanism with these numbers,

and the grounding check confirms that every number appears in the detector output. While preparing these screenshots, a defect was found and fixed: the evidence builder had not loaded the order log for layering cases, so the first version of this note reported zero orders, and the raw-order view did not filter by commodity. Both are corrected in the version shown.

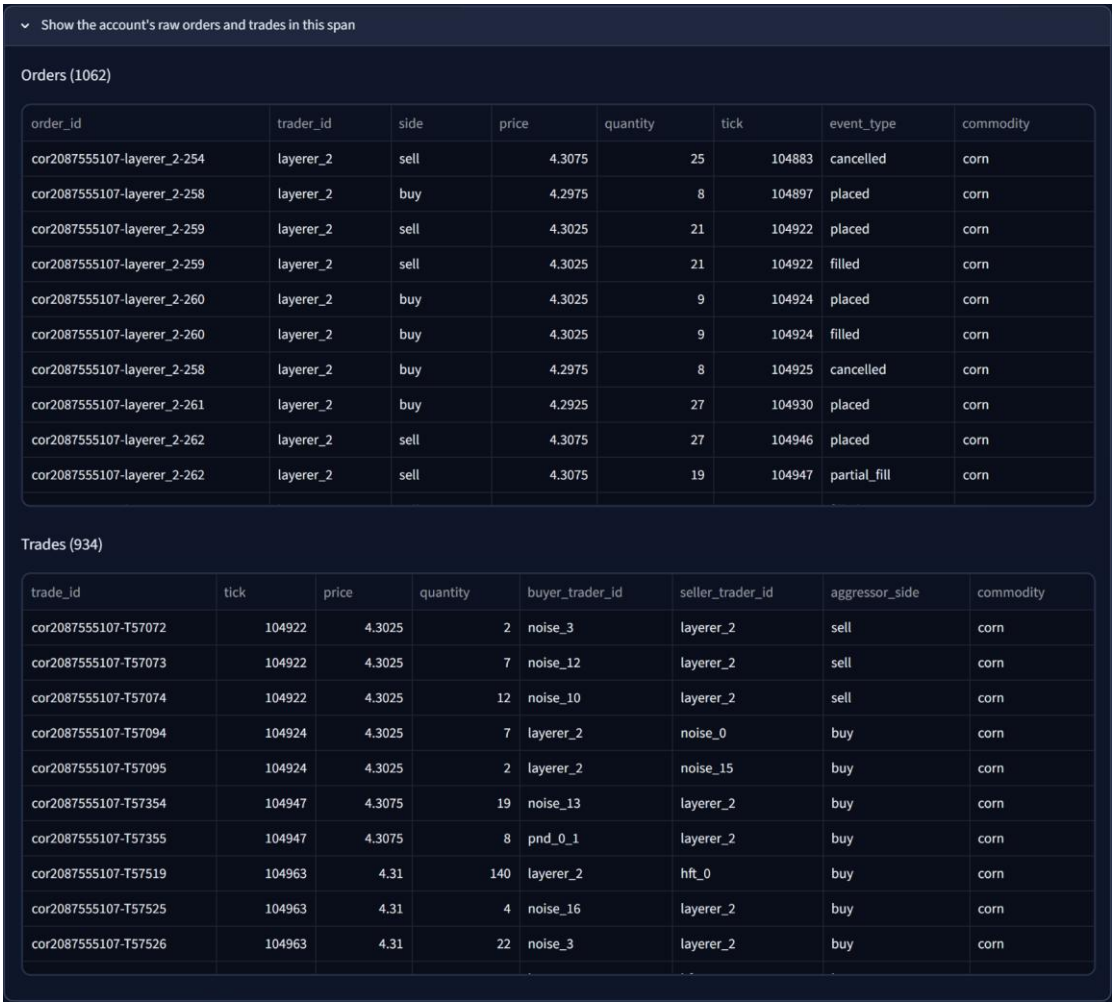


Figure 7.14: Raw orders and trades of the flagged account in the case span

7.5.4 MODEL PERFORMANCE

The Model Performance tab (Figure 7.15) summarises the latest round (average F1, precision, recall and false-positive rate) and opens with an *Evasion vs. Recovery* panel. A sentence generated from the data highlights the largest drop: in round 5, the spoofing classifier's recall fell to 5% against the frozen detector and recovered to 77% after retraining. The chart beneath it plots recall (or F1, on the second tab) for every persona, with dashed lines for the frozen detector and solid lines after retraining (Figure 7.16).



Figure 7.15: Model Performance: headline metrics and the Evasion vs. Recovery panel

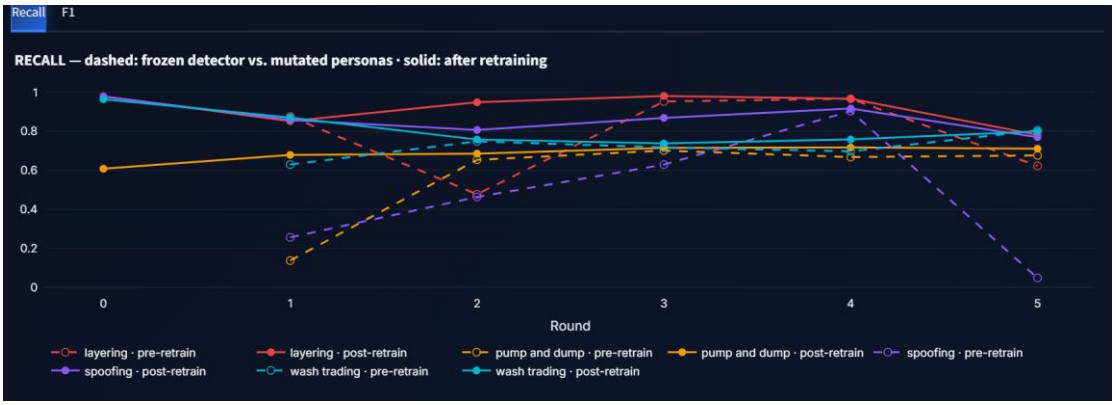


Figure 7.16: Recall of the frozen (dashed) and retrained (solid) detector by round

A performance overview follows: an F1 heatmap of persona against round (Figure 7.17) and a radar chart comparing average precision, recall and F1 in round 0 and the latest round (Figure 7.18). The metric-evolution section has tabs for F1, precision, recall and false-positive rate (Figure 7.19), and a precision–recall scatter places every persona-round with marker size proportional to support, against iso-F1 curves (Figure 7.20).



Figure 7.17: F1 heatmap, persona by round (post-retrain)

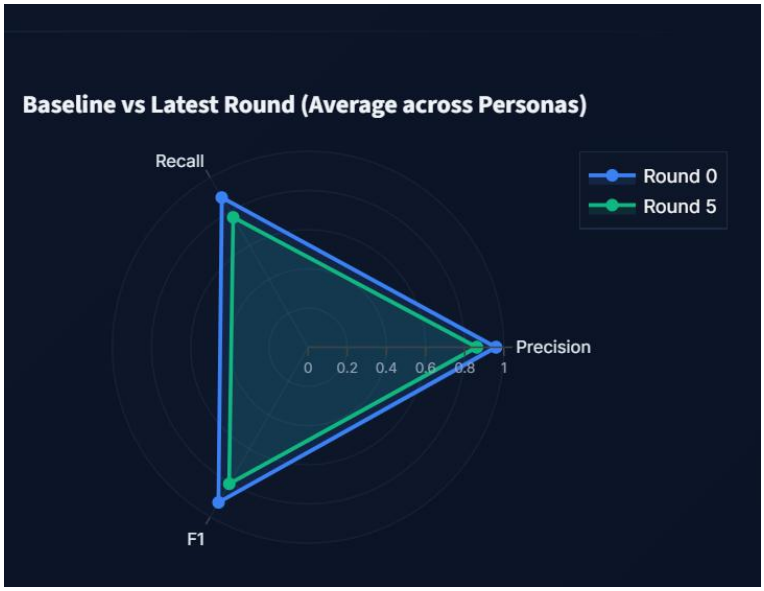


Figure 7.18: Average precision, recall and F1: round 0 against round 5

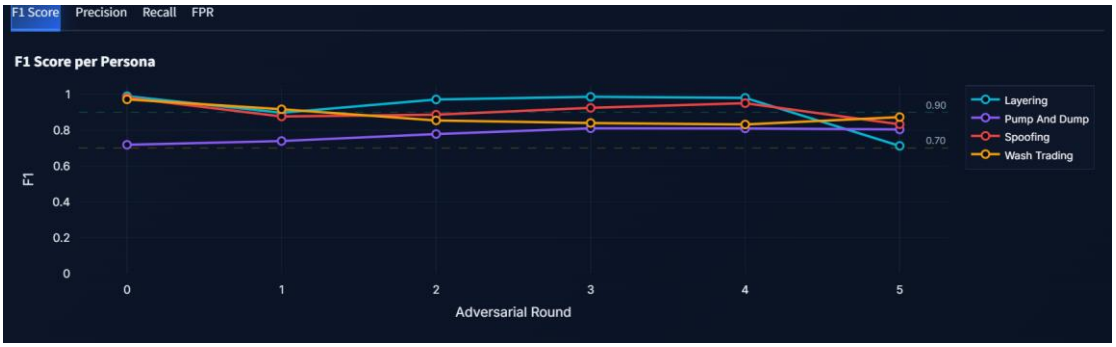


Figure 7.19: Metric evolution: post-retrain F1 per persona across rounds

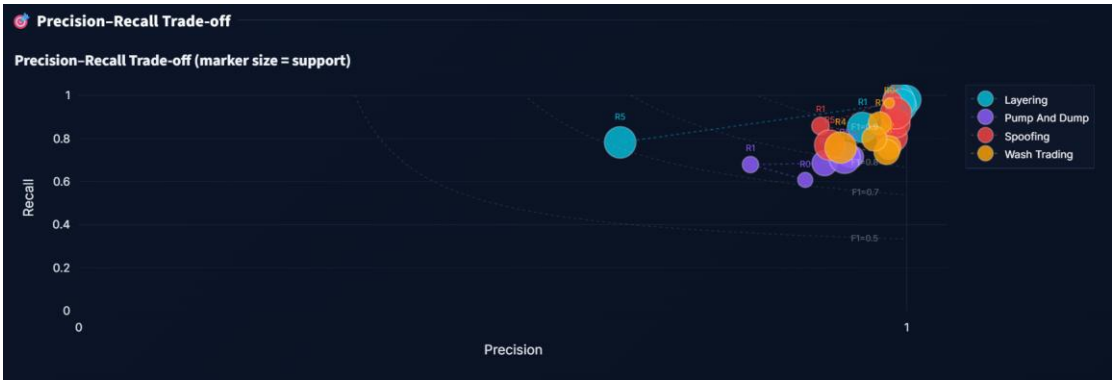


Figure 7.20: Precision–recall trade-off of every persona and round (marker size = support)

Two tables complete the quantitative view: the full metrics table, with pre- and post-retrain rows for every persona and round (Figure 7.21), and the parameter-evolution table showing the parameters each persona actually used in each round (Figure 7.22). Finally, the *Adversarial Strategist Decisions* section shows one card per persona and round (Figure 7.23). Each card states whether the LLM or the hand-written fallback chose the parameters, the frozen detector's any-alert coverage and the mutation score, the own-label coverage when it differs, episodes caught and mean delay, retrained recall, effect relative to round 0, the exact parameter changes, and an expandable panel with the strategist's reasoning.

round	persona	phase	precision	recall	f1	fpr	support	events_total	events_detected	detection_latency_ticks	episode_coverage	effectiveness_ratio
0	Spoofing	post	0.9826	0.9784	0.9805	0.0000	231	16	16	33.5	0.9778	1
0	Wash Trading	post	0.9792	0.9642	0.9716	0.0001	586	16	16	30.6	0.9602	1
0	Pump And Dump	post	0.8776	0.6079	0.7183	0.0004	908	16	16	39	0.6102	1
0	Layering	post	1.0000	0.9796	0.9897	0.0000	196	16	16	32.8	0.9809	1
1	Spoofing	post	0.8960	0.8578	0.8765	0.0001	211	16	16	63.8	0.8364	0.986
1	Wash Trading	post	0.9676	0.8704	0.9164	0.0004	3294	16	16	28	0.8628	0.97
1	Pump And Dump	post	0.8114	0.6789	0.7393	0.0006	1065	16	16	64.1	0.68	1.139
1	Layering	post	0.9468	0.8517	0.8967	0.0000	209	16	16	61.4	0.8404	0.911
1	Spoofing	pre	1.0000	0.2559	0.4075	0.0000	211	16	13	86.5	0.2678	0.986
1	Wash Trading	pre	0.9933	0.6293	0.7705	0.0001	3294	16	16	28	0.6152	0.97

Figure 7.21: Full metrics table (excerpt)

Persona	Parameter	R0	R1	R2	R3	R4	R5
Layering	busy_period_bias	0	0	0	0	0	0.5
Layering	cancel_jitter	0	0.4	0.6	0.7	0.8	0.8
Layering	cancellation_delay_ticks	5	12	12	12	12	20
Layering	cover_rate	0	0	0.2	0.3	0.4	0.5
Layering	cover_size	0	0	0	0	0	0.5
Layering	frequency	0.3	0.3	0.3	0.3	0.3	0.3
Layering	layer_size_multiplier	2	2	2	2	2	2
Layering	layer_spacing	0.0002	0.0002	0.0002	0.0002	0.0002	0.0002
Layering	n_layers	4	4	4	4	4	4
Layering	size_taper	0	0.5	0.8	0.9	0.8	0.8

Figure 7.22: Adversarial parameter evolution table (excerpt)

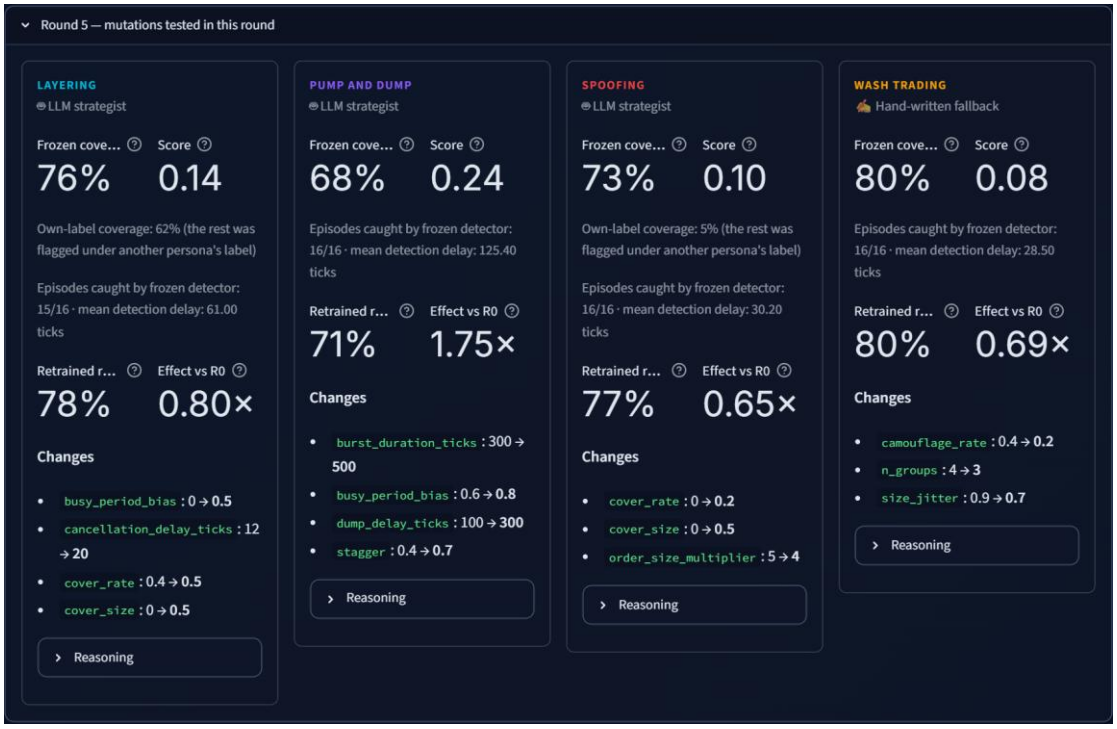


Figure 7.23: Strategist decision cards for round 5

7.5.5 ANALYST CHAT

The Analyst Chat tab offers suggested questions and a chat box. Figure 7.24 shows the assistant answering "What were the post-retrain precision, recall and F1 for each persona in round 5?". It called `query_round_metrics` for round 5, returned a table whose values match `metrics.json` exactly, and listed the tool call under *Sources*.

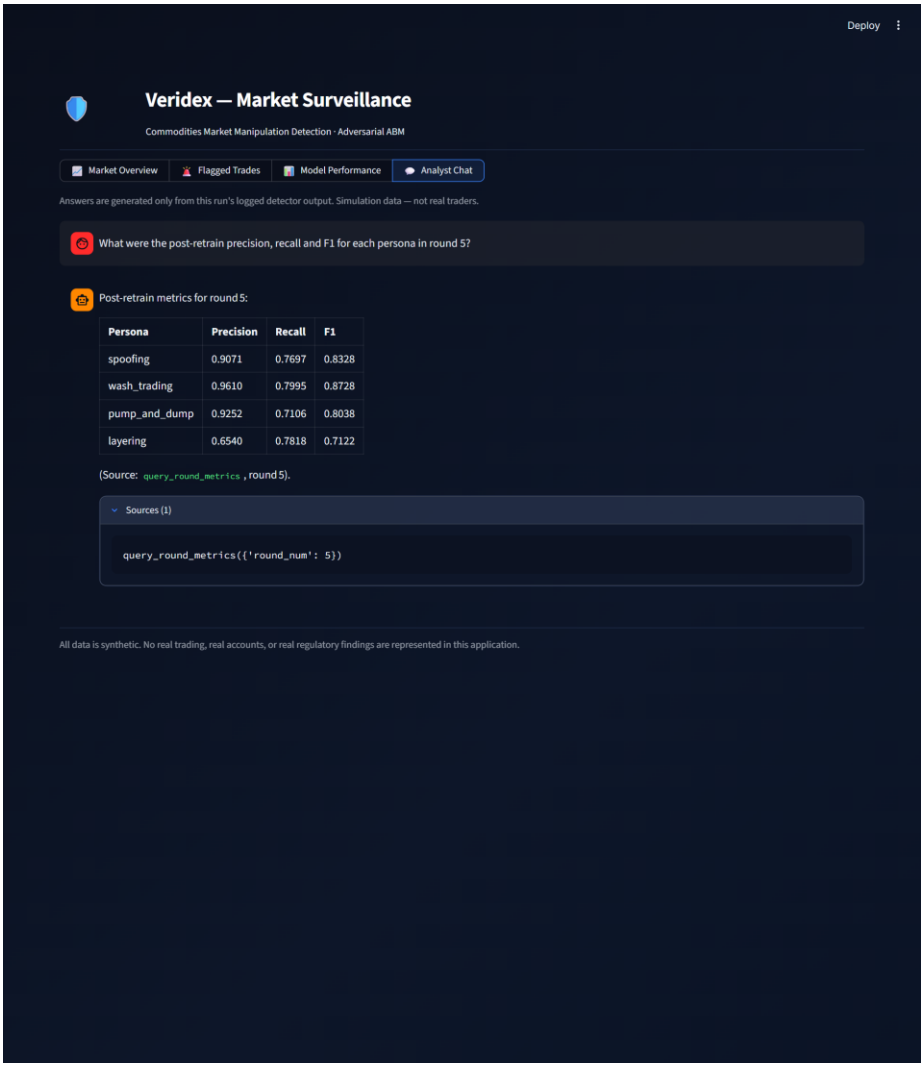


Figure 7.24: A grounded answer with its sources

Chapter 8
Results and Discussion

This chapter reports the results of a complete six-round adversarial experiment, run `adversarial_20261014T151727Z_seed42`, executed with the final version of the system described in Chapters 5 to 7. All numbers are read directly from the run's output files. Unless stated otherwise, *pre-retrain* (or *frozen*) results score a round's mutated personas with the detector trained before that round, and *post-retrain* results use the detector retrained on all rounds up to and including that round.

8.1 EXPERIMENTAL SETUP

Table 8.1: Experimental setup

Item	Value
Run identifier / seed	adversarial_20261014T151727Z_seed42 / 42
Rounds	6 (round 0 baseline, rounds 1–5 adversarial)
Markets per round	1 training market + 2 independent test markets
Commodities per market	8 (WTI, Brent, gold, silver, natural gas, copper, wheat, corn)
Length of each market	10,000 ticks (about 2.8 simulated hours)
Legitimate population per commodity	20 noise, 8 momentum, 6 mean-reversion, 1 market maker, 3 HFT, 3 execution algos
Personas	Spoofing, wash trading, layering, pump-and-dump (one episode of each per commodity and market)
Episodes evaluated per round	16 per persona (8 commodities × 2 test markets)
Detector	3-minute windows, 30-second step; trader Isolation Forest (1%); four XGBoost classifiers; OR ensemble
Strategist	Cerebras gpt-oss-120b, medium reasoning effort, up to 2,500 output tokens and 5 turns per round

Table 8.2: Data volume per round (training and test markets combined)

Round	Order events	Trades	Episodes	Account-window rows
0	12,401,165	2,495,337	96	326,851
1	12,696,246	2,589,925	96	407,371
2	12,820,864	2,636,067	96	438,061
3	12,763,849	2,606,536	96	421,061
4	12,871,072	2,646,646	96	453,621
5	12,815,646	2,629,457	96	422,161

Each round therefore processes about 12.8 million order events and 2.6 million trades. The number of account-window rows grows in later rounds because the strategist added accounts (larger wash rings, more pump-and-dump accounts) and longer episodes.

8.2 BASELINE DETECTION (ROUND 0)

Table 8.3: Round 0 baseline detection (post-retrain, both test markets)

Persona	Precision	Recall	F1	FPR	Episodes detected	Mean latency (ticks)	Coverage
Spoofing	0.983	0.978	0.981	0.0000	16/16	33.5	0.988
Wash trading	0.979	0.964	0.972	0.0001	16/16	30.6	0.960
Layering	1.000	0.980	0.990	0.0000	16/16	32.8	0.985
Pump-and-dump	0.878	0.608	0.718	0.0004	16/16	39.0	0.610

Against the obvious, documented versions of the schemes, the detector is strong for the three order-book schemes: F1 of 0.981 for spoofing, 0.972 for wash trading and 0.990 for layering, with every episode detected within about 31–39 ticks (roughly half a minute) of its start and false-positive rates at or below 0.0001. Pump-and-dump is harder (F1 0.718, recall 0.608): its accumulation phase consists of ordinary passive buying that looks like any other trader, so only the ignition and distribution phases are distinctive. On its own, the persona-agnostic trader Isolation Forest reaches precision 0.104 and recall 0.134 against “any manipulation”. This low value is expected once the HFT and execution-algorithm accounts are present: the forest's 1% budget is largely spent on legitimate accounts that are genuinely unusual. Its role in the ensemble is a safety net for schemes the classifiers have not seen.

8.3 EVASION: FROZEN-DETECTOR RESULTS

Table 8.4 is the central result of the project. For each mutated round it shows how much of each episode the frozen detector still caught (any-alert coverage, the measure used in the adversary's score), the coverage by the persona's own classifier, the number of episodes caught at all and the mean detection delay.

Table 8.4: Frozen (pre-retrain) detection of the mutated personas

Round	Persona	Any-alert coverage	Own-label coverage	Own-label recall	Episodes caught	Mean delay (ticks)
1	Spoofing	0.955	0.268	0.256	16/16	41.2
1	Wash trading	0.619	0.615	0.629	16/16	28.0
1	Layering	0.896	0.878	0.876	16/16	61.4
1	Pump-and-dump	0.137	0.137	0.137	15/16	136.5
2	Spoofing	0.586	0.479	0.463	16/16	53.2
2	Wash trading	0.754	0.750	0.748	16/16	37.6
2	Layering	0.689	0.472	0.477	15/16	98.9
2	Pump-and-dump	0.654	0.654	0.654	16/16	51.9
3	Spoofing	0.652	0.620	0.629	16/16	51.2
3	Wash trading	0.714	0.712	0.717	16/16	35.5
3	Layering	0.952	0.952	0.953	16/16	34.6
3	Pump-and-dump	0.702	0.702	0.701	16/16	64.0
4	Spoofing	0.912	0.900	0.903	16/16	29.3
4	Wash trading	0.699	0.697	0.697	16/16	25.8
4	Layering	0.967	0.967	0.967	16/16	37.8

Round	Persona	Any-alert coverage	Own-label coverage	Own-label recall	Episodes caught	Mean delay (ticks)
4	Pump-and-dump	0.670	0.668	0.668	16/16	74.0
5	Spoofing	0.733	0.045	0.048	16/16	30.2
5	Wash trading	0.804	0.802	0.805	16/16	28.5
5	Layering	0.758	0.616	0.623	15/16	61.0
5	Pump-and-dump	0.679	0.677	0.676	16/16	125.4

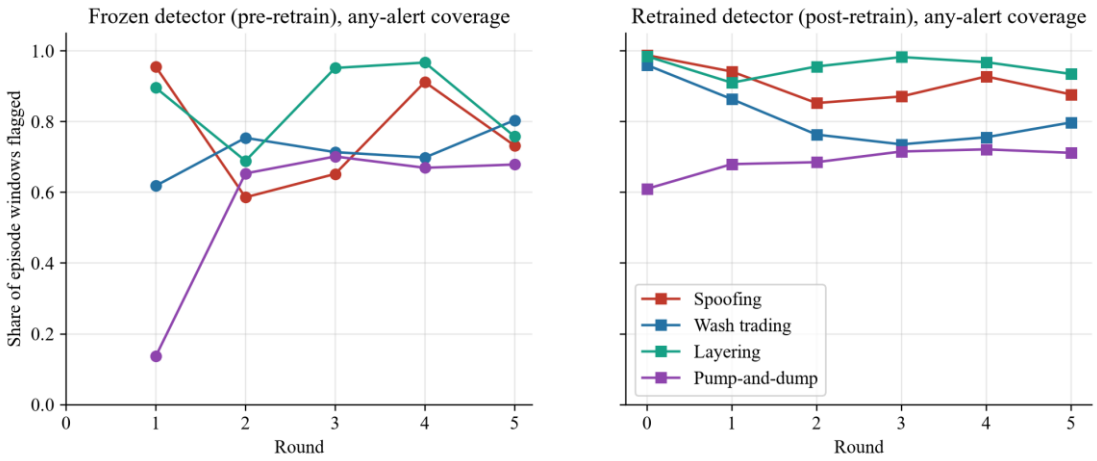


Figure 8.1: Any-alert coverage of the frozen detector (left) and the retrained detector (right)

The LLM's mutations clearly challenged the frozen detector for every persona, at different times:

- **Pump-and-dump** was the most successful evasion. In round 1, busy-period timing, staggered account starts and decoy trades reduced the frozen detector's coverage from about 0.61 at baseline to 0.14, missed one of sixteen episodes entirely and delayed detection of the rest to 137 ticks on average. The detector learned the new pattern after retraining, and later mutations held coverage at about 0.65–0.70.
- **Wash trading** fell to 0.62 coverage in round 1 after the strategist split the ring into three groups of four accounts with randomised sizes, a change that dilutes the counterparty-concentration features without reducing the self-traded volume (effect 0.97×). Coverage stayed between 0.70 and 0.80 in later rounds, which makes wash trading the persona the retrained detector recovered least: its retrained coverage also fell from 0.96 to about 0.74–0.80.
- **Spoofing** dropped to 0.59 coverage in round 2 when the strategist introduced cross-account spoofing (genuine and spoof legs on different accounts), smaller spoofs and more size randomisation. In round 5, adding cover trading produced the most striking own-label result of the experiment: the spoofing classifier's

recall fell to 0.048 (own-label coverage 0.045), yet any-alert coverage remained 0.73 because the windows were flagged under the layering label or by the anomaly layer. Scoring the adversary on any alert is what prevents this relabelling from being counted as an escape.

- **Layering** dropped to 0.69 coverage in round 2 (one episode missed entirely, delay 99 ticks) after the strategist tapered layer sizes and started cover trading. In round 5, heavier cover trading lowered the layering classifier's own-label coverage to 0.62 and its precision to 0.32, as many layering windows resembled spoofing and the classifier began flagging other accounts.

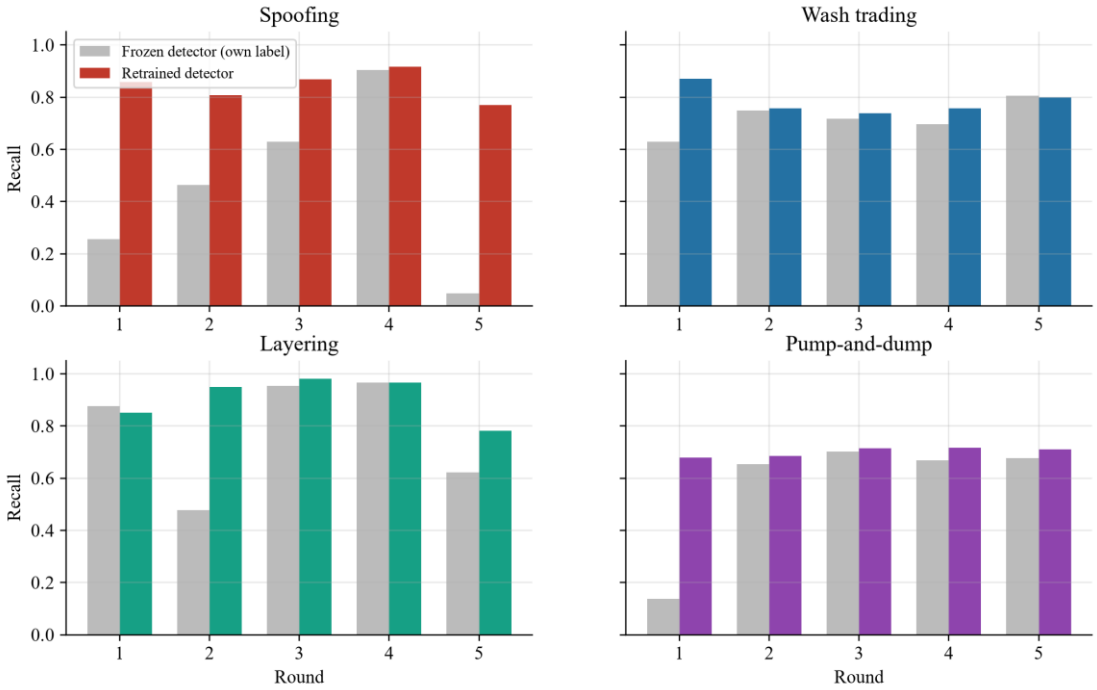


Figure 8.2: Own-label recall before (grey) and after (colour) retraining, by persona and round

Figure 8.2 shows the recovery side. In almost every round the retrained detector restores most of the recall the mutation removed, for example spoofing from 0.46 to 0.81 in round 2 and from 0.05 to 0.77 in round 5, and pump-and-dump from 0.14 to 0.68 in round 1. This is the expected behaviour of a regularly retrained supervised detector: a new tactic is dangerous until the detector has seen examples of it.

8.4 RECOVERY: POST-RETRAIN PERFORMANCE

Table 8.5: Post-retrain F1 by persona and round

Persona	R0	R1	R2	R3	R4	R5
Spoofing	0.981	0.876	0.886	0.924	0.951	0.833

Persona	R0	R1	R2	R3	R4	R5
Wash trading	0.972	0.916	0.854	0.840	0.831	0.873
Layering	0.990	0.897	0.971	0.986	0.980	0.712
Pump-and-dump	0.718	0.739	0.778	0.810	0.810	0.804

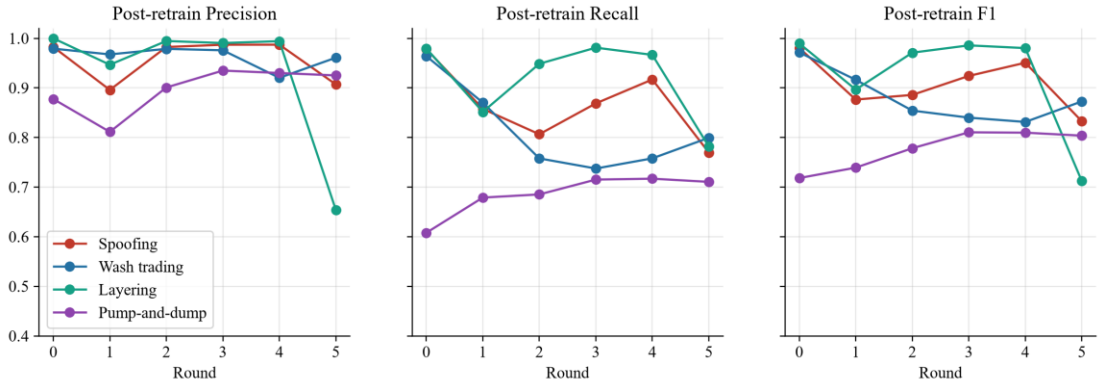


Figure 8.3: Post-retrain precision, recall and F1 by persona and round

Retraining keeps precision high throughout (above 0.89 for spoofing and wash trading in every round, and between 0.81 and 0.94 for pump-and-dump), and most of the variation is in recall. Two trends stand out. Wash trading's F1 declines steadily from 0.97 to about 0.83–0.87 because ring and group mutations make its accounts look less like a single pair. Layering stays at 0.90 to 0.99 until round 5, when heavy cover trading brings it to 0.71, mostly through lower precision. Pump-and-dump improves from 0.72 to about 0.80 as the detector accumulates examples of more varied coalitions. Averaged over the four personas, the final round's F1 is 0.805, compared with 0.915 at baseline.

8.5 MANIPULATION EFFECTIVENESS AND MUTATION SCORES

Table 8.6: Effectiveness ratio and mutation score by round

Round	Spoof eff.	Wash eff.	Layer eff.	P&D eff.	Spoof score	Wash score	Layer score	P&D score
0	1.00×	1.00×	1.00×	1.00×	–	–	–	–
1	0.99×	0.97×	0.91×	1.14×	0.05	0.20	0.08	0.54
2	0.83×	0.99×	0.82×	1.70×	0.20	0.15	0.20	0.21
3	0.62×	0.72×	0.80×	0.96×	0.13	0.12	0.04	0.18
4	0.74×	0.71×	0.80×	1.93×	0.05	0.12	0.03	0.21
5	0.65×	0.69×	0.80×	1.75×	0.10	0.08	0.14	0.24

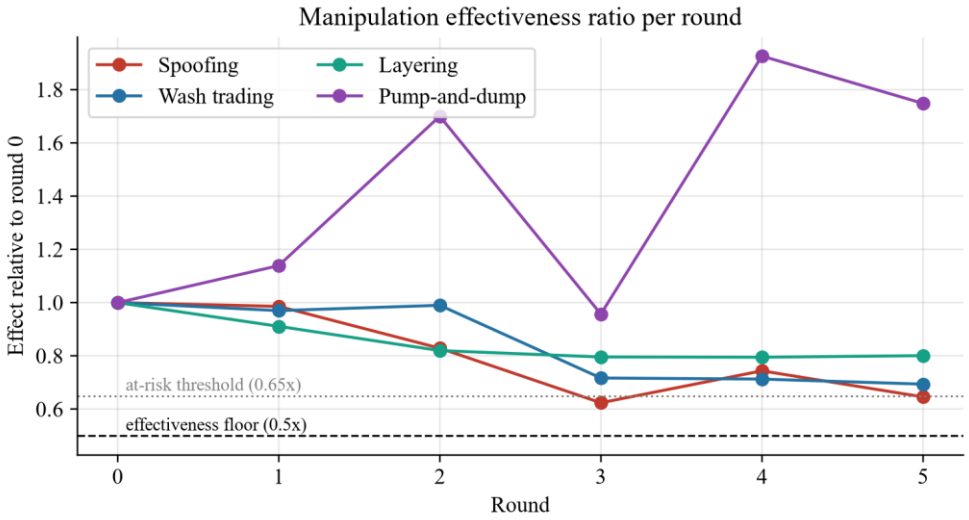


Figure 8.4: Effectiveness ratio of each persona relative to round 0

No mutation fell below the 0.5× floor, so every evasion in Table 8.4 was achieved while the scheme was still working. The lowest effect was spoofing in round 3 (0.62×), when the spoof size had been reduced to four times the level depth; the effect guard flagged it as at risk, and the strategist restored part of the size in round 4. Pump-and-dump's ignition pressure rose above the baseline (up to 1.93×) because the strategist spread the coalition's buying over more accounts and longer bursts that happened to concentrate marketable volume; the objective gives no credit for effect above 1.0×, and the surplus note told the strategist so. The highest mutation score of the run, 0.54, was pump-and-dump in round 1 (Figure 8.5).

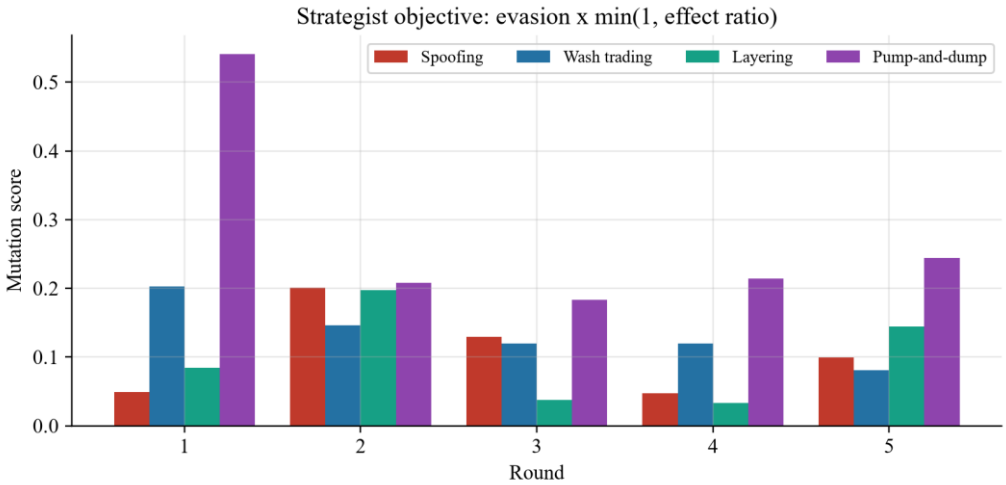


Figure 8.5: Mutation score (the strategist's objective) per persona and round

8.6 WHAT THE STRATEGIST DID

Table 8.7: Parameter changes proposed in each round

Round	Persona	Source	Changes
1	Spoofing	LLM	cancel_jitter 0.0→0.4; cancellation_delay_ticks 5→12; n_split_orders 1→3; size_jitter 0.0→0.4
1	Wash trading	LLM	n_groups 1→3; ring_size 2→4; size_jitter 0.0→0.5
1	Layering	LLM	cancel_jitter 0.0→0.4; cancellation_delay_ticks 5→12; size_taper 0.0→0.5
1	Pump-and-dump	LLM	busy_period_bias 0.0→0.6; decoy_rate 0.0→0.2; stagger 0.0→0.4
2	Spoofing	LLM	n_accounts 1→2; order_size_multiplier 8.0→6.0; size_jitter 0.4→0.6
2	Wash trading	LLM	busy_period_bias 0.0→0.7; camouflage_rate 0.0→0.2; size_jitter 0.5→0.8
2	Layering	LLM	cancel_jitter 0.4→0.6; cover_rate 0.0→0.2; size_taper 0.5→0.8
2	Pump-and-dump	LLM	accumulation_size 150→120; busy_period_bias 0.6→0.8; n_coordinated_accounts 3→6
3	Spoofing	LLM	n_split_orders 3→4; order_size_multiplier 6.0→4.0; size_jitter 0.6→0.8
3	Wash trading	LLM	busy_period_bias 0.7→0.9; camouflage_rate 0.2→0.3; size_jitter 0.8→0.9
3	Layering	LLM	cancel_jitter 0.6→0.7; cover_rate 0.2→0.3; size_taper 0.8→0.9
3	Pump-and-dump	LLM	burst_duration_ticks 200→300; ignition_aggression 0.002→0.001; n_coordinated_accounts 6→4
4	Spoofing	LLM	busy_period_bias 0.0→0.5; cancel_jitter 0.4→0.6; order_size_multiplier 4.0→5.0
4	Wash trading	LLM	camouflage_rate 0.3→0.4; n_groups 3→4
4	Layering	LLM	cancel_jitter 0.7→0.8; cover_rate 0.3→0.4; size_taper 0.9→0.8
4	Pump-and-dump	LLM	busy_period_bias 0.8→0.6; decoy_rate 0.2→0.3; ignition_aggression 0.001→0.0015
5	Spoofing	LLM	cover_rate 0.0→0.2; cover_size 0.0→0.5; order_size_multiplier 5.0→4.0
5	Wash trading	Fallback	camouflage_rate 0.4→0.2; n_groups 4→3; size_jitter 0.9→0.7
5	Layering	LLM	busy_period_bias 0.0→0.5; cancellation_delay_ticks 12→20; cover_rate 0.4→0.5; cover_size 0.0→0.5
5	Pump-and-dump	LLM	burst_duration_ticks 300→500; busy_period_bias 0.6→0.8; dump_delay_ticks 100→300; stagger 0.4→0.7

Table 8.7 lists every mutation. Nineteen of the twenty persona mutations came from the LLM; in round 5 the model wrote its wash-trading proposal as plain text instead of calling the tool, then reported that all personas had been mutated, and after five turns the system fell back to the hand-written library for that persona. Several patterns show that the strategist used its context rather than changing knobs at random:

- It targeted the features the detector relied on. In round 1, with `order_size_cv` and `fill_ratio` the spoofing classifier's most important features, it increased size jitter, cancel jitter, cancel delay and order splitting, noting that the total spoof depth, and therefore the effect, was unchanged.
- It escalated along documented tactics: cross-account spoofing (round 2), smaller spoofs (round 3), busy-period timing (round 4) and cover trading (round 5), each grounded in a tactic card.
- It reacted to the effect guard: after spoofing's effect fell to 0.62× in round 3, its round-4 reasoning began “order_size_multiplier 4.0->5.0: undo prior down move, restores effect”.

- It adopted the new cover-trading knobs for layering from round 2 and for spoofing in round 5, raising `cover_rate` step by step from 0.2 to 0.5 for layering.

The strategist also showed limitations. It tended to push the same knobs further each round (for example layering's `size_taper` and `cancel_jitter`) rather than revisiting knobs it had never tried, such as `reprice_to_touch` or `flip_after_cancel`, and its stated expectations about which features a change would weaken were sometimes wrong. Figure 8.6 shows the round-2 strategist cards in the dashboard.

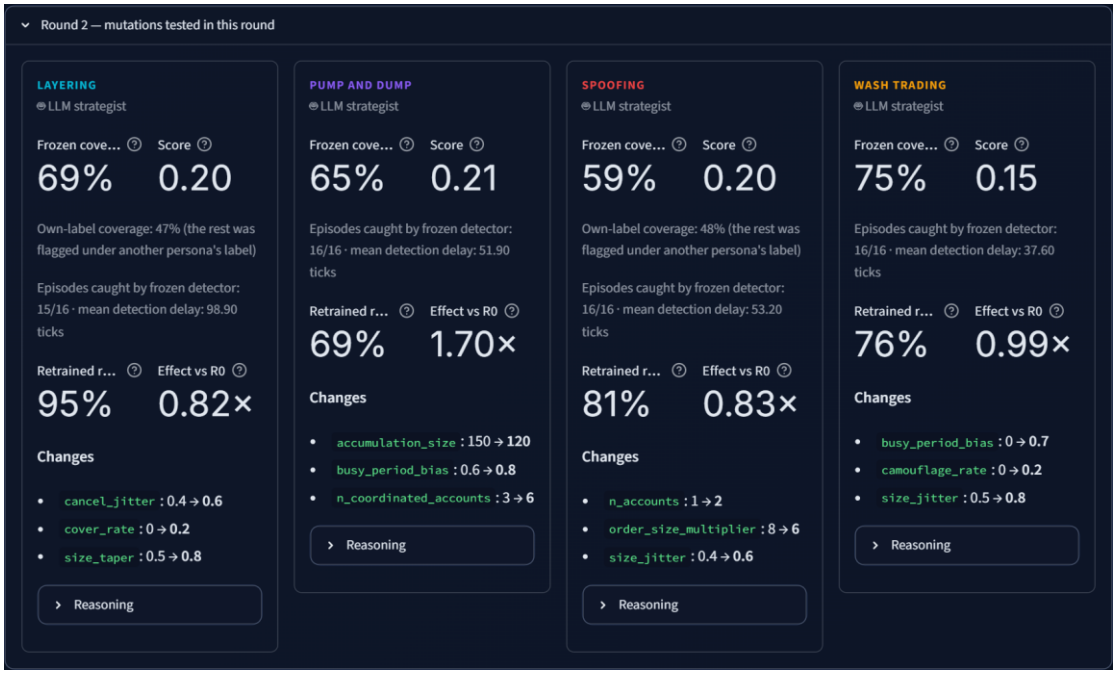


Figure 8.6: Strategist decision cards for round 2 (dashboard)

8.7 WHAT THE DETECTOR RELIES ON

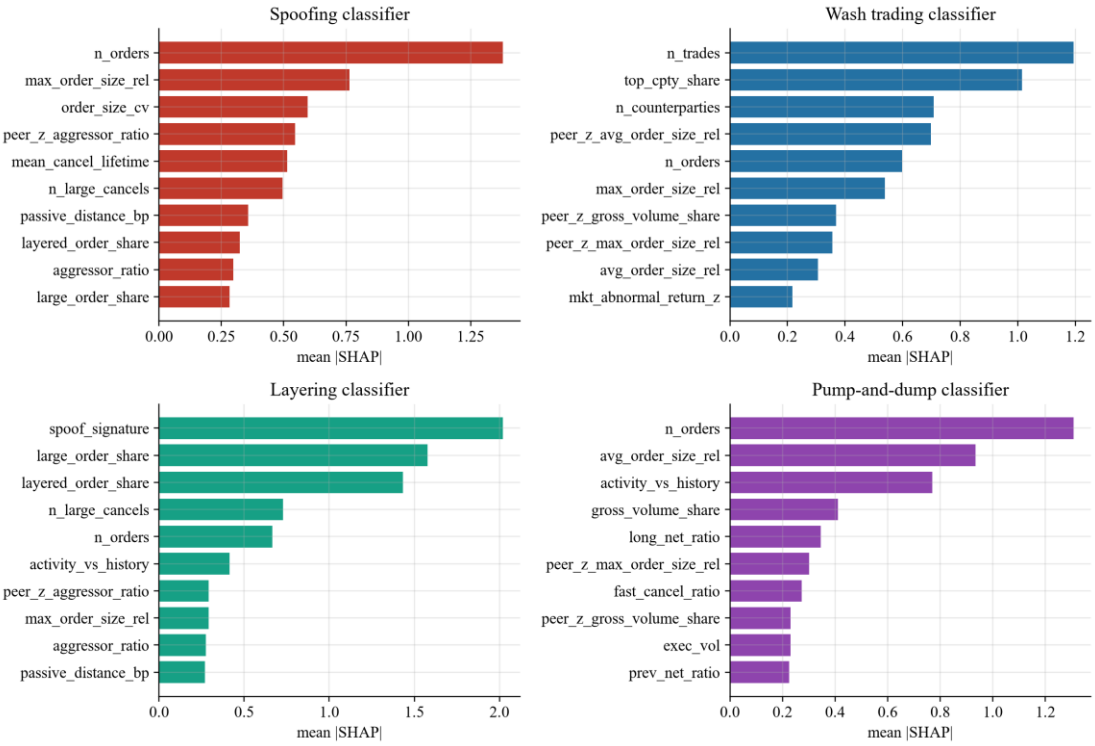


Figure 8.7: Mean absolute SHAP values of the ten most important features of each classifier, round 5

Figure 8.7 shows which features each retrained classifier relied on in the final round. The spoofing classifier relies most on `n_orders`, `max_order_size_rel` and `order_size_cv`: after five rounds of size randomisation, splitting and cover trading, the detector has shifted from "large orders cancelled quickly" to "many orders of very irregular size, including some that are still much larger than normal". The layering classifier relies on `spoof_signature`, `large_order_share` and `layered_order_share`, the direct fingerprints of the scheme. The wash-trading classifier relies on `n_trades`, `top_cpty_share` and `n_counterparties`, and the pump-and-dump classifier on `n_orders`, `avg_order_size_rel` and `activity_vs_history`. This drift of reliance between rounds is itself a useful output: it is passed to the strategist, and it shows an analyst which behaviours the detector currently treats as evidence.

8.8 FALSE ALERTS ON LEGITIMATE LOOK-ALIKE TRADERS

Table 8.8: Share of account-windows flagged, by trader type and round (post-retrain detector)

Trader type	R0	R1	R2	R3	R4	R5
Manipulation windows (detected share)	80.1%	83.3%	74.8%	75.1%	76.7%	77.4%
Execution algorithms	51.7%	51.0%	54.5%	54.0%	60.2%	61.1%

Trader type	R0	R1	R2	R3	R4	R5
HFT liquidity providers	1.2%	0.5%	1.3%	0.7%	0.9%	1.4%
Noise traders	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Momentum traders	0.0%	0.0%	0.0%	0.0%	0.1%	0.1%
Mean-reversion traders	0.0%	0.1%	0.0%	0.1%	0.1%	0.1%
Spoofing accounts outside episodes	0.3%	0.4%	0.1%	0.1%	0.4%	0.4%
Layering accounts outside episodes	0.2%	0.3%	0.3%	0.3%	0.4%	1.0%
Wash accounts outside episodes	0.3%	0.3%	0.2%	0.1%	0.3%	0.2%
Pump-and-dump accounts outside episodes	0.2%	0.3%	0.1%	0.1%	0.3%	0.2%

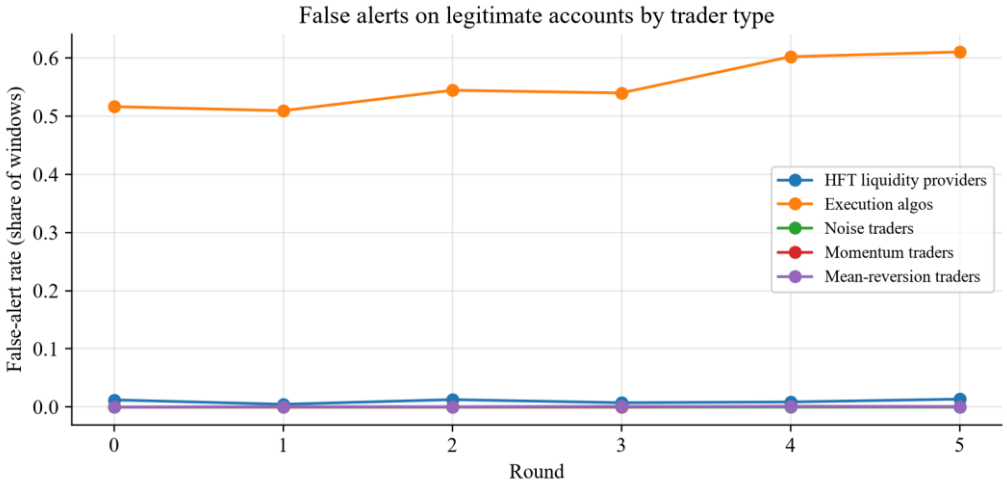


Figure 8.8: False-alert rate on legitimate accounts by trader type

Table 8.8 separates the detector's alerts by the type of account that received them. Ordinary participants are almost never flagged: noise traders 0.0%, momentum and mean-reversion traders 0.1% or less, and manipulator accounts outside their episodes below 1%. The HFT liquidity providers, despite cancelling over 90% of their volume, are flagged in only about 0.5–1.4% of their windows. The execution algorithms, however, are flagged in **52–61%** of their windows, and the share grows over the rounds. This is the most important weakness revealed by the experiment. The algorithms rest and pull orders that are many times the median size, and as the strategist pushed spoofing and layering towards smaller, more irregular orders mixed with genuine trades, the retrained detector came to treat large cancelled orders as suspicious in themselves. In absolute terms the algorithms produce only a few thousand windows per round, compared with over a hundred thousand for noise traders, but in a real surveillance desk this would mean a steady stream of false cases against legitimate institutions. Section 9.2 discusses remedies, such as features that describe whether an account's large orders are ever executed and the relation between its cancellations and its own fills on the other side.

8.9 UNSUPERVISED LAYER AND DETECTION DELAY

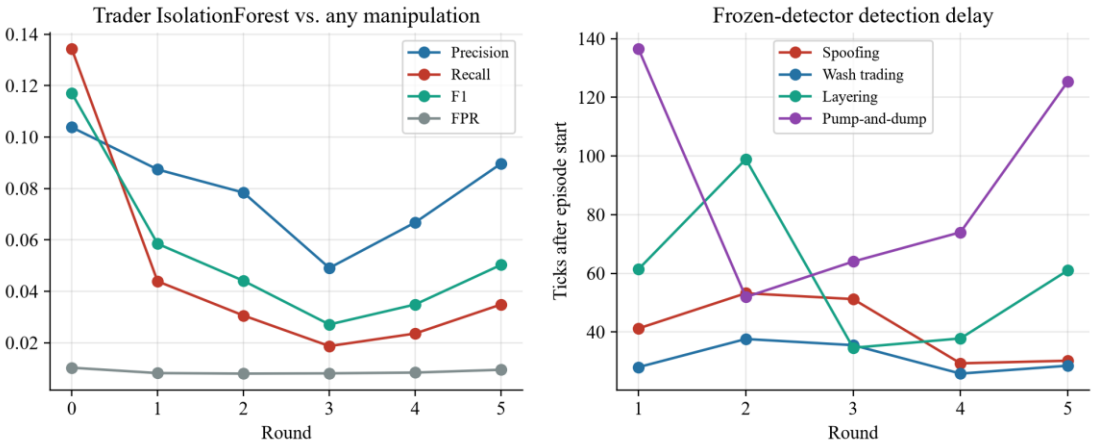


Figure 8.9: Trader Isolation Forest against any manipulation (left); frozen-detector detection delay (right)

The left panel of Figure 8.9 shows that the Isolation Forest's recall against all manipulation falls from 0.13 at baseline to about 0.02–0.04 in later rounds, while its false-positive rate stays near its 1% design budget. As manipulators become less extreme and the legitimate look-alikes remain extreme, the most anomalous 1% of accounts is increasingly made up of legitimate firms. The right panel shows the frozen detector's mean detection delay: for most persona-rounds it lies between about 26 and 75 ticks (layering reached 99 ticks in round 2), and successful pump-and-dump mutations stretch it further, to 137 ticks for pump-and-dump in round 1 and 125 ticks in round 5, when longer bursts and slower exits made the scheme's footprint emerge later.

8.10 LLM USAGE AND COST

Table 8.9: Strategist LLM usage in the experiment

Measure	Value
Strategist calls (5 rounds)	22
Prompt tokens / completion tokens	158,601 / 13,965
Mean prompt size per call	7,209 tokens
Total strategist cost	USD 0.0660
Mean latency per call	1.4 s
Mutations produced by the LLM / by the fallback library	19 / 1
Cumulative spend of the project on this key (all runs)	about USD 0.45 of the USD 4.50 cap

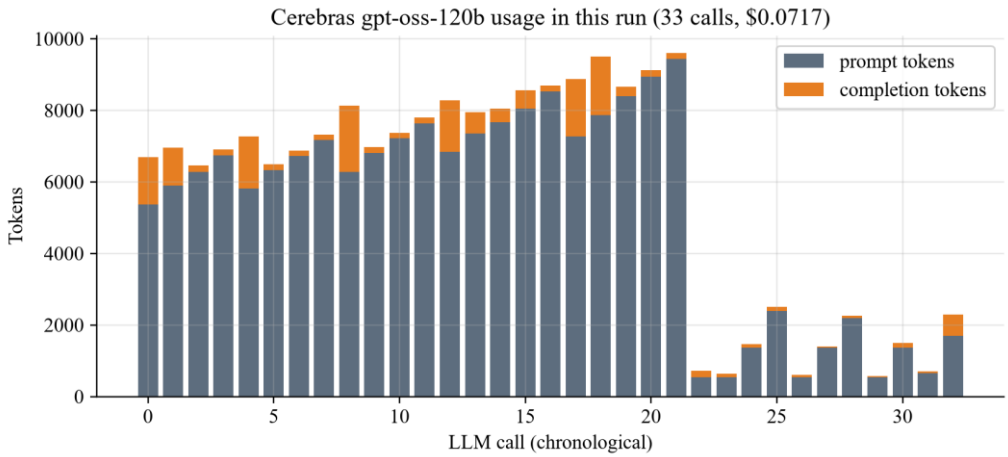


Figure 8.10: Tokens per LLM call in the run (strategist calls, case notes and chat questions)

The strategist used about 7,200 prompt tokens per call on average, most of it the pre-fetched context and the tactic cards, and its replies were short, so a complete five-round adversarial experiment cost well under one US cent per persona per round. The budget ledger, cache and call spacing were never triggered as limits in this run; they exist to make failures cheap rather than to slow normal use. Answer quality was generally good but not perfect. During the dashboard demonstration, asked how "the frozen detector's spoofing recall" changed from round 4 to round 5, the assistant cited the correct tool and the correct round, but reported the post-retrain recall values (0.917 and 0.770) under the label "frozen detector" (Figure 8.11). The numbers were real, so the grounding rule was respected, but the interpretation was wrong, which shows why the dashboard exposes every tool call as a source.

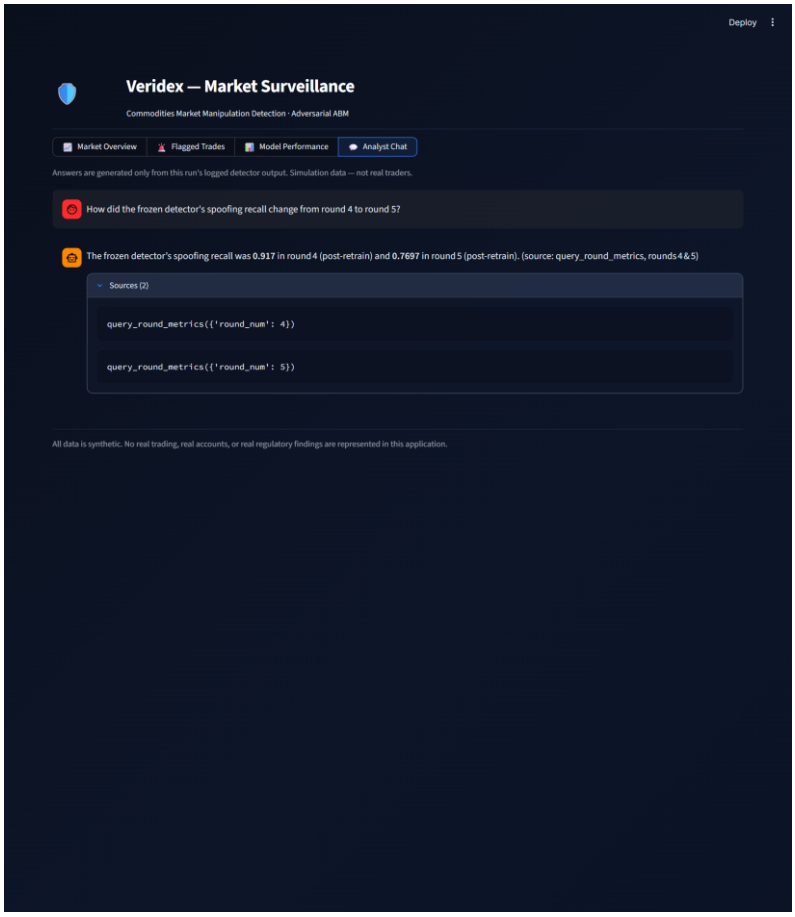


Figure 8.11: An analyst-chat answer that cites real numbers but mislabels them as frozen-detector recall

8.11 SOFTWARE TESTING

Table 8.10: Automated test suite

Suite	What it verifies
Order book (unit and property-based)	Price-time priority, partial fills, cancels, no crossed book, quantity conservation under random order streams
Agents and personas	Each persona's orders, legs and audit tags; parameter mapping; enabled flags; cross-account spoofing; episode merging
Detector	Feature values on hand-built logs, labels, threshold tuning, flags schema, SHAP and persona scores, metrics
Effectiveness and objective	Effectiveness measures, objective formula, any-alert scoring, effect guard (reset, at-risk revert, surplus), cover trading
LLM layer	Provider switch, budget ledger refusal, cache, error classification, strategist validation and fallback, tool outputs
Integration	Byte-identical reproducibility for a fixed seed; a run with no personas enabled
Schemas	Every output file validates against its pandera schema

The suite contains 118 automated tests, all passing on the final code. The tests that involve the LLM use stubbed responses, so the suite never spends API credit.

8.12 DISCUSSION

The experiment supports the main hypothesis of the project: a detector that looks excellent on static data (F1 of 0.97–0.99 for three of the four schemes at baseline) can lose a large part of its coverage as soon as an adaptive adversary changes tactics, here down to 0.14 for pump-and-dump, 0.59 for spoofing, 0.62 for wash trading and 0.69 for layering, while the schemes keep at least 62% of their market effect. Evaluating only the retrained detector would have hidden almost all of this, because retraining restores most of the loss within one round. The pre/post-retrain protocol therefore measures two different and complementary properties: robustness to tactics not yet seen, and the ability to learn new ones.

The experiment also shows why the evaluation details matter. Without the effectiveness constraint, the strategist's easiest route would be to shrink the schemes until they stopped manipulating. Without any-alert scoring, relabelling, for example spoofing flagged as layering, would appear as a near-complete escape (round 5 spoofing would have scored own-label coverage 0.045). And without legitimate look-alike traders, the anomaly layer alone catches every spoofer, so no tactic could ever succeed and the detector's apparent strength would be an artefact of an unrealistically easy market. Finally, the high false-alert rate on execution algorithms is a realistic warning: the same retraining that recovers recall against evolving manipulators can shift the detector towards flagging legitimate large traders, so precision on look-alike populations must be monitored alongside recall.

Chapter 9
Conclusion and Future Work

9.1 CONCLUSION

This project set out to measure how well a market-manipulation detector holds up when manipulators adapt to it, and built Veridex, a complete simulated surveillance laboratory, to do so. Its agent-based simulator produces realistic order books for eight commodity futures with exact manipulation ground truth, and, importantly, contains legitimate high-frequency liquidity providers and execution algorithms whose behaviour overlaps with manipulation. Its two-layer detector combines an Isolation Forest with persona-specific XGBoost classifiers over 51 trader-level features and explains every alert with SHAP values, evidence tables and grounded case notes. Its adversarial loop lets a bounded LLM strategist re-tune the manipulators each round, judges every mutation on both evasion of the frozen detector and remaining market effect, and enforces that judgement in code through any-alert scoring and an effect guard. A Streamlit dashboard makes the whole process inspectable.

The results support the project's central claim. Against the documented baseline schemes, the detector reached F1 scores of 0.97–0.99 for spoofing, wash trading and layering and 0.72 for pump-and-dump. Within one or two rounds, the LLM strategist found mutations that reduced the frozen detector's coverage to 0.14 for pump-and-dump, 0.59 for spoofing, 0.62 for wash trading and 0.69 for layering while keeping at least 62% of each scheme's effect, using documented tactics such as cross-account spoofing, staggered coalitions, ring splitting and cover trading. Cumulative retraining recovered most of each loss, and the final round's post-retrain F1 averaged 0.805. The whole experiment cost about USD 0.07 of API credit.

The project also produced several lessons that were not obvious at the start. An adversary must be required to keep manipulating, or its best strategy is to stop. It must be scored on whether it was caught at all, not on whether it was caught under the right label. A simulator without realistic look-alike traders makes detection artificially easy and adversarial evaluation meaningless. And retraining that restores recall against evolving manipulators can, at the same time, push the detector towards flagging legitimate large traders, which in this experiment reached a false-alert rate of 52–61%

on execution-algorithm windows. Each of these lessons is now built into the system or measured by it.

9.2 LIMITATIONS

- **Single experiment.** The results come from one six-round run with one seed. Effects of the size reported here are large, but confidence intervals would require repeated runs with different seeds.
- **False alerts on execution algorithms.** The detector flags a large share of legitimate execution-algorithm windows, which would be unacceptable in production.
- **Simulator fidelity.** One-minute volatility is up to about twice the calibrated target for copper, silver, wheat and corn, and the market has no cross-commodity linkages, news events or intraday seasonality.
- **Scope of personas.** Cross-market spoofing, cornering, banging the close and collusion across personas are not modelled, and each episode uses one persona at a time.
- **LLM behaviour.** The strategist sometimes repeated the same knobs instead of exploring, once wrote a proposal as text instead of calling its tool, and the analyst assistant once reported correct numbers under the wrong label.
- **Requirements not implemented.** Counterfactual replay and the "price impact avoided by early detection" measure (SRS FR-1.9 and FR-3.5) were not built.

9.3 FUTURE WORK

- **Reduce false alerts on legitimate large traders** with features that separate execution from deception: whether an account's large orders are ever filled, the timing relation between its cancels and its own opposite-side fills, and consistency of direction over the parent-order horizon. Monitoring precision on look-alike populations as a first-class metric would make this trade-off visible every round.
- **Repeated experiments and statistics**: run each configuration with several seeds and report confidence intervals, and compare the LLM strategist with the hand-authored library and with random or Bayesian search over the same knobs.

- **A co-evolving defender**: let a second agent propose detector changes (new features, thresholds or retraining schedules) in response to the strategist, turning the loop into a two-player game.
- **Richer markets and schemes**: correlated commodities and cross-market spoofing, news-driven volatility, opening and closing auctions (for banging the close), and multi-persona collusion.
- **Counterfactual impact**: replay each market from the detection tick without the manipulator to estimate the price impact that early detection would avoid.
- **Better LLM grounding**: give the analyst assistant explicit pre- and post-retrain query parameters, and validate that labels as well as numbers in its answers match the tool output.
- **Validation on real data**: test whether features and models trained in simulation transfer to anonymised exchange data or published enforcement cases.

REFERENCES

[1]. Aggarwal, R. K., & Wu, G. (2006). Stock market manipulations. *Journal of Business, 79*(4), 1915–1953.

[2]. Allen, F., & Gale, D. (1992). Stock-price manipulation. *The Review of Financial Studies, 5*(3), 503–529.

[3]. Avellaneda, M., & Stoikov, S. (2008). High-frequency trading in a limit order book. *Quantitative Finance, 8*(3), 217–224.

[4]. Biggio, B., & Roli, F. (2018). Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition, 84*, 317–331.

[5]. Byrd, D., Hybinette, M., & Balch, T. H. (2020). ABIDES: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation* (pp. 11–22). ACM.

[6]. Cartea, Á., Jaimungal, S., & Wang, Y. (2020). Spoofing and price manipulation in order-driven markets. *Applied Mathematical Finance, 27*(1–2), 67–98.

[7]. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM.

[8]. Chiarella, C., & Iori, G. (2002). A simulation analysis of the microstructure of double auction markets. *Quantitative Finance, 2*(5), 346–353.

[9]. Commodity Futures Trading Commission. (2015). *CFTC charges U.K. resident Navinder Singh Sarao and his company Nav Sarao Futures Limited PLC with price manipulation and spoofing* [Press release]. https://www.cftc.gov

[10]. Commodity Futures Trading Commission. (2019). *CFTC orders Tower Research Capital LLC to pay record $67.4 million for spoofing in the futures markets* [Press release]. https://www.cftc.gov

[11]. Commodity Futures Trading Commission. (2020). *CFTC orders JPMorgan to pay record $920 million for spoofing and manipulation* [Press release]. https://www.cftc.gov

[12]. Commodity Futures Trading Commission. (2021). *CFTC orders Coinbase Inc. to pay $6.5 million for false, misleading, or inaccurate reporting and wash trading* [Press release]. https://www.cftc.gov

[13]. Commodity Futures Trading Commission v. Kraft Foods Group, Inc., & Mondelez Global LLC, No. 15-cv-2881 (N.D. Ill. 2015).

[14]. Commodity Futures Trading Commission v. Oystacher, No. 15-cv-9196 (N.D. Ill. 2015).

[15]. Cong, L. W., Li, X., Tang, K., & Yang, Y. (2023). Crypto wash trading. *Management Science, 69*(11), 6427–6454.

[16]. Cont, R. (2001). Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance, 1*(2), 223–236.

[17]. Deaton, A., & Laroque, G. (1996). Competitive storage and commodity price dynamics. *Journal of Political Economy, 104*(5), 896–923.

[18]. Dodd-Frank Wall Street Reform and Consumer Protection Act, Pub. L. No. 111-203, 124 Stat. 1376 (2010).

[19]. Erb, C. B., & Harvey, C. R. (2006). The strategic and tactical value of commodity futures. *Financial Analysts Journal, 62*(2), 69–97.

[20]. Geman, H. (2005). *Commodities and commodity derivatives: Modeling and pricing for agriculturals, metals and energy*. John Wiley & Sons.

[21]. Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*.

[22]. Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., & Howison, S. D. (2013). Limit order books. *Quantitative Finance, 13*(11), 1709–1742.

[23]. Kamps, J., & Kleinberg, B. (2018). To the moon: Defining and detecting cryptocurrency pump-and-dumps. *Crime Science, 7*(1), Article 18.

[24]. Lee, E. J., Eom, K. S., & Park, K. S. (2013). Microstructure-based manipulation: Strategic behavior and performance of spoofing traders. *Journal of Financial Markets, 16*(2), 227–252.

[25]. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE.

[26]. Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence, 2*(1), 56–67.

[27]. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765–4774).

[28]. OpenAI. (2025). *gpt-oss-120b & gpt-oss-20b model card* (arXiv:2508.10925). arXiv.

[29]. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

[30]. Putniņš, T. J. (2012). Market manipulation: A survey. *Journal of Economic Surveys, 26*(5), 952–967.

[31]. Regulation (EU) No 596/2014 of the European Parliament and of the Council of 16 April 2014 on market abuse (market abuse regulation). (2014). *Official Journal of the European Union, L 173*, 1–61.

[32]. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36*.

[33]. Uhlenbeck, G. E., & Ornstein, L. S. (1930). On the theory of the Brownian motion. *Physical Review, 36*(5), 823–841.

[34]. United States v. Coscia, 866 F.3d 782 (7th Cir. 2017).

[35]. Vasicek, O. (1977). An equilibrium characterization of the term structure. *Journal of Financial Economics, 5*(2), 177–188.

[36]. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

APPENDICES

APPENDIX 1 RUNNING VERIDEX

All commands are run from the repository root on Windows PowerShell, using the project's virtual environment.

```
# 1. Create the environment and install dependencies
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

# 2. Configure the LLM provider (the key is never committed)
#    .env:  LLM_PROVIDER=cerebras
#           CEREBRAS_API_KEY=<your key>

# 3. Run the full adversarial experiment used in Chapter 8
.\.venv\Scripts\python.exe scripts/run_full_pipeline.py --rounds 6 --ticks 10000 --
seed 42 --commodity all

#    Optional: hand-authored mutations only (no API calls)
.\.venv\Scripts\python.exe scripts/run_full_pipeline.py --rounds 6 --mutations hand

# 4. Open the dashboard
.\.venv\Scripts\python.exe -m streamlit run dashboard/app.py

# 5. Run the automated tests
.\.venv\Scripts\python.exe -m pytest tests -q
```

APPENDIX 2 DETECTOR CONFIGURATION

The complete detector configuration used in the experiment (`configs/detector_config.yaml`).

```
# ============================================================================
# configs/detector_config.yaml
#
# Hyperparameters for the two-layer manipulation detector (Module 2).
# Unit of detection: one (commodity, window, trader_id) row.
# ============================================================================

# --- Feature Engine ---
feature_engine:
  window: "3min"           # rolling window length (1 tick = 1 second)
  step: "30s"              # stride between windows
  top_k_traders: 5
  vol_spike_lookback: 10   # windows of history for volume / return baselines
  # Designated liquidity providers are surveilled under separate quoting rules;
  # their requote cancels would otherwise dominate every cancellation feature.
  exclude_trader_prefixes: ["mm_"]

# --- Unsupervised Layer: Isolation Forests ---
isolation_forest:
  contamination: 0.01         # trader-level flag rate (percentile threshold = 1 -
contamination)
  market_contamination: 0.05  # market-regime forest (context feature only)
  n_estimators: 200
  max_samples: "auto"
  random_state: 42

# --- Supervised Layer: per-persona XGBoost ---
xgboost:
  n_estimators: 300
```

```
  max_depth: 4
  min_child_weight: 1
  n_estimators_small: 150
  max_depth_small: 3
  min_child_weight_small: 1
  learning_rate: 0.05
  subsample: 0.8
  colsample_bytree: 0.8
  gamma: 0.5
  reg_alpha: 0.1
  reg_lambda: 2.0
  eval_metric: "logloss"
  max_scale_pos_weight: 50.0   # scale_pos_weight = n_neg / n_pos, capped
  random_state: 42

supervised:
  min_positives: 5             # fewer positive rows → classifier skipped with a warning
  require_all_personas: false  # true → raise InsufficientLabelsError instead (SRS FR-2.3
strict mode)
  cv_folds: 3                  # GroupKFold folds for out-of-fold threshold tuning
  cv_block_ticks: 600          # time-block size for CV groups (keeps overlapping windows
together)
  threshold_bounds: [0.25, 0.9]  # tuned thresholds are kept inside this range (few-episode
robustness)

# --- Ensemble Scorer ---
ensemble:
  combination_rule: "or"       # "or": flag if a classifier OR the IsolationForest fires
  iso_weight: 0.4              # used only if combination_rule = "weighted"
  supervised_weight: 0.6
  supervised_threshold: 0.5    # fallback when a threshold cannot be tuned
  top_shap_features: 5

# --- Personas to train classifiers for ---
personas:
  - spoofing
  - wash_trading
  - pump_and_dump
  - layering
```

APPENDIX 3 LLM CONFIGURATION

The LLM configuration (`configs/llm_config.yaml`): provider switch, token limits and the paid-credit guard.

```
# ============================================================================
# configs/llm_config.yaml — Module 4 (LLM & Agentic Intelligence Layer)
#
# PROVIDER SWITCH: every LLM call in the project goes to ONE provider.
#   active_provider below, or the env var LLM_PROVIDER (wins if set) = cerebras | gemini
#   cerebras → only CEREBRAS_API_KEY is used. gemini → the Gemini key pool.
# Values written as {gemini: x, cerebras: y} differ per provider.
# ============================================================================

active_provider: cerebras

providers:
  cerebras:
    model: "gpt-oss-120b"
    temperature: 0.2
    reasoning_effort: "low"     # gpt-oss bills its reasoning as output tokens; keep it short
    cache_responses: true       # identical requests are answered from data/llm_cache, never
re-billed
  gemini:
    model: "gemini-3.6-flash"
    temperature: 1.0
    cache_responses: true

# --- Paid-credit guard (Cerebras only) ---------------------------------------
# Before every call the worst case (estimated input + max_tokens output) must fit
# in the remaining budget, otherwise no request is sent. Spend is persisted across
```

```
# runs in ledger_file. Delete/edit the ledger only if you top up credit.
cerebras_budget:
  budget_usd: 4.50                # hard stop, leaves headroom under the $5 credit
  input_usd_per_million: 0.35     # gpt-oss-120b pricing
  output_usd_per_million: 0.75
  max_calls_per_process: 60       # a whole pipeline run uses ~1-3 calls per round
  min_seconds_between_calls: 6.0  # stays under the account's requests-per-minute limit
  ledger_file: "data/llm_cache/cerebras_spend.json"

# --- 4A: Narrative generator (one call per button click; 1 retry on grounding failure)
narrative_generator:
  max_tokens: {gemini: 2048, cerebras: 700}   # 4-6 sentence mechanism note
  max_retries: 1
  timeout_seconds: 45

# --- 4B: Adversarial strategist (once per round)
adversarial_strategist:
  max_tokens: {gemini: 4096, cerebras: 2500}
  reasoning_effort: {cerebras: "medium"}  # the one place deeper thinking pays off (~$0.002-
0.004/round)
  max_tool_calls: 5              # model turns per round (4 personas; gpt-oss sometimes
proposes one per turn)
  max_retries: 1
  timeout_seconds: 60

# --- 4C: Analyst assistant (per chat message)
analyst_assistant:
  max_tokens: {gemini: 2048, cerebras: 600}
  max_retries: 1
  timeout_seconds: 30

# --- Audit log: data/runs/<run_id>/llm_calls.jsonl (prompt, response, usage, cost; never API
keys)
```

APPENDIX 4 SAMPLE STRATEGIST EXCHANGE

The first part of the context the strategist received before round 1 of the experiment (spoofing section), followed by its tool call. This call used 5,366 prompt tokens and 1,333 completion tokens (1,183 of them reasoning) and cost USD 0.0029.

Context (excerpt):

```
Last evaluated round: 0. Propose parameters for round 1. Personas: spoofing, wash_trading,
pump_and_dump, layering.

=== spoofing ===
Knobs (bounds; current value):
 - spoofing:
   * order_size_multiplier (float, 1.0..15.0; now 8.0): spoof size vs. average visible level depth
   * cancellation_delay_ticks (int, 1..30; now 5): mean ticks the spoof rests before cancel
   * cancel_jitter (float, 0.0..0.8; now 0.0): cancel delay drawn from delay x U(1-j, 1+j)
   * size_jitter (float, 0.0..0.8; now 0.0): spoof size x U(1-j, 1+j)
   * n_split_orders (int, 1..4; now 1): spoof split into k orders on consecutive ticks
   * n_accounts (int, 1..3; now 1): >=2: genuine and spoof legs from different accounts
   * frequency (float, 0.05..1.0; now 0.4): chance per tick to start a new spoof cycle
   * price_aggressiveness (float, 0.0001..0.01; now 0.0005): distance behind best price; >0.002 loses
impact
   * reprice_to_touch (int, 0..1; now 0): 1 = keep the resting spoof pinned behind the moving best
price (Sarao)
   * flip_after_cancel (float, 0.0..1.0; now 0.0): chance of trading aggressively in the genuine
direction when the
     spoof is pulled (Oystacher)
   * busy_period_bias (float, 0.0..1.0; now 0.0): skip quiet periods; act mainly when the market is
busy (advisory #5)
   * cover_rate (float, 0.0..0.5; now 0.0): per-account chance per tick of a genuine cover order
inside the window
     (Sarao/Coscia traded genuinely too)
   * cover_size (float, 0.0..3.0; now 0.0): cover order size x average level depth; the detector's
ratios are volume-
     weighted, so small cover barely dilutes
```

```
Track record (round [source] params | frozen: coverage, episodes caught, delay | retrained recall |
effect (x round0)
     | score):
 r0 [baseline]
{"busy_period_bias":0.0,"cancel_jitter":0.0,"cancellation_delay_ticks":5,"cover_rate":0.0,"cover_size"
:

0.0,"flip_after_cancel":0.0,"frequency":0.4,"n_accounts":1,"n_split_orders":1,"order_size_multiplier":
8.0,"price_
     aggressiveness":0.0005,"reprice_to_touch":0,"size_jitter":0.0} | cov -, caught -, delay - | 0.98
|
     {"fake_depth_share":0.5946} (1.00x) | score -
Never changed so far: ['order_size_multiplier', 'cancellation_delay_ticks', 'cancel_jitter',
'size_jitter',
     'n_split_orders', 'n_accounts', 'frequency', 'price_aggressiveness', 'reprice_to_touch',
'flip_after_cancel',
     'busy_period_bias', 'cover_rate', 'cover_size']; at a bound now: ['cancel_jitter=0.0 (min)',
'size_jitter=0.0
     (min)', 'n_split_orders=1 (min)', 'n_accounts=1 (min)', 'reprice_to_touch=0 (min)',
'flip_after_cancel=0.0
     (min)', 'busy_period_bias=0.0 (min)', 'cover_rate=0.0 (min)', 'cover_size=0.0 (min)']
Detector now relies on (mean |SHAP|):
{"order_size_cv":1.508,"fill_ratio":0.971,"peer_z_large_order_share":0.505,"max_
     order_size_rel":0.494,"spoof_signature":0.442,"mkt_trade_imbalance":0.406}
Tactic cards:
 - Size randomisation [CFTC 2020 staff advisory #1; Coscia round 2]: knobs {'size_jitter': 'up',
     'order_size_multiplier': 'down'}; weakens max_order_size_rel, large_order_share, order_size_cv;
cost:
     smaller/irregular fake depth moves the book less
 - Cancel-timing randomisation [advisory #2; Coscia "introduce random delays"]: knobs
{'cancel_jitter': 'up',
     'cancellation_delay_ticks': 'up'}; weakens fast_cancel_ratio, mean_cancel_lifetime; cost: longer-
resting spoofs
     risk being filled
 - Split into batches / layers [Coscia evasion; Tower Research layering]: knobs {'n_split_orders': '2-
4'}; weakens
     max_order_size_rel, large_cancel_ratio, n_large_cancels; cost: more cancel events per cycle
 - Cross-account (cross-trader) spoofing [JPMorgan 2020 — one account holds the genuine order, another
spoofs]: knobs
     {'n_accounts': '2-3'}; weakens spoof_signature, cancel_exec_opposition; cost: needs coordination;
each account
     still cancels heavily
 - Reprice to stay behind the touch [Sarao 2015 — orders auto-modified to stay a few ticks from the
best price,
     avoiding execution]: knobs {'reprice_to_touch': 1}; weakens spoof_signature (fewer fills),
passive_distance_bp
     drift; cost: every re-place is a cancel, so orders live ~1 tick — cancel_replace_ratio and
fast_cancel_ratio RISE
     (trades detectability for fewer fills)
 - Flip after cancel [Oystacher 2015 — cancel the spoof, then immediately trade aggressively the other
way]: knobs
     {'flip_after_cancel': 'up'}; weakens fill_ratio, cancel_ratio (more real executions); cost: real
aggressive
     trades (execution cost, visible aggressor flow)
 - Trade in busy periods [CFTC 2020 staff advisory #5 — manipulate when high activity explains the
noise]: knobs
     {'busy_period_bias': 'up'}; weakens activity_vs_history, peer_z features, mkt-relative signals;
cost: fewer
     cycles (less effect) when the market is quiet
 - Spoof further from the touch [generic]: knobs {'price_aggressiveness': 'up'}; weakens
passive_distance_bp; cost:
     STRONG — depth beyond ~0.2% barely moves prices (effectiveness collapses)
 - Hide spoofs inside genuine trading [Sarao (NAV) and Coscia traded genuinely alongside the spoof
orders; the spoofs
     were a minority of their flow]: knobs {'cover_rate': 'up', 'cover_size': 'up'}; weakens
avg_order_size_rel,
     large_order_share, fill_ratio, cancel_ratio, fast_cancel_ratio, aggressor_ratio; cost: inventory
risk; genuine
     resting orders slightly offset the fake depth. Does not hide max_order_size_rel or
spoof_signature. Ratios are
     volume-weighted: cover_size ~1 needed to dilute (measured with cover 0.3 x 1.0: spoofing caught
0.52, layering
     0.65); cover_size near 3 makes the account an outlier again
 - Algo-sized orders [Real execution algorithms rest and chase large child orders (1-8x level depth);
spoofs near that
     size blend in]: knobs {'order_size_multiplier': 'down'}; weakens max_order_size_rel,
avg_order_size_rel,
     large_cancel_ratio vs. look-alike algos; cost: STRONG on effect: measured 4x size + cover 0.2 ->
caught 0.31 but
     effect 0.64x; stay above ~4
```

Tool call returned by the model:

```
propose_mutation(
  {
    "persona": "spoofing",
    "parameters": {
      "size_jitter": 0.4,
      "cancel_jitter": 0.4,
      "n_split_orders": 3,
      "cancellation_delay_ticks": 12
    },
    "reasoning": "1. size_jitter up weakens order_size_cv (high SHAP). 2. cancel_jitter up
weakens fast_cancel_ratio & mean_cancel_lifetime (SHAP). 3. n_split_orders=3 reduces
max_order_size_rel and peer_z_large_order_share. 4. longer cancellation_delay_ticks lowers
fast_cancel_ratio further. 5. Effect impact stays >0.6x as total spoof depth unchanged."
  }
)
```

APPENDIX 5 TACTIC CARDS (SPOOFING)

The spoofing tactic cards given to the strategist (`docs/tactic_cards.yaml`, excerpt).

```
spoofing:
  - tactic: Size randomisation
    source: "CFTC 2020 staff advisory #1; Coscia round 2"
    knobs: {size_jitter: up, order_size_multiplier: down}
    weakens: [max_order_size_rel, large_order_share, order_size_cv]
    cost: smaller/irregular fake depth moves the book less
  - tactic: Cancel-timing randomisation
    source: 'advisory #2; Coscia "introduce random delays"'
    knobs: {cancel_jitter: up, cancellation_delay_ticks: up}
    weakens: [fast_cancel_ratio, mean_cancel_lifetime]
    cost: longer-resting spoofs risk being filled
  - tactic: Split into batches / layers
    source: Coscia evasion; Tower Research layering
    knobs: {n_split_orders: 2-4}
    weakens: [max_order_size_rel, large_cancel_ratio, n_large_cancels]
    cost: more cancel events per cycle
  - tactic: Cross-account (cross-trader) spoofing
    source: JPMorgan 2020 — one account holds the genuine order, another spoofs
    knobs: {n_accounts: 2-3}
    weakens: [spoof_signature, cancel_exec_opposition]
    cost: needs coordination; each account still cancels heavily
  - tactic: Reprice to stay behind the touch
    source: Sarao 2015 — orders auto-modified to stay a few ticks from the best price, avoiding
execution
    knobs: {reprice_to_touch: 1}
    weakens: [spoof_signature (fewer fills), passive_distance_bp drift]
    cost: every re-place is a cancel, so orders live ~1 tick — cancel_replace_ratio and
fast_cancel_ratio RISE (trades
      detectability for fewer fills)
  - tactic: Flip after cancel
    source: Oystacher 2015 — cancel the spoof, then immediately trade aggressively the other way
    knobs: {flip_after_cancel: up}
    weakens: [fill_ratio, cancel_ratio (more real executions)]
    cost: real aggressive trades (execution cost, visible aggressor flow)
  - tactic: Trade in busy periods
    source: "CFTC 2020 staff advisory #5 — manipulate when high activity explains the noise"
    knobs: {busy_period_bias: up}
    weakens: [activity_vs_history, peer_z features, mkt-relative signals]
    cost: fewer cycles (less effect) when the market is quiet
  - tactic: Spoof further from the touch
    source: generic
    knobs: {price_aggressiveness: up}
    weakens: [passive_distance_bp]
    cost: STRONG — depth beyond ~0.2% barely moves prices (effectiveness collapses)
  - tactic: Hide spoofs inside genuine trading
    source: "Sarao (NAV) and Coscia traded genuinely alongside the spoof orders; the spoofs were a
minority of their
      flow"
    knobs: {cover_rate: up, cover_size: up}
    weakens: [avg_order_size_rel, large_order_share, fill_ratio, cancel_ratio, fast_cancel_ratio,
aggressor_ratio]
    cost: "inventory risk; genuine resting orders slightly offset the fake depth. Does not hide
max_order_size_rel or
      spoof_signature. Ratios are volume-weighted: cover_size ~1 needed to dilute (measured with cover
0.3 x 1.0:
      spoofing caught 0.52, layering 0.65); cover_size near 3 makes the account an outlier again"
  - tactic: Algo-sized orders
```

```
    source: "Real execution algorithms rest and chase large child orders (1-8x level depth); spoofs
near that size
      blend in"
    knobs: {order_size_multiplier: down}
    weakens: [max_order_size_rel, avg_order_size_rel, large_cancel_ratio vs. look-alike algos]
    cost: "STRONG on effect: measured 4x size + cover 0.2 -> caught 0.31 but effect 0.64x; stay above
~4"
```

APPENDIX 6 REPOSITORY STRUCTURE

```
Veridex/
  configs/            detector_config.yaml, llm_config.yaml, persona_config.yaml
  dashboard/          app.py, data_loader.py, styles.py, components/ (sidebar, market, flags,
metrics, chat)
  docs/               CHANGES_overhaul.md, enforcement_case_notes.md, tactic_cards.yaml
  proj_details/       SRS.md, Implementation_Plan.md, Commodities_ABM_Project_Plan.md
  scripts/            run_simulation.py, run_full_pipeline.py
  src/
    core/             order_book.py, simulation.py, calibration.py, persona_params.py
      agents/         base, noise, momentum, mean_reversion, market_maker,
hft_liquidity_provider, execution_algo
        manipulation/ common, spoofing, layering, wash_trading, pump_and_dump
    detector/         features.py, anomaly.py, classifiers.py, pipeline.py
    adversarial/      orchestrator.py, metrics.py, effectiveness.py
    agentic/          llm_client.py, adversarial_strategist.py, tools.py, mutation_library.py,
                      narrative_generator.py, case_evidence.py, analyst_assistant.py
    utils/            schemas.py, io.py, logging_config.py
  tests/              unit/, integration/, property/
  data/runs/<run_id>/ round_N/{train,test,test_1}/, metrics.json, flags.parquet,
strategist.json, llm_calls.jsonl
```